Golden: Lightweight Non-Interactive Distributed Key Generation

Benedikt Bünz[1], Kevin Choi[2⋆], and Chelsea Komlo[3]

[1] New York University, Espresso Systems
[2] Seoul National University
[3] University of Waterloo & NEAR One
bb@nyu.edu, kc2853@snu.ac.kr, ckomlo@uwaterloo.ca

Abstract. We present Golden, a non-interactive Distributed Key Generation (DKG) protocol. Golden achieves public verifiability in a lightweight, non-interactive manner, outputting Shamir secret shares of a field element $\mathsf{sk} \in \mathbb{Z}_p$ to all participants, and a public key $\mathsf{PK} = g^{\mathsf{sk}}$ that is a discrete-logarithm commitment to sk.

Golden avoids the overhead of public-key encryption schemes like ElGamal, Pallier, or class groups by using a novel building block: a two-party exponent Verifiable Random Function (two-party eVRF), which may be of independent interest. We present a concretely efficient construction based on the Diffie-Hellman non-interactive key exchange and an efficient zero-knowledge proof. DKG participants use this two-party eVRF in a pairwise manner to generate a one-time pad to encrypt shares, while maintaining public verifiability. Golden can be securely instantiated in the random oracle model over any elliptic curve for which the discrete logarithm assumption holds.

Golden drastically improves efficiency, compared to other non-interactive DKGs. For 50 participants, Golden requires only 223 kb bandwidth and 13.5 seconds of total runtime for each participant, in comparison to ElGamal-based non-interactive DKG, which requires 27.8 MB bandwidth and 40.5 seconds runtime per participant.

⋆ Work partly done while at NYU

Table of Contents

1 Introduction

Distributed key generation (DKG) is a fundamental building block for multi-party applications, as DKGs allow n parties to cooperate to generate a secret key, which all parties have contributed to but no single participant knows, yet any t parties can cooperate to recover for some threshold parameter $t \leq n$. DKGs underpin threshold cryptosystems such as threshold encryption [31, 32], threshold PRFs [50], and threshold signature schemes [9, 56, 59]. Furthermore, many secure distributed applications build upon DKGs, including distributed randomness beacons [19, 24, 55, 58, 72], distributed secret storage applications [73], and consensus protocols [33, 61]. Threshold schemes are currently undergoing increased use in practice, and are the subject of ongoing standardization efforts [13]. As such, real-world considerations, such as reducing round complexity, computational and bandwidth efficiency, and protocol simplicity are of increased importance.

Limiting round complexity is of particular importance, due to the complexity of managing interaction among distributed parties. In a distributed environment, parties may fail to send or receive messages due to network issues. For distributed applications that require either a large number of participants (as in consensus protocols) or frequent share rotation, limiting rounds of participant interaction improves the probability of protocol success. For this reason, while interactive (multi-round) DKGs exist in the literature [45, 46, 48, 56], non-interactive (one-round) DKGs are of particular interest. They allow participants to go offline after posting a single message and do not require stable connections or IP addresses (to compute their key share they still need to receive messages but this can happen at an arbitrary later point). However, in spite of their desirability, non-interactive DKGs incur significant complexity over interactive (multi-round) DKGs, for reasons we explore next.

Public Verifiability Versus Round Complexity. One difference between non-interactive and interactive DKGs is the requirement for *public verifiability*. In particular, in the non-interactive DKG setting, participants must be able to verify the correctness of the protocol for all participants without incurring additional rounds of communication. More specifically, DKGs must prevent against "splitting" attacks, where an adversary might send valid contributions to some parties but invalid contributions (either malformed or vacuous) to other parties, resulting in a denial-of-service attack. Interactive DKGs prevent against such attacks by having participants broadcast complaints if any other player misbehaves. In the non-interactive setting, it must be possible to verify correctness without incurring additional rounds of communication.

Existing non-interactive DKGs in the literature achieve public verifiability by following a similar approach using public-key encryption based on ElGamal [47], Paillier [41], or class groups [18, 52], along with zero-knowledge proofs to prove correctness. However, the use of public-key encryption introduces downsides. In the case of ElGamal encryption, public-key decryption must be done over a uniformly random field element, incurring significant performance overhead. In the

case of Paillier or class-group-based encryption, additional security assumptions are required (if the DKG outputs discrete-logarithm key material).

Alternative approaches to generating keys in a distributed manner exist in the literature, but with the tradeoff of increased round complexity. One approach is to perform a one-time setup using a multi-round protocol, after which participants can generate subsequent distributed keys non-interactively [54]. A second approach is to use private channels to distribute secret shares, while sending public commitments and proofs of correctness over a public broadcast channel [12]. However, the use of private channels prevents public verifiability, and thus necessitates a second consensus round for participants to agree on correctness, as an adversary can simply send valid shares to some participants and invalid shares to other participants.

We give a concrete comparison of our techniques with two case studies in the literature most related to our work, in Section 1.2.

1.1 Our Contribution

In this work, we present Golden[4], a novel non-interactive distributed key generation scheme that does not require public-key encryption, and so offers efficiency improvements over prior non-interactive DKGs in the literature. We show a comparison of Golden and related schemes in the literature in Table 2. We give an asymptotic comparison of the performance of Golden in Table 2, and a concrete comparison to Groth's DKG [47] in Table 1.

Golden uses the standard approach of each participant i generating Shamir secret shares of a random field element $\omega_i \in \mathbb{Z}_p$, and then distributing shares to all other participants. The output secret key of the DKG is simply $\mathsf{sk} = \sum_{j=1}^{n} \omega_j$, and the public key is g^{sk}. Furthermore, at the end of the protocol, each participant holds a secret key share sk_i that is a Shamir secret share of sk, which is the sum of their received shares $\mathsf{sk}_i = \sum_{j=1}^{n} f_j(i)$, such that $f_j(0) = \omega_j$.

The core innovation of Golden is a new technique to achieve public verifiability; i.e., how each participant j transmits $f_j(i)$ to its intended recipient i, such that only participant j learns its value, but all other participants can verify its correctness with respect to the polynomial commitment to f_j.

A Two-Party Exponent Verifiable Random Function (two-party eVRF). As a building block towards Golden, we introduce the notion of a two-party exponent verifiable random function (two-party eVRF) [12]. First formalized by Boneh et al. [12] but used in prior work by Nick et al. [63], an eVRF is simply a verifiable random function (VRF) [34] that outputs a tuple (α, A, π), where α is a pseudorandom value, $A = g^{\alpha}$ is a commitment to α, and π is a zero-knowledge proof that A was correctly generated. In effect, a two-party eVRF combines a non-interactive key exchange (NIKE), with an eVRF under the shared key, in one primitive. Overall, our two-party eVRF allows any two parties with the corresponding public keys to derive the *same* pseudorandom

[4] Silence is golden, and the goal of this work is to minimize communication overhead.

	Golden (optimized)		Groth DKG [47]	
Participants	Communication	Runtime	Communication	Runtime
2	1.7 kb	0.4 s	59.4 kb	3.9 s
10	22 kb	2.4 s	1.3 MB	5.6 s
50	223 kb	13.5 s	27.8 MB	40.5 s
100	699 kb	35.8 s	108.6 MB	130.8 s

Table 1: Benchmarks for the optimized variant of Golden and Groth's DKG [47]. Estimates for Groth's DKG are done using benchmarks and implementation from [52]; estimates for Golden use zkalc [36]. Because the Groth implementation uses BLS12-381, we likewise estimate Golden for this curve. We consider running the DKG in an n-of-n configuration; kb represents kilobytes, and MB represents megabytes. Runtime represents the running time for each participant of performing both Round 0 and Round 1.

output, while ensuring public verifiability of the correctness of this output with a zero-knowledge proof. We provide novel security definitions for our two-party eVRF, which take into account, both the pseudorandomness of the output, as well as the symmetry of the construction. Concretely, our two-party eVRF is only pseudorandom if both input keys are honestly generated. This is sufficient for our DKG application, as we require each party to produce proofs of ownership for their public keys.

We introduce a concrete two-party eVRF construction, based on the Decisional Diffie-Hellman (DDH) assumption. It combines a Diffie-Hellman-based NIKE as a building block to derive a shared secret key, with a single-party eVRF construction using the shared key. In the NIKE, just as in the eVRF, we map group elements to field elements by taking the x-coordinate of the elliptic curve point, making the verifiability proof efficient. In more detail, the two-party eVRF requires as input a message m, a secret key sk_1 (serving as a secret key both in the eVRF sense and in the Diffie-Hellman sense), and a public key PK_2. Generating the pseudorandom output α involves the following (simplified) steps. First, the function finds the Diffie-Hellman shared secret $S \leftarrow \mathsf{PK}_2^{\mathsf{sk}_1}$ and derives the x-coordinate $k \leftarrow S.X$, where "$.X$" indicates the operation to output the x-coordinate. Then, the function derives $T \leftarrow \mathsf{H}(m)^k$ (where H is a random oracle) and derives $\alpha \leftarrow T.X$. The commitment is simply $A \leftarrow g^\alpha$, where g is the generator for a prime-order group, and π is a zero-knowledge proof generated with the Bulletproofs argument system [15]. Verification requires as input the message m, public keys $(\mathsf{PK}_1, \mathsf{PK}_2)$, the commitment A, and the proof π.

The efficiency of our two-party eVRF is bounded by the efficiency to generate and verify π. The Bulletproofs argument system outputs a proof for 3598 constraints (for security parameter $\lambda = 256$), whose computation is dominated by a multi-scalar multiplication (MSM) of size 10794 (i.e. involving 10794 group elements) and two MSMs of size 8192. The proof consists of 29 group elements,

Scheme	Rounds	Bandwidth	Computation	Assumption
Fouque & Stern [41]*	1	$\mathcal{O}(n^2t)$	$\mathcal{O}(n^2t)$	Paillier
ElGamal-DKG/Groth [47]	1	$\mathcal{O}(n^2t\lambda)$	$\mathcal{O}(n^2t\lambda)$	DL/DDH
Cascudo & David [18] Kate et al. [52]	1	$\mathcal{O}(n^2t)$	$\mathcal{O}(n^2t)$	Class Groups
Katz [54]	3 (2 preprocessing)	$\mathcal{O}(n^2t)$	$\mathcal{O}(\binom{n}{t})$	DL
BHLS25-DKG [12]	$2^\dagger$	$\mathcal{O}(n^2t)$	$\mathcal{O}(n^2t)$	DL/DDH
Golden (this work)	1	$\mathcal{O}(n^2t)$	$\mathcal{O}(n^2t)$	DL/DDH

Table 2: Comparison of performance overhead of Golden to prior DKGs that output discrete-log based keys, where the resulting secret key is Shamir secret shared among all participants. Here, n is the total number of parties, t is the threshold, and λ is the security parameter. † indicates that an additional round of communication is required for participants to agree on the output status of the protocol (success or abort). * indicates that the scheme does not output discrete-logarithm key material, but we include for comparison of the scheme's relative round complexity.

---

and the verifier's computation is dominated by an MSM of size 14419. We estimate the performance using zkalc [36] to be 0.3 and 0.1 seconds for proving and verification, respectively. While these are for one output's proof only, we provide more details and multiple batching techniques in Section 5.6.

Distributed Key Generation via Golden. Building on this two-party eVRF, Golden is defined as follows. All parties begin by performing a one-time setup operation, by first generating an identity public key. We require that all parties provide a zero-knowledge proof of the corresponding private key, during an initialization phase.

When performing the DKG, all participants perform the role of a Feldman verifiable secret sharing (VSS) dealer. Specifically, each participant i samples a secret ω_i uniformly at random, and then generates Shamir secret shares $f_i(j)$ of this secret, where f_i is a random polynomial defined by Shamir secret sharing. Additionally, each participant generates a Feldman VSS commitment to this polynomial f_i. Then for all other parties $j \neq i$, party i uses our two-party eVRF to (deterministically) generate a uniformly random output $r_{i,j}$ by evaluating the two-party eVRF on m_i with sk_i and PK_j, as well as a commitment output $R_{i,j} = g^{r_{i,j}}$ and a proof. This output $r_{i,j}$ is then used as a one-time pad to derive the ciphertext $z_{i,j} \leftarrow r_{i,j} + f_i(j)$, a linear encryption of party j's Shamir secret share. Party i then broadcasts to all other parties (1) its VSS commitment to f_i, (2) all ciphertexts $(R_{i,j}, z_{i,j})_{j=1}^n$, and (3) all corresponding two-party eVRF proofs.

After receiving VSS commitments, ciphertexts, and proofs from all other parties, each party first checks that the two-party eVRF proof is correct. Then, they check that the encryptions of all shares are correct, using the two-party eVRF commitment output in conjunction with the VSS commitment. Because this check is a linear operation (as it simply checks a linear relation "in the exponent"), this check can be done efficiently. If all checks are correct, each party simply aggregates their received shares $\mathsf{sk}_i = \sum_{j=1}^{n} f_j(i)$ to derive their output secret key share; public keys are derived using the VSS commitments.

As such, Golden achieves public verifiability through the combination of two correctness checks:

(1) the correctness check that each participant's received secret share $f_i(j)$ is valid, using the ciphertext $z_{i,j} \leftarrow r_{i,j} + f_i(j)$, the two-party eVRF commitment $R_{i,j}$, and the VSS commitment to f_i, and
(2) the correctness check that the ciphertext $z_{i,j}$ is valid and participant j can indeed decrypt it, using the two-party eVRF proof to check $r_{i,j}$ was derived correctly. Because our two-party eVRF uses a NIKE to derive $r_{i,j}$, this check guarantees that either participant i or participant j can evaluate the two-party eVRF to generate $r_{i,j}$.

Performance of Golden and Optimizations. We give an asymptotic comparison of the performance of Golden in Table 2, and a concrete comparison to Groth's DKG [47] in Table 1. Similarly to other DKGs, as the number of parties n grows large, the naive performance of Golden by each party scales quadratically in n, as each party must receive $(n-1)^2$ messages and check the correctness of all parties' contributions, as discussed above. In the unoptimized variant, the dominant computational cost is for each participant to perform $(n-1)$ two-party eVRF evaluate operations, and then verify $(n-1)^2$ two-party eVRF proofs.

However, it is possible to define Golden in an optimized manner, by batching the most expensive computation performed by each party (generating two-party eVRF proofs), and hence performing this costly computation only once. More specifically, each party will generate only one (as opposed to $n-1$) proof that all their outputs are correctly evaluated. When instantiated with Bulletproofs, the prover time still grows quasi-linearly in n ($n/\log(n)$). However, the proof size, and thus the overall communication complexity, now drastically decreases, as each party sends only one proof whose size is essentially independent of n. A proof for 1 statement is of size 1.5 kb, using BLS12-381, and a batch proof for 99 statements is only 2.2 kb. Each party also sends t group elements representing the polynomial that encodes their contribution. For large n and t, this becomes the communication bottleneck. The total receiving communication per participant for a 2-out-of-2 DKG is 1.7 kb and for a 100-out-of-100 DKG it is 699 kb.

In addition to batch proving, Bulletproofs supports batch verification [15], where the cost of checking an MSM can be shared across proofs. Using this, the total runtime for each participant in the DKG application, which includes one proof generation and $n-1$ proof verifications, is estimated in Table 5, and we estimate the total runtime for 100 participants to be less than 36 seconds.

UC Security of Golden. We prove simulation-based security of Golden with respect to the idealized key generation functionality $\mathcal{F}_{\mathsf{KeyGen}}^{\mathsf{B}}$ introduced by Kate et al. [53]. While Katz [54] gives an impossibility result that shows one-round DKGs cannot produce unbiased keys that are from the same distribution as a target single-party key generation algorithm (such as discrete log, in the case of Golden), it is possible for Golden to achieve $\mathcal{F}_{\mathsf{KeyGen}}^{\mathsf{B}}$. The ideal functionality $\mathcal{F}_{\mathsf{KeyGen}}^{\mathsf{B}}$ considers a rushing adversary (one which chooses to speak last in the protocol) that can bias the joint secret key (and honest parties' secret key shares) by some additive bias, but only in a limited way. In particular, the adversary chooses its bias only having seen public commitments from honest parties, as opposed to the secret values directly.

Many DKGs used in practice achieve the $\mathcal{F}_{\mathsf{KeyGen}}^{\mathsf{B}}$ functionality, as the adversary is allowed to observe honest parties' commitments and speak last in the protocol. One such example is the one-round ElGamal-based DKG by Groth [47] (which we discuss more in Section 1.2), as the adversary can view participants' commitments in the clear before publishing its own, the same as Golden. Another example is the two-round Pedersen DKG [67] and variants thereof [56], as all participants publish their VSS commitments in the clear without a commitment round. In each of these cases, a rushing adversary can choose its contribution of the DKG as a function of honest parties' VSS commitments. However, intuitively, this advantage to an adversary is bounded, assuming the VSS commitments are generated within a group where the discrete logarithm problem is hard. Towards this end, it has been shown that Pedersen DKG with proofs of possession [56] is safe to use as a DKG for the FROST threshold signature scheme [5, 29, 56]. Similarly, Groth shows a DKG that can be safely used for threshold BLS [47] (and hence, as the DKG for randomness beacons [58]). We postulate that $\mathcal{F}_{\mathsf{KeyGen}}^{\mathsf{B}}$ can be used to securely build other applications, such as threshold encryption.

Because the Katz [54] impossibility rules out only full security for one-round DKGs, it is possible to use our techniques to define a non-interactive DKG which realizes the idealized notion $\mathcal{F}_{\mathsf{KeyGen}}^{\perp}$ of security of a DKG with aborts. We show one such example in Appendix K. However, this example incurs additional performance and complexity overhead to Golden. It remains an open research question to define a DKG with performance equivalent to Golden, while achieving the notion of $\mathcal{F}_{\mathsf{KeyGen}}^{\perp}$ (security with aborts).

1.2 Related DKG Constructions

To illustrate the limitations of prior work, we highlight two case studies of DKG constructions that are most relevant to our work. Similarly to Golden, in both of the below case studies, each participant plays the role of a Shamir secret sharing dealer, where sk is a Shamir-secret shared element in $\mathbb{Z}_p$. The key difference in the below case studies is how participants arrive at consensus on the correctness of the protocol. The case studies differ on how they choose to distribute $f_j(i)$ such that only player j learns its value, while ensuring all other participants can verify its correctness.

Case Study One: ElGamal-DKG and Range Proofs. One possible approach to building a non-interactive DKG is to use public-key encryption, and then generate zero-knowledge proofs that the encryption was performed correctly. This approach was taken by Groth [47], who used ElGamal encryption and range proofs in a pairing setting, such that the DKG outputs a BLS public key in a pairing group. Groth proves security of the DKG with respect to the threshold BLS signature scheme. However, a DKG following the same approach could be used without pairings, which is why we use the term ElGamal-DKG for broader context.

Recall that to encrypt a message m to a public key PK using ElGamal, the ElGamal ciphertext is the tuple $C = (C_0, C_1)$ such that $C_0 = g^r$ and $C_1 = g^m \cdot \mathsf{PK}^r$ where $r \leftarrow\!\!\$\, \mathbb{Z}_p$ is a random nonce. Decryption requires computing m from $C_1 \cdot C_0^{-\mathsf{sk}}$, which bounds the efficiency of the overall scheme.

In ElGamal-DKG, each party j produces an ElGamal ciphertext that encrypts each party's Shamir share $f_j(k)$ to party k's public key. Each participant then broadcasts all ElGamal ciphertexts, along with zero-knowledge proofs that the encryption was performed correctly. To complete the DKG protocol, each party k must then perform $n-1$ ElGamal decryptions. However, performing ElGamal decryption naively would require each party to find the discrete logarithms of the group elements $g^{f_j(k)}$ for $j \in [n]$, a computationally prohibitive task.

To enable efficient decryption, Groth instead uses a chunking approach, to split the bits representing each secret share into multiple plaintexts of a small enough size. Then, each participant ElGamal encrypts each chunk. Each party then generates a zero-knowledge proof that (1) each chunk falls within a reasonable range, and (2) the chunks correctly combine to the secret key share.

Let c be the number of chunks, and ℓ be the chunk size. The bandwidth overhead of ElGamal-DKG requires each party to send $(n-1) \cdot c$ ElGamal ciphertexts (and one SNARK proof), and receive $c \cdot (n-1)^2$ ElGamal ciphertexts (and $n-1$ SNARK proofs). The computational overhead to each participant is dominated by performing $c \cdot (n-1)$ ElGamal decryptions. As such, there is a fundamental tradeoff in bandwidth and computational overhead relative to the chunk size ℓ with ElGamal-DKG. On one end of the spectrum, for a security parameter of 128, setting $\ell = 1$ will result in each participant sending $128(n-1)$ ElGamal ciphertexts. and receiving $128(n-1)^2$ ElGamal ciphertexts in return. However, while increasing the chunk size ℓ in turn reduces each participant's bandwidth overhead, the computational cost for each participant increases, requiring (roughly) $c \cdot (n-1) \cdot \sqrt{2^\ell}$ operations to perform ElGamal decryption, via Baby-step Giant-step [43].

Case Study Two: BHLS25-DKG, Private Channels, and eVRFs. Closest to Golden is prior work by Boneh et al. [12], who first define the notion of exponent verifiable random functions (eVRFs), and gives a DKG construction that uses exponent VRFs as a building block. We refer to this DKG as BHLS25-DKG. BHLS25-DKG achieves security with aborts, meaning that an adversary can force the protocol to terminate early and hence introduce bias into the output public key. In BHLS25-DKG, participants deterministically generate random coefficients for a secret polynomial using eVRFs, and then distribute Shamir secret shares to

all other participants, which are points on this polynomial. Notably, participants distribute these shares over a private channel, and so avoid the use of public-key encryption. The authors claim that this construction requires only one round of communication, as all parties can verify the correctness of all other parties' commitments to their respective sharing polynomial, via proofs generated by the eVRF. However, BHLS25-DKG in fact requires two rounds of interaction to protect against denial-of-service attacks, due to the use of a private channel to distribute shares. If any participant receives a malformed share, they must communicate this fact to all other participants. More specifically, an adversary may "split" the view of the honest participants, as it might follow the protocol honestly with some of the honest participants, and send invalid shares to the remaining honest participants. To prevent such an attack and to ensure that the output of the protocol is usable (i.e., to ensure that the secret key can in fact be recovered), all honest participants must perform an additional "consistency-checking" round (over a broadcast channel), indicating if the DKG completed successfully or not. As such, BHLS25-DKG requires two synchronous rounds of communication, because the scheme is not publicly verifiable.

2 Additional Related Work

Non-Interactive (One-Round) DKGs. We focus our discussion of related work to schemes which generate key material of the form $\mathsf{sk} \in \mathbb{Z}_p, \mathsf{PK} = g^{\mathsf{sk}}$, where the output to each DKG participant is a Shamir secret share of sk.

Katz [54] gives a non-interactive DKG with two rounds of preprocessing, building on pseudorandom secret sharing by Cramer et al. [28] in the CRS model. However, the complexity of the scheme scales exponentially in n and t. More specifically, pseudorandom secret sharing by Cramer et al. requires each party to maintain additive secret shares with all subsets of possible signers, which requires the size of the secret key maintained by each party to be of size $\binom{n}{t}$.

Publicly Verifiable Secret Sharing (PVSS) is a common building block to non-interactive distributed key generation, where each party in the DKG acts as a Verifiable Secret Sharing (VSS) dealer, but generates in addition a proof or commitment to the secret sharing, such that anyone can verify the correctness of the share, without learning its value. A straightforward PVSS was defined by Schoenmakers [70], employing a variant of ElGamal public-key encryption and Chaum-Pedersen Discrete Logarithm Equality (DLEQ) proofs [22]. The resulting secret key $\mathsf{sk}_k \in \mathbb{G}$ for each participant k is the group element $\mathsf{sk}_k = g^{\sum_{i=1}^{n} f_i(k)}$, where $f_j(k) \in \mathbb{Z}_p$ is the Shamir secret share for party k issued by party j, and g is the generator for a prime-order group $\mathbb{G}$. While DKGs that build on this blueprint by Schoenmakers [19] sidestep the need for public-key decryption, the resulting secret key is an element of $\mathbb{G}$, and hence is incompatible for alternative parameter choices, such as discrete-logarithm schemes over elliptic curve groups, where secret keys are in the domain $\mathbb{Z}_p$.

Kate et al. [52] and Cascudo and David [18] define non-interactive DKGs from class groups, employing a similar public-key encryption approach as Groth [47],

but with improved decryption efficiency (as they leverage a subgroup where discrete logarithm is easy). However, these require additional security assumptions, whereas Golden requires only the discrete logarithm assumption and DDH.

Fouque and Stern [41] describe a one-round DKG that builds on the Paillier cryptosystem [65]. While this approach can be employed to generate discrete-logarithm key material, Golden is simpler by requiring only elliptic-curve groups.

Finally, we note there are recent batch optimizations to range proofs in the forms of MissileProof [44] and DekartProof [10], which may be applied to a scheme like Groth's DKG so that the size of range proof for n receivers is constant or scales logarithmically instead of linearly. However, these proofs require a structured reference string (via trusted setup) to achieve the specified efficiency, which in practice requires an involved multi-party operation to generate and stronger trust assumptions [64, 74].

Synchronous Interactive (Multi-Round) DKGs. We next review a range of multi-round DKGs. However, we limit our review to synchronous DKGs, as the techniques employed in synchronous DKGs can be extended to the asynchronous settings. We refer to a systematization of knowledge by Bacho and Kavousi [3] for a review of asynchronous DKGs.

Pedersen-DKG [67] builds upon Feldman Verifiable Secret Sharing (VSS) [37], such that each party plays the role of the dealer. The protocol requires all parties to publicly broadcast their Feldman VSS polynomial commitment, and send each party their respective secret share over a private channel. At the end of the protocol, each party aggregates the secret shares received from all other parties, to determine their output secret key share. The public key is likewise the aggregation of the commitments to the secrets shared by each participant. To prevent a rogue-key attack, a zero-knowledge proof of knowledge of each party's contribution to the DKG is required (as in the PedPop DKG defined by Komlo and Goldberg [56]). Furthermore, because the DKG does not allow for public verifiability, an additional consistency checking round is likewise required [69]. The protocol achieves security with aborts, and so does not protect against an adversary which forces the protocol to terminate early, thereby introducing bias into the output key.

Gurkan et al. [48] show that Pedersen-DKG can be used securely in any setting where the cryptographic scheme is "rekeyable," such as in pairing-based groups. However, outside of such settings, it is possible for a malicious party to perform rogue-key attacks [6, 11].

Gennaro et al. [46] present a multi-round (unbiasable) DKG construction that ensures robustness against up to $t - 1$ players which may abort or deviate from the protocol. The DKG uses Pedersen commitments and verifiable secret sharing to distribute blinded commitments, and, in a subsequent round, opens the commitments after all participants have verified the integrity of their shares. The blinded commitments allow the scheme to ensure protection against a rushing adversary, which might otherwise perform a biasing attack, by waiting to receive contributions from all honest participants before selecting contributions for dishonest parties.

Cascudo et al. [20] define a multi-round (unbiasable) DKG that builds upon the ElGamal-based PVSS, similarly to Fouque and Stern [41]. However, to avoid using Paillier encryption, the authors rely on a commit-and-prove strategy, where participants rely on the additively homomorphic property of ElGamal to demonstrate knowledge of their (aggregated) public key share.

Feng et al. [38] define a multi-round (unbiasable) DKG that builds upon the SCRAPE [19] technique for verifiable secret sharing, as well as ElGamal encryption.

Several synchronous DKGs have been introduced with adaptive security [1, 16, 38, 51], but require multiple rounds of interaction. For this work, we target static security. It is an interesting open question how our DKG can be proven secure against an adaptive adversary, following recent impossibility results for threshold signatures with public keys with a unique correspondence to secret keys (as is the case for keys generated by Golden) [30].

Bacho and Kavousi [3] give a systematization of knowledge for discrete-logarithm key generation schemes, comparing on differences such as number of rounds, synchronicity, and communication overhead. The authors focus on DKGs that satisfy the ideal functionality of Katz [54], for a fully secure DKG, which outputs key material that is indistinguishable from key material generation by a single-party key generation algorithm. However, our work targets one-round DKGs which cannot satisfy this ideal functionality [54], and instead must satisfy a slightly weaker notion, which we give in Figure 1.

Concurrent Work. Parker and Madrigal [60] gave a description of how to move the two-round DKG by Boneh et al. [12] to the non-interactive setting using a similar technique to Golden, by using an eVRF to generate a pairwise shared secret among DKG participants. Note however the proof of security given by Madrigal does not hold, as in order to show a reduction to the hardness of the discrete logarithm problem, the reduction requires executing a non-PPT algorithm A' that can solve for arbitrary discrete logarithm values, when provided with the transcript from the DKG and the state from a DKG adversary A. We show a variant of the DKG given by Parker and Madrigal in Appendix K (which uses our two-party eVRF T-eVRF-DH for encrypting shares), and give a proof of its security under abort.

3 Preliminaries

General Notation and Definitions. Let $\lambda \in \mathbb{N}$ denote the security parameter and 1^λ its unary representation. A function $f : \mathbb{N} \to \mathbb{R}$ is called *negligible* if for all $c \in \mathbb{R}, c > 0$, there exists $k_0 \in \mathbb{N}$ such that $|f(k)| < \frac{1}{k^c}$ for all $k \in \mathbb{N}, k \geq k_0$. For a non-empty set S, let $x \leftarrow\!\!\$\, S$ denote sampling an element of S uniformly at random and assigning it to x. We use $[n]$ to represent the set $\{1, \ldots, n\}$ and $[0..n]$ to represent the set $\{0, \ldots, n\}$. Additionally, $\mathsf{Funs}_{\mathcal{X},\mathcal{Y}}$ denotes the set of all functions $\mathsf{F} : \mathcal{X} \to \mathcal{Y}$ given two sets $\mathcal{X}$ and $\mathcal{Y}$.

Let PPT denote probabilistic polynomial time. Algorithms are randomized unless explicitly noted otherwise. Let $y \leftarrow A(x; \omega)$ denote running algorithm

A on input x and randomness ω and assigning its output to y. Let $y \leftarrow\!\!\$\ A(x)$ denote $y \leftarrow A(x;\omega)$ for a uniformly random ω.

Group Generation. Let $\mathsf{GroupGen}$ be a polynomial-time algorithm that takes as input a security parameter 1^λ and outputs a group description $(E(\mathbb{F}_p), p, \mathbb{G}, s, g)$ consisting of an elliptic curve group $E(\mathbb{F}_p)$ over $\mathbb{F}_p$ for some prime p and a subgroup $\mathbb{G} \subset E(\mathbb{F}_p)$ of order s such that $|E(\mathbb{F}_p)| = h \cdot s$ where h is the cofactor; s is a λ-bit prime, and g is a generator of $\mathbb{G}$.

3.1 Distributed Key Generation (DKG) with Additive Bias

We follow the definition and idealized notion of security $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$ for a distributed key generation (DKG) protocol where the adversary is allowed to bias the output (joint) secret key with some (constrained) additive shift Δ_0, and can bias each honest party's secret key share likewise by some limited additive bias.

The functionality $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$ was introduced by Kate et al. [53]; and models the real-world settings where a rushing adversary in a round-reduced protocol may view public commitments from all participants, and choose its contribution as a function of these commitments. Hence, the adversary can bias the output key material, but only in a limited way.

The functionality $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$ is weaker than the notion of full (unbiasable) security for distributed key generation [46, 54], where the output public key is required to be from the same distribution as single-party key material. However, it is impossible to realize the notion of full security for any asynchronous (one-round) DKG, as proven by Katz [54]. However, the additive shift Δ_0 for the secret key is tolerable by some applications, such as threshold signatures [25, 29, 47], and so this weaker notion of security is relevant in practice. The upside for achieving $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$ is that of reduced communication rounds among parties.

We give a stand-alone definition of security for $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$. Because our proof for Golden uses only online straight-line simulation, our proof with respect to $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$ in the standalone setting is also by extension secure in the Universal Composability (UC) framework [57]. We assume an adversary that performs only static corruptions.

Let Π be an (n, t)-party DKG protocol, where n is the total number of parties and t is the threshold. A real-world execution of Π with respect to a PPT adversary $\mathcal{A}$ is as follows.

1. $\mathcal{A}$ specifies a set of corrupted parties $\mathsf{corr} \subset [n]$, such that $|\mathsf{corr}| < t$.
2. The honest parties participate in the protocol with the parties in corr. Honest parties follow the protocol, while corrupted parties are controlled by $\mathcal{A}$.
3. When an honest party terminates, it outputs a value as determined by the protocol description.
4. The view of $\mathcal{A}$ consists of 1) the randomness used by $\mathcal{A}$, 2) messages exchanged by honest and corrupt parties, and 3) messages sent over a broadcast channel.

Let $\mathsf{Real}_{\Pi,\mathcal{A}}(1^\lambda)$ denote the random variable parameterized by the security parameter λ that is the tuple, consisting of 1) the corrupted parties corr, 2) the outputs of honest parties, and 3) the view of $\mathcal{A}$.

Now, let $\mathcal{F}$ be an (n,t) idealized (and randomized) functionality. An execution of $\mathcal{F}$ with respect to a PPT adversary $\mathcal{S}$ is as follows:

1. $\mathcal{S}$ specifies a set of corrupted parties $\mathsf{corr} \subset [n]$, such that $|\mathsf{corr}| < t$, and provides corr to $\mathcal{F}$.
2. When an honest party terminates, it outputs a value as determined by the protocol description.
3. $\mathcal{S}$ is allowed to instruct corrupted parties to interact with $\mathcal{F}$ however it likes.
4. Honest parties output the value(s) sent to them by $\mathcal{F}$.
5. $\mathcal{S}$ outputs an arbitrary function of its view.

Let $\mathsf{Ideal}_{\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}},\mathcal{S}}(1^\lambda)$ denote the random variable parameterizd by the security parameter λ that is the tuple, consisting of 1) the corrupted parties corr, 2) the outputs of honest parties, and 3) the output of $\mathcal{S}$.

Definition 1. *A* **distributed key generation (DKG)** *protocol Π (n,t)-securely realizes an ideal functionality $\mathcal{F}$ if for every PPT real-world adversary $\mathcal{A}$, there exists a PPT ideal-world adversary $\mathcal{S}$, such that no efficient PPT distinguisher $\mathcal{D}$ can distinguish between $\mathsf{Real}_{\Pi,\mathcal{A}}$ and $\mathsf{Ideal}_{\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}},\mathcal{S}}$.*

Definition 2 (Secure Key Generation with Additive Bias). *A* **distributed key generation (DKG)** *protocol Π (n,t)-securely realizes the ideal functionality $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$ as given in Figure 1, if for every PPT real-world adversary $\mathcal{A}$, there exists a PPT ideal-world adversary $\mathcal{S}$, such that no efficient PPT distinguisher $\mathcal{D}$ can distinguish between $\mathsf{Real}_{\Pi,\mathcal{A}}$ and $\mathsf{Ideal}_{\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}},\mathcal{S}}$.*

Multi-Session Extension of $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$. As remarked by Katz [54], it is straightforward to verify that in the setting where parties maintain state after a setup phase, $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$ remains secure across repeated executions of the protocol, a requirement for UC security [17]. We similarly do not formalize this multi-session extension of $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$, but likewise remark $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$ can be extended to this setting.

3.2 Polynomial Interpolation and Shamir Secret Sharing

A polynomial $f(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_{t-1} x^{t-1}$ of degree $t-1$ over a field $\mathbb{F}$ can be interpolated by t points. Let $\eta \subseteq [n]$ be the list of t distinct indices corresponding to the x-coordinates $x_i \in \mathbb{F}$ of these points. Then the Lagrange polynomial $L_i(x)$ has the form:

$$L_i(x) = \prod_{j \in \eta; j \neq i} \frac{x - x_j}{x_i - x_j}$$

Given a set of t points $(x_i, f(x_i))_{i \in \eta}$, any point $f(\widetilde{x})$ on the polynomial f can be determined by Lagrange interpolation as follows:

$$f(\widetilde{x}) = \sum_{k \in \eta} f(x_k) \cdot L_k(\widetilde{x}) \ .$$

$\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$

1. Let n, t be parameters defining the number of parties and threshold, and let g be the generator of group $\mathbb{G}$ of order p where the discrete logarithm is hard, provided to $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$.
2. Send (n, t) to the adversary $\mathcal{A}$; receive corr from $\mathcal{A}$.
3. Sample a random $(t-1)$-degree polynomial $f^{\dagger}$ by sampling coefficients $(\gamma'_0, \ldots \gamma'_{t-1}) \leftarrow_{\$} \mathbb{Z}_p^t$, and compute commitments $Y_0 \leftarrow g^{\gamma'_0}, \ldots, Y_{t-1} \leftarrow g^{\gamma'_{t-1}}$.
4. If $|\mathsf{corr}| > 0$, perform the following steps:
   (a) Send $(\{Y_i\}_{i=0}^{t-1}, \{f^{\dagger}(j)\}_{j \in C})$ to the adversary $\mathcal{A}$; receive a bias polynomial $\Delta(x)$ in return.
   (b) For each $i \in \{0, \ldots, t-1\}$, set $\gamma_i \leftarrow \gamma'_i + \Delta(i)$.
5. Let f be the polynomial of degree $t-1$ such that $f(x) = \sum_{i=0}^{t-1} \gamma_i x^i$.
6. Set $\mathsf{PK} = g^{\gamma_0}$.
7. Set $\mathsf{sk}_i = f(i)$, for $i \in \mathsf{hon}$.
8. For $i \in [n]$, set $\mathsf{PK}_i = g^{\mathsf{sk}_i}$.
9. Send $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i \in [n]}, \mathsf{sk}_i)$ to each party $i \in [n]$; send $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i \in [n]})$ to the adversary.

Fig. 1: Ideal functionality for key generation allowing (limited) additive bias.

Feldman Verifiable Secret Sharing. Shamir secret sharing is an (n, t)-threshold secret sharing scheme. Feldman verifiable secret sharing [37] (VSS) builds upon Shamir secret sharing, allowing participants to verify the correctness of their secret shares in zero knowledge, by committing to the sharing polynomial "in the exponent." We show Feldman's VSS consisting of algorithms $(\mathsf{Share}, \mathsf{Verify}, \mathsf{Recover})$, defined as follows:

- $\mathsf{Share}(s, n, t) \rightarrow (\{(1, \bar{x}_1), \ldots, (n, \bar{x}_n)\}, \bar{C})$. Accepts as input a secret $s \in \mathbb{Z}_p$, the number of participants $n \in \mathbb{N}$, and the threshold $t \in \mathbb{N}$ such that $1 \leq t \leq n$. Perform the following steps:
  1. Sample $t-1$ coefficients at random: $(a_1, \ldots, a_{t-1}) \leftarrow_{\$} \mathbb{Z}_p^{t-1}$.
  2. Using s as the constant term and $a_1, \ldots, a_{t-1}$ as the remaining coefficients, define the $(t-1)$-degree polynomial $f(x) = s + \sum_{i=1}^{t-1} a_i x^i$.
  3. Generate n shares $\bar{x}_j \in \mathbb{Z}_p$ by deriving $\bar{x}_j \leftarrow f(j),\ j \in [n]$.
  4. Define $A_0 = g^s$ and $A_i = g^{a_i}$, for $i \in [t-1]$.
  5. Define the VSS commitment $\bar{C}$ as the vector of commitments to the coefficients of f: $\bar{C} \leftarrow (A_0, \ldots, A_{t-1})$.
  6. Output $(\{(i, \bar{x}_i)\}_{i \in [n]}, \bar{C})$, where each i represents the identifier of the recipient of $\bar{x}_i$.
- $\mathsf{Verify}(i, \bar{x}_i, \bar{C}) \rightarrow \{0, 1\}$: Accepts as input a participant identifier $i \in [n]$, the share $\bar{x}_i$ that belongs to participant i, and a VSS commitment $\bar{C}$ that is a

tuple of group elements, such that $|\bar{C}| = t$. Verifies that $(i, \bar{x}_i)$ is a valid point on f as follows:

1. Parse $(A_0, \ldots, A_{t-1}) \leftarrow \bar{C}$.
2. Output 1 if Equation 1 holds; otherwise, output 0.

$$g^{\bar{x}_i} \stackrel{?}{=} \prod_{k=0}^{t-1} A_k^{i^k} \tag{1}$$

– $\mathsf{Recover}(t, \{(i, \bar{x}_i)\}_{i \in C}) \to \perp / s$: On input threshold t and a set of shares $\{(i, \bar{x})\}_{i \in C}$, output $\perp$ if the coalition $C \not\subseteq [n]$ or if $|C| < t$. Otherwise, recover s as follows:

$$s \leftarrow \sum_{i \in C} \bar{x}_i \cdot L_i(0) \ .$$

3.3 The Bulletproofs Argument System

The Bulletproofs argument system [15] can prove computation represented in rank-1 constraint system (R1CS) and is suitable when the statement is given in the exponent.

Consider the following R1CS relation:

$$\mathcal{R}_{R1CS} = \left\{ (A, B, C \in \mathbb{F}_p^{N \times M},\ \theta \in \mathbb{G}_{\mathsf{out}})\ ;\ (\mathbf{x} \in \mathbb{F}_p^W,\ \mathbf{y} \in \mathbb{F}_p^{M-W}) \right\} \quad \text{where}$$
$$(1) \quad (A\mathbf{z}) \circ (B\mathbf{z}) = (C\mathbf{z}) \quad \text{where} \quad \mathbf{z} = (\mathbf{x}, \mathbf{y}) \in \mathbb{F}_p^M$$
$$(2) \quad \theta = \mathbf{g}^{\mathbf{x}}$$

The notation $u \circ v$ denotes the Hadamard product of vectors. The *prefix* $\mathbf{x} \in \mathbb{F}_p^W$ is provided as a Pedersen commitment, where $\mathbf{g}^{\mathbf{x}}$ denotes $\prod_{i=1}^W g_i^{x_i}$ for some vector $\mathbf{g}$ of generators in $\mathbb{G}_{\mathsf{out}}$. Each row in the matrices A, B, C is called a *constraint*, and we refer to $\mathbf{z}$ as the *extended witness*.

Theorem 1 (Bulletproofs [14, 71]). *Bulletproofs is a zero-knowledge non-interactive argument of knowledge for the relation $\mathcal{R}_{R1CS}$ in the random oracle model, assuming the hardness of discrete logarithm in $\mathbb{G}_{\mathsf{out}}$ for expected polynomial-time algorithms. The length of the proof is $2\log_2(N + M) + 3$ group elements in $\mathbb{G}_{\mathsf{out}}$ and 3 field elements.*

The Bulletproofs argument system is complete, computationally knowledge sound, and zero-knowledge. Computational knowledge soundness assumes the hardness of discrete logarithm and means that otherwise there is an expected polynomial-time algorithm that finds a nontrivial linear relation among the generators. The argument system can be made non-interactive using the Fiat-Shamir transform [39] in the random oracle model [2].

The length of the proof is $2\log_2(N+M)+3$ group elements in $\mathbb{G}_{\mathsf{out}}$ and 3 field elements. The verifier complexity is dominated by a multi-scalar multiplication (MSM) of size $2(N + M)$, which can be done via Pippenger's algorithm [68] as

opposed to computing all exponentiations individually. Batching is also possible when verifying multiple proofs at once [15].

4 Two-Party Exponent Verifiable Random Functions

A two-party eVRF enables any two parties with the corresponding keys to derive the same pseudorandom output, whereas a single-party eVRF involves only one party generating a pseudorandom output. While in both cases public verifiability is offered, the symmetric nature of a two-party eVRF is what we leverage to construct a publicly-verifiable encryption scheme that is also symmetric, departing from prior DKGs which rely on public-key encryptions. Golden is the resulting protocol from this approach.

In the following, we define a two-party eVRF by building on the definition and notion of security of exponent verifiable random functions (eVRFs) as presented by Boneh et al. [12] (provided in Appendix A).

Definition 3. *Let* $\{(\mathcal{X}_\lambda, \mathcal{Y}_\lambda)\}$ *be an ensemble of domains and ranges, and* $\mathcal{Y}_\lambda$ *be a finite cyclic group with generator* g*. A* **two-party exponent verifiable random function (two-party eVRF)** $\mathsf{T}\text{-eVRF}$ *with respect to* $\{(\mathcal{X}_\lambda, \mathcal{Y}_\lambda)\}$ *is represented by the following tuple of algorithms:*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{par}$*: Accepts as input the security parameter* λ*, and outputs public parameters* par*, which are implicitly given as input to all other algorithms.*
- $\mathsf{KeyGen}() \to (\mathsf{sk}, \mathsf{PK})$*: Outputs a secret key* sk *and corresponding verification key* PK*.*
- $\mathsf{Evaluate}(\mathsf{sk}_1, \mathsf{PK}_2, \mathsf{msg}) \to (\alpha, A, \pi)$*: Accepts as input a secret key* sk_1 *from one party, a public key* PK_2 *from another, and input to evaluate* $\mathsf{msg} \in \mathcal{X}_\lambda$*, and outputs the evaluation output* α*, a commitment* $A = g^\alpha \in \mathcal{Y}_\lambda$*, and proof* π *that the evaluation was performed correctly.*
- $\mathsf{Verify}(\mathsf{PK}_1, \mathsf{PK}_2, \mathsf{msg}, A, \pi)$*: Accepts as input public keys from two parties* $\mathsf{PK}_1, \mathsf{PK}_2$*, input to evaluate* $\mathsf{msg} \in \mathcal{X}_\lambda$*, a commitment* $A \in \mathcal{Y}_\lambda$*, and proof* π*. Outputs* 1 *to indicate that* A *is a valid commitment to* α *on input* msg*.*

4.1 Game-Based Notions of Security

A two-party eVRF is secure if it is correct, pseudorandom, verifiable, and simulatable. Informally, our correctness definition requires that the honestly generated two-party eVRF outputs of $(\mathsf{sk}_i, \mathsf{PK}_j)$ and $(\mathsf{sk}_j, \mathsf{PK}_i)$ for any message are identical and accepted by Verify. Our pseudorandomness definition says that for honestly generated $(\mathsf{PK}_i, \mathsf{PK}_j)$ the two-party eVRF output is indistinguishable from random. Verifiability guarantees uniqueness of the two-party eVRF output, and simulation guarantees that the two-party eVRF proofs are zero-knowledge. We give the precise definitions next.

Correctness. A two-party eVRF is *correct* if the following holds:

$$\Pr\left[\begin{array}{l}(\alpha, A) = (\alpha', A') \\ \wedge\ \mathsf{Verify}(\mathsf{PK}_1, \mathsf{PK}_2, \mathsf{msg}, A, \pi) = 1 \\ \wedge\ \mathsf{Verify}(\mathsf{PK}_1, \mathsf{PK}_2, \mathsf{msg}, A, \pi') = 1\end{array}\right] = 1$$

where $\mathsf{par} \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda)$, $(\mathsf{sk}_i, \mathsf{PK}_i) \leftarrow\!\!\$\ \mathsf{KeyGen}()$ for $i \in [2]$, $\mathsf{msg} \leftarrow\!\!\$\ \mathcal{X}_\lambda$, $(\alpha, A, \pi) \leftarrow \mathsf{Evaluate}(\mathsf{sk}_1, \mathsf{PK}_2, \mathsf{msg})$, and $(\alpha', A', \pi') \leftarrow \mathsf{Evaluate}(\mathsf{sk}_2, \mathsf{PK}_1, \mathsf{msg})$. In particular, our two-party eVRF is *symmetric* and allows two parties to derive the same pseudorandom output, with respectively generated proofs.

Pseudorandomness. Extending the notion of pseudorandomness of a single-party eVRF (Appendix A), we consider pseudorandomness where the adversary needs to distinguish the pseudorandom outputs from a two-party eVRF queried on any arbitrary (as opposed to fixed) choice of two parties given a set of n independently generated key pairs, from random. We require honestly generated keys, as our pseudorandomness experiment mirrors the non-interactive key exchange experiment [42] (Appendix B) in which the test queries are only accepted for honestly generated public keys.

As such, n is an input to the pseudorandomness experiment of a two-party eVRF in addition to λ. This naturally captures cases, as in a non-interactive key exchange, where the adversary is able to make adaptive queries.

A two-party eVRF is *pseudorandom* if for every PPT adversary $\mathcal{A}$ the following function $\mathsf{Adv}^{\mathrm{psdr}}_{\mathsf{T\text{-}eVRF},\mathcal{A}}(\lambda, n)$ for n that is bounded from above by a polynomial in λ is negligible (see Figure 2):

$$\mathsf{Adv}^{\mathrm{psdr}}_{\mathsf{T\text{-}eVRF},\mathcal{A}}(\lambda, n) = \left|\Pr[\mathsf{Exp}^{\mathrm{psdr}}_{\mathsf{T\text{-}eVRF},0,\mathcal{A}}(\lambda, n) = 1] - \Pr[\mathsf{Exp}^{\mathrm{psdr}}_{\mathsf{T\text{-}eVRF},1,\mathcal{A}}(\lambda, n) = 1]\right|$$

Verifiability. The definition of verifiability is the same as that of a single-party eVRF, except that Verify takes two public keys as input. A two-party eVRF is *verifiable* if for every PPT adversary $\mathcal{A}$ the following function $\mathsf{Adv}^{\mathrm{verf}}_{\mathsf{T\text{-}eVRF},\mathcal{A}}(\lambda)$ is negligible:

$$\mathsf{Adv}^{\mathrm{verf}}_{\mathsf{T\text{-}eVRF},\mathcal{A}}(\lambda) = |\Pr[A_0 \neq A_1 \wedge \mathsf{Verify}(\mathsf{PK}_1, \mathsf{PK}_2, \mathsf{msg}, A_i, \pi_i) = 1\ \forall i \in \{0,1\}]|$$

where $\mathsf{par} \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda)$ and $(\mathsf{PK}_1, \mathsf{PK}_2, \mathsf{msg}, (A_0, \pi_0), (A_1, \pi_1)) \leftarrow\!\!\$\ \mathcal{A}(\mathsf{par})$.

Simulatability. The definition of simulatability is the same as that of a single-party eVRF, except that SIM takes two public keys as input. A two-party eVRF is *simulatable* if there exists a PPT algorithm SIM such that for every PPT distinguishing algorithm $\mathcal{D}$, the following function $\mathsf{Adv}^{\mathrm{sim}}_{\mathsf{T\text{-}eVRF},\mathcal{D},\mathsf{SIM}}(\lambda)$ is negligible:

$\mathsf{Exp}^{\mathrm{psdr}}_{\mathsf{T\text{-}eVRF},b,\mathcal{A}}(\lambda, n)$

1 : $\mathsf{par} \leftarrow\!\!\$\ \mathsf{T\text{-}eVRF.Setup}(1^\lambda)$
2 : **for** $i \in [n]$, **do**
3 : $\quad(\mathsf{sk}_i, \mathsf{PK}_i) \leftarrow\!\!\$\ \mathsf{T\text{-}eVRF.KeyGen}()$
4 : $b' \leftarrow\!\!\$\ \mathcal{A}^{\mathcal{O}^{\mathrm{Evaluate}}(\cdot,\cdot,\cdot)}(\mathsf{par}, \{\mathsf{PK}_i\}_{i=1}^n)$
5 : **return** b'

$\mathcal{O}^{\mathrm{Evaluate}}(i, j, \mathsf{msg})$

1 : **return** $\perp$ **if** $i, j \notin [n]$ **or** $i = j$
2 : **if** $b = 0$
3 : $\quad(\alpha, A, \pi) \leftarrow \mathsf{T\text{-}eVRF.Evaluate}(\mathsf{sk}_i, \mathsf{PK}_j, \mathsf{msg})$
4 : **else** // $b = 1$ *case*
5 : $\quad$// $\mathsf{F} \leftarrow\!\!\$\ \mathsf{Funs}_{\hat{\mathcal{S}}\times\hat{\mathcal{S}}\times\mathcal{X}_\lambda,\mathbb{F}_{|\mathcal{Y}_\lambda|}}$
6 : $\quad$**if** $\mathsf{PK}_i < \mathsf{PK}_j$
7 : $\quad\quad\alpha \leftarrow \mathsf{F}(\mathsf{PK}_i, \mathsf{PK}_j, \mathsf{msg})$
8 : $\quad$**else**
9 : $\quad\quad\alpha \leftarrow \mathsf{F}(\mathsf{PK}_j, \mathsf{PK}_i, \mathsf{msg})$
10 : **return** α

Fig. 2: Pseudorandomness experiment defining the advantage of an adversary $\mathcal{A}$ against a two-party exponent verifiable random function (two-party eVRF) T-eVRF. The domain of public keys is denoted by $\hat{\mathcal{S}}$. In the $b = 1$ case, the two keys are first sorted according to an ordering of public keys and then are taken as input to the randomly sampled function to preserve symmetry.

$$\mathsf{Adv}^{\mathrm{sim}}_{\mathsf{T\text{-}eVRF},\mathcal{D},\mathsf{SIM}}(\lambda) = \left|\Pr[\mathcal{D}^{\mathcal{O}^{\mathsf{Evaluate}}(\mathsf{sk}_1,\mathsf{PK}_2,\cdot)}(\mathsf{par}) = 1] - \Pr[\mathcal{D}^{\mathsf{SIM}(\mathsf{PK}_1,\mathsf{PK}_2,\cdot)}(\mathsf{par}) = 1]\right|$$

where $\mathsf{par}_0 \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda)$, $(\mathsf{sk}_1, \mathsf{PK}_1) \leftarrow\!\!\$\ \mathsf{KeyGen}()$, $(\mathsf{sk}_2, \mathsf{PK}_2) \leftarrow\!\!\$\ \mathsf{KeyGen}()$, $\mathsf{par} \leftarrow (\mathsf{par}_0, \mathsf{PK}_1, \mathsf{PK}_2)$, and $\mathcal{O}^{\mathsf{Evaluate}}(\mathsf{sk}, \mathsf{PK}', \cdot)$ returns the output of $\mathsf{Evaluate}(\mathsf{sk}, \mathsf{PK}', \cdot)$.

4.2 Simulation-Based Notion of Security

Following a similar approach by Boneh et al. [12], it is possible to give a proof of equivalence between a two-party eVRF that satisfies our our game-based notions of security, and a two-party eVRF that satisfies an ideal functionality. Doing so is useful, to demonstrate composability (UC); i.e., the security of using a two-party eVRF as a building block within some broader protocol.

We refer to the ideal functionality for a two-party eVRF as $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$, and give this functionality in Appendix D. We establish this proof of equivalence in Appendix E.

5 A New Two-Party Exponent Verifiable Random Function

We now give a new two-party eVRF construction, which we refer to as T-eVRF-DH. Our construction builds upon the DDH construction presented by Boneh et al. [12]

Notation	Description
$E(\mathbb{F}_p)$	Elliptic curve defined over $\mathbb{F}_p$ for some prime p
$\mathbb{G}_{\mathsf{in}}$	Subgroup of $E(\mathbb{F}_p)$ of prime order s
h	Cofactor such that $\lvert E(\mathbb{F}_p)\rvert = h \cdot s$
$\mathbb{G}_{\mathsf{out}}$	Subgroup of a different elliptic curve with the requirement that the order be p
$g_{\mathsf{in}}, g_{\mathsf{out}}$	Generators of $\mathbb{G}_{\mathsf{in}}$ and $\mathbb{G}_{\mathsf{out}}$, respectively
sk, PK	A registered key pair, where $\mathsf{sk} \in \mathbb{F}_s$ and $\mathsf{PK} \in \mathbb{G}_{\mathsf{in}}$
$\widetilde{S}.X$	"$.X$" retrieves the x-coordinate of a group element $\widetilde{S}$
$\mathsf{int}(\cdot)$	A function that casts an input to integer
$\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}, \mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}$	Random oracles mapping from $\{0,1\}^* \times \mathbb{G}_{\mathsf{in}}^2$ to $\mathbb{G}_{\mathsf{in}}$
β	Public field element in $\mathbb{F}_p$, a constant used by the leftover hash lemma

Table 3: Notation for T-eVRF-DH.

in that our construction likewise makes use of the DDH PRF [62] $\mathsf{H}_{\mathbb{G}_{\mathsf{in}}}(\cdot)^k$, where $\mathsf{H}_{\mathbb{G}_{\mathsf{in}}}$ is modeled as a random oracle, and $\mathbb{G}_{\mathsf{in}}$ is a cyclic subgroup of prime order s where DDH is assumed to be hard. However, our construction derives k as a shared value between two participants via a non-interactive key exchange (NIKE) performed in $\mathbb{G}_{\mathsf{in}}$.

To derive the NIKE shared secret, our two-party eVRF uses the Diffie-Hellman key exchange (denoted by DH). We define NIKE as a separate building block (Appendix B) and accordingly instantiate it with the following tuple of algorithms (Setup, KeyGen, GetSharedKey), which we provide here for an explicit presentation:

- $\mathsf{DH.Setup}(1^\lambda) \to \mathsf{par}$: Generate public parameters $\mathsf{par} \leftarrow (E(\mathbb{F}_p), p, \mathbb{G}_{\mathsf{in}}, s, g_{\mathsf{in}})$ from the group generation algorithm $\mathsf{GroupGen}(1^\lambda)$. Output par.
- $\mathsf{DH.KeyGen}() \to (\mathsf{sk} \in \mathbb{F}_s, \mathsf{PK} \in \mathbb{G}_{\mathsf{in}})$: Generate a secret key $\mathsf{sk} \leftarrow\!\!\$\ \mathbb{F}_s$ and corresponding public key $\mathsf{PK} \leftarrow g_{\mathsf{in}}^{\mathsf{sk}}$. Output $(\mathsf{sk}, \mathsf{PK})$.
- $\mathsf{DH.GetSharedKey}(\mathsf{sk}_1 \in \mathbb{F}_s, \mathsf{PK}_2 \in \mathbb{G}_{\mathsf{in}}) \to S \in \mathbb{G}_{\mathsf{in}}$: Given the first party's secret key sk_1 and the second party's public key PK_2, output $S \leftarrow \mathsf{PK}_2^{\mathsf{sk}_1}$.

We present T-eVRF-DH next; we refer to Table 3 for further notation used within our construction.

- $\mathsf{Setup}(1^\lambda) \to \mathsf{par}$: Generate public parameters $(E(\mathbb{F}_p), p, \mathbb{G}_{\mathsf{in}}, s, g_{\mathsf{in}})$ from $\mathsf{DH.Setup}(1^\lambda)$; set $\mathsf{par}_{\mathsf{in}} \leftarrow (E(\mathbb{F}_p), p, \mathbb{G}_{\mathsf{in}}, s, g_{\mathsf{in}})$ and $\mathsf{par}_{\mathsf{out}} \leftarrow (\mathbb{G}_{\mathsf{out}}, p, g_{\mathsf{out}})$, where $\mathbb{G}_{\mathsf{out}} = \langle g_{\mathsf{out}} \rangle$ is a subgroup of a different elliptic curve with the requirement that the order be p, $\beta \leftarrow\!\!\$\ \mathbb{F}_p$ is used by the leftover hash lemma [49] described in Section F, and $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}$ and $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}$ are random oracles mapping from $\{0,1\}^* \times \mathbb{G}_{\mathsf{in}}^2$ to $\mathbb{G}_{\mathsf{in}}$. Set $\mathsf{par} \leftarrow (\mathsf{par}_{\mathsf{in}}, \mathsf{par}_{\mathsf{out}}, \mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}, \mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}, \beta)$, and output par.
- $\mathsf{KeyGen}() \to (\mathsf{sk} \in \mathbb{F}_s, \mathsf{PK} \in \mathbb{G}_{\mathsf{in}})$: Derive $(\mathsf{sk}, \mathsf{PK}) \leftarrow \mathsf{DH.KeyGen}(\mathsf{par}_{\mathsf{in}})$. Output $(\mathsf{sk}, \mathsf{PK})$.

$$\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}} = \Big\{ (\mathbb{G}_{\mathsf{in}} \subset E(\mathbb{F}_p); \{\mathsf{H}_{\mathbb{G}_{\mathsf{in}},j}(\mathsf{msg},\mathsf{PK}_1,\mathsf{PK}_2), \mathsf{PK}_j \in \mathbb{G}_{\mathsf{in}}\}_{j=1}^{2}, R \in \mathbb{G}_{\mathsf{out}}, \beta \in \mathbb{F}_p) \ ; \ (\mathsf{sk}_1 \in \mathbb{F}_s) \Big\}$$

$$\begin{aligned}
\text{iff} \quad & (0) \quad \mathsf{PK}_1 \stackrel{?}{=} g_{\mathsf{in}}^{\mathsf{sk}_1} \in \mathbb{G}_{\mathsf{in}} \\
& (1) \quad S \leftarrow \mathsf{PK}_2^{\mathsf{sk}_1} \in \mathbb{G}_{\mathsf{in}} \\
& (2) \quad k_0 \leftarrow S.X \in \mathbb{F}_p \\
& (3) \quad k \leftarrow \mathsf{int}(k_0) \in \mathbb{Z} \\
& (4) \quad T_1 \leftarrow \mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k \in \mathbb{G}_{\mathsf{in}} \\
& (5) \quad T_2 \leftarrow \mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k \in \mathbb{G}_{\mathsf{in}} \\
& (6) \quad r_1 \leftarrow T_1.X \in \mathbb{F}_p \\
& (7) \quad r_2 \leftarrow T_2.X \in \mathbb{F}_p \\
& (8) \quad r \leftarrow \beta \cdot r_1 + r_2 \in \mathbb{F}_p \\
& (9) \quad R \stackrel{?}{=} g_{\mathsf{out}}^r \in \mathbb{G}_{\mathsf{out}}
\end{aligned}$$

Fig. 3: The relation $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ used in the two-party eVRF defined in Section 5. R is the commitment output. See Table 3 for additional information on notation.

- $\mathsf{Evaluate}(\mathsf{sk} \in \mathbb{F}_s, \mathsf{PK}' \in \mathbb{G}_{\mathsf{in}}, \mathsf{msg} \in \{0,1\}^*) \rightarrow (\alpha \in \mathbb{F}_p, A \in \mathbb{G}_{\mathsf{out}}, \pi \in \{0,1\}^*)$: Perform the following steps:
  1. Derive $S \leftarrow \mathsf{DH.GetSharedKey}(\mathsf{sk}, \mathsf{PK}')$.
  2. Derive $k_0 \leftarrow S.X$ where "$.X$" retrieves the x-coordinate of S.
  3. Derive $k \leftarrow \mathsf{int}(k_0)$ where $\mathsf{int}(\cdot)$ is a function that casts an input to integer (so that we interpret k_0 as an integer in the form of k).
  4. Derive $\alpha \leftarrow \beta \cdot (\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k.X) + (\mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k.X)$. Here, $\mathsf{PK}_1 \leftarrow \min\{\mathsf{PK}, \mathsf{PK}'\}$ and $\mathsf{PK}_2 \leftarrow \max\{\mathsf{PK}, \mathsf{PK}'\}$, assuming $(\mathsf{sk}, \mathsf{PK})$ and $(\mathsf{sk}', \mathsf{PK}')$ are two valid key pairs from $\mathsf{KeyGen}()$.
  5. Derive commitment $A \leftarrow g_{\mathsf{out}}^{\alpha}$.
  6. Generate a zero-knowledge proof π for the relation $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ as given in Figure 3.
  7. Output (α, A, π).
- $\mathsf{Verify}(\mathsf{PK}, \mathsf{PK}', \mathsf{msg}, A, \pi)$: Verify the relation $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ (Figure 3), and output 1 if the relation holds; otherwise, output 0.

We next demonstrate that T-eVRF-DH is secure, i.e. that it is correct, pseudorandom, verifiable, and simulatable.

Theorem 2 (Security of T-eVRF-DH). *Let $(E(\mathbb{F}_p), p, \mathbb{G}_{\mathsf{in}}, s, g_{\mathsf{in}})$ be a tuple of public parameters for an explicit elliptic curve subgroup where DDH is hard. Then assuming a non-interactive zero-knowledge argument system proving the relation $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$, T-eVRF-DH is a secure two-party exponent verifiable random function.*

In particular, the adversary advantage in the pseudorandomness experiment is bounded from above as follows: given an adversary $\mathcal{A}$ against the pseudorandomness of T-eVRF-DH*, we construct adversaries $\mathcal{B}_1, \mathcal{B}_2$ against the session-key unrecoverability of the Diffie-Hellman non-interactive key exchange and against Decisional Diffie-Hellman, respectively, such that*

$$\mathsf{Adv}^{\mathrm{psdr}}_{\mathsf{T\text{-}eVRF\text{-}DH},\mathcal{A}}(\lambda, n) \leq \mathsf{Adv}^{\mathrm{unrec}}_{\mathsf{DH},\mathcal{B}_1}(\lambda, \binom{n}{2}) + \frac{2hpn^2}{p-2h}\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}_{\mathsf{in}},\mathcal{B}_2}(\lambda) + \frac{2h\sqrt{p}}{p-2h}q_e \quad (2)$$

$$\leq \mathsf{Adv}^{\mathrm{unrec}}_{\mathsf{DH},\mathcal{B}_1}(\lambda, n^2) + \mathcal{O}(n^2)\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}_{\mathsf{in}},\mathcal{B}_2}(\lambda) + \mathcal{O}(\frac{1}{\sqrt{p}})q_e \quad (3)$$

where h is the cofactor, s is the prime order of the subgroup $\mathbb{G}_{\mathsf{in}}$ of $E(\mathbb{F}_p)$ satisfying $|E(\mathbb{F}_p)| = h \cdot s$, and q_e denotes the number of evaluate queries made by $\mathcal{A}$.

Reducing Theorem 2 to DDH only. Recalling a theorem by Freire et al. [42, Theorem 1], we can bound $\mathsf{Adv}^{\mathrm{unrec}}_{\mathsf{DH},\mathcal{B}_1}(\lambda, \binom{n}{2})$ in our setting in which the pseudorandomness adversary is allowed queries to $\mathcal{O}^{\mathrm{Evaluate}}$ (Figure 2) but not honest-reveal-like queries or extract-like queries as in the session-key unrecoverability of the NIKE (Appendix B) as shown in Figure 6. Another observation is that we require honestly generated keys; $\mathsf{Adv}^{\mathrm{unrec}}_{\mathsf{DH},\mathcal{B}_1}(\lambda, 1)$ with 2 honestly generated keys is in fact upper bounded by $\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}_{\mathsf{in}},\mathcal{B}_2}(\lambda)$ (which denotes the adversary advantage against DDH in $\mathbb{G}_{\mathsf{in}}$). From these, the simplified bound is given by

$$\mathsf{Adv}^{\mathrm{psdr}}_{\mathsf{T\text{-}eVRF\text{-}DH},\mathcal{A}}(\lambda, n) \leq \left(\frac{n^4}{4} + \frac{2hpn^2}{p-2h}\right)\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}_{\mathsf{in}},\mathcal{B}_2}(\lambda) + \frac{2h\sqrt{p}}{p-2h}q_e \quad (4)$$

$$\leq O(n^4) \cdot \mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}_{\mathsf{in}},\mathcal{B}_2}(\lambda) + O(\frac{q_e}{\sqrt{p}}) \quad (5)$$

where we use $\mathsf{Adv}^{\mathrm{unrec}}_{\mathsf{DH},\mathcal{B}_1}(\lambda, \binom{n}{2}) \leq \frac{n^3(n-1)}{4}\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}_{\mathsf{in}},\mathcal{B}_2}(\lambda)$ from the theorem by Freire et al., taking $\frac{n^4}{4}$ for simplicity. The statement of Theorem 2, on the other hand, is given in a general form for clarity. Note that an $\binom{n}{2}$ factor comes from the maximum number of pairs of honestly generated public keys; thus we could also write $n^2 \cdot \min\{\binom{n}{2}, q_e\}$, instead of n^4.

We give the proof of Theorem 2 in Appendix G. The main idea is that we make a reduction to the session-key unrecoverability (Appendix B) of the Diffie-Hellman NIKE, make a reduction to DDH in $\mathbb{G}_{\mathsf{in}}$, and show pseudorandomness via the leftover hash lemma [49] (Section F). Our analysis supports curves with nontrivial cofactors, which is a practical merit not considered in prior work.

UC-Security of T-eVRF-DH**.** When conjoined with an initialize functionality where all parties prove knowledge of their secret two-party eVRF keys, T-eVRF-DH is UC-secure. This equivalence is established in Appendix E, which shows the equivalence of any two-party eVRF T-eVRF proven secure with respect to our game-based notions, and an ideal functionality $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ as shown in Appendix D.

5.1 Argument System for the Relation $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$

While a generic zkSNARK (zero-knowledge succinct non-interactive argument of knowledge) system [8] could be used to prove $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$, we prove it using Bulletproofs [15] (see Section 3.3).

More precisely, we arithmetize $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ except the last step (9) (from Figure 3), let this arithmetization in R1CS define the A, B, C matrices for our Bulletproofs argument system, and furthermore combine the required Pedersen commitment $\theta := g_{\mathsf{out},1} \cdot g_{\mathsf{out}}^r$ (such that our prefix is $(1, r)$ and $g_{\mathsf{out},1}$ is another independent generator) with a proof of knowledge of discrete logarithm of g_{out}^r to effectively prove $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ as a whole with $R = g_{\mathsf{out}}^r$. Note that this is different from naively turning $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ itself into A, B, C matrices for Bulletproofs, which emits different properties as remarked later.

At a high level, we need to efficiently represent an exponentiation of a group element (of an elliptic curve group) as R1CS constraints. Attributed to Boneh et al. [12], the techniques we leverage to prove computation of $\widetilde{g}^k$ for some arbitrary k (private) and group element $\widetilde{g}$ (public) involve a two-step process. First, we bit-decompose k into $(k_0, k_1, \ldots, k_\lambda) \in \{0,1\}^{\lambda+1}$ such that $k = \sum_{i=0}^{\lambda} 2^i \cdot k_i$. Second, we incrementally construct $\widetilde{g}^k$ using $(k_0, k_1, \ldots, k_\lambda)$ and a set of precomputed points such that all incremental constructions can be represented as R1CS constraints. To facilitate analysis, we refer to each of these two steps as a "gadget" and thus offer a modular presentation in discussing the four required exponentiations $g_{\mathsf{in}}^{\mathsf{sk}_1}$, $\mathsf{PK}_2^{\mathsf{sk}_1}$, $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k$, and $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k$.

The end result for our prover is that our constraint system has $14\lambda + 14$ ($= 3598$ for $\lambda = 256$) constraints. We delineate the two gadgets and the derivation of the overall number of constraints below.

5.2 First Gadget: Bit-Decomposition Check

Checking that k bit-decomposes into $(k_0, k_1, \ldots, k_\lambda) \in \{0,1\}^{\lambda+1}$ in the form of R1CS constraints is relatively easier. The constraints are given by

- $k_i(k_i - 1) = 0$ for $i = 0, \ldots, \lambda$
- $k = \sum_{i=0}^{\lambda} 2^i \cdot k_i$

totaling $\lambda + 2$ constraints. The extended witness variables are $k_0, k_1, \ldots, k_\lambda, k$.

5.3 Second Gadget: Exponentiation Check

Given the bit-decomposition $(k_0, k_1, \ldots, k_\lambda)$ of k, it is possible to efficiently represent the computation of $\widetilde{g}^k$ (for some public group element $\widetilde{g}$) as R1CS constraints as follows. We incrementally construct $\widetilde{g}^k$ via a sequence of $\lambda + 1$ points $\{L_i\}_{i=0}^{\lambda}$ where $L_\lambda = \widetilde{g}^k$. Specifically, we define $L_i := \prod_{j=0}^{i} \Delta_j$ where:

- $\Delta_j := P_j^{k_j} \cdot C_j$
- $P_j := \widetilde{g}^{2^j}$

- $C_j := g^{c_j}$
- $c_j := \begin{cases} 1 & j = 0 \\ 2 & j = 1, \ldots, \lambda - 1 \\ s - 2\lambda - 1 & j = \lambda \end{cases}$

Note P_j, C_j, and c_j can be publicly computed. The explanation is as follows. Without C_j, it is easy to see that $\widetilde{g}^k = \prod_{j=0}^{\lambda} P_j^{k_j}$ simply from the bit-decomposition of k. Likewise, $\widetilde{g}^k = \prod_{j=0}^{\lambda} P_j^{k_j} \cdot C_j = \prod_{j=0}^{\lambda} \Delta_j = L_\lambda$ from the fact that $\prod_{j=0}^{\lambda} C_j = 1$. In fact, the c_j values are deliberately chosen to satisfy this. More importantly, the c_j values (hence the C_j values) are deliberately chosen to offer a sort of a "correction" term to each $P_j^{k_j}$ so that each resulting incremental computation of $L_i = L_{i-1} \cdot \Delta_i$ is guaranteed to be a group operation involving two finite points in the elliptic curve group that have distinct x-coordinates, except with negligible probability (as shown by Boneh et al. [12, Lemma 6]). As a result, only the chord rule (as opposed to the tangent rule) of the elliptic curve group operation formula is used to calculate $\{L_i\}_{i=0}^{\lambda}$.

This means that the incremental computation starting from L_0 and ending with $L_\lambda = \widetilde{g}^k$ uses the same formula and thus can be represented in R1CS in a consistent manner. Specifically, we achieve this using 4 intermediate (i.e. part of the extended witness) variables $(k_i, s_i, x_{L_i}, y_{L_i}) \in \mathbb{F}_p^4$ introduced in each iteration of computing $L_i = L_{i-1} \cdot \Delta_i$, where:

- k_i is from the bit-decomposition of k
- $s_i = (y_{L_{i-1}} - y_{\Delta_i}) \cdot (x_{L_{i-1}} - x_{\Delta_i})^{-1}$ is the slope of the line (which is needed to compute L_i) going through L_{i-1} and Δ
- x_{L_i} is the x-coordinate of L_i
- y_{L_i} is the y-coordinate of L_i

The needed constraints (for $i = 1, \ldots, \lambda$) are as follows, where:

- $s_i \cdot (x_{L_{i-1}} - x_{\Delta_i}) = y_{L_{i-1}} - y_{\Delta_i}$ shows computation of the slope s_i
- $s_i \cdot s_i = x_{L_{i-1}} + x_{L_i} + x_{\Delta_i}$ shows computation of x_{L_i}
- $s_i \cdot (x_{L_{i-1}} - x_{L_i}) = y_{L_{i-1}} + y_{L_i}$ shows computation of y_{L_i}

We note that x_{Δ_i} and y_{Δ_i} are actually public linear functions of k_i (as Δ_i is either C_i or $P_i \cdot C_i$ while P_i and C_i are public), so there is no need to include them as part of the extended witness (though helpful for ease of presentation). Respectively, $(x_{P_i \cdot C_i} - x_{C_i}) \cdot k_i + x_{C_i}$ and $(y_{P_i \cdot C_i} - y_{C_i}) \cdot k_i + y_{C_i}$ can replace x_{Δ_i} and y_{Δ_i}. In addition, we do need to count the base case when $i = 0$ to show computation of x_{L_0} and y_{L_0} (which are equal to x_{Δ_0} and y_{Δ_0} by definition and thus are also linear functions of k_0), as they do not involve the above slope computation; this incurs two additional constraints.

In sum, this gadget incurs $3\lambda + 2$ constraints, and the extended witness variables are $k_0, x_{L_0}, y_{L_0}, k_1, s_1, x_{L_1}, y_{L_1}, \ldots, k_\lambda, s_\lambda, x_{L_\lambda}$, and y_{L_λ}.

5.4 No Need to Arithmetize the Last Exponentiation

While the first four exponentiations $g_{\mathsf{in}}^{\mathsf{sk}_1}$, $\mathsf{PK}_2^{\mathsf{sk}_1}$, $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k$, and $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k$ are indeed represented as R1CS constraints (using the above gadgets), we can omit this process for $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$'s last exponentiation g_{out}^r and thus save constraints. The reason is that the two-party eVRF commitment output $R = g_{\mathsf{out}}^r$ and the Bulletproofs Pedersen commitment $\theta := g_{\mathsf{out},1} \cdot g_{\mathsf{out}}^r$ are in fact related in form such that providing g_{out}^r and its proof of knowledge of discrete logarithm essentially achieves the same effect as providing θ and the correctness of computation of g_{out}^r. The verifier can check this proof of knowledge to verify the ninth step of $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$.

5.5 Combining Everything for $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$

Now, proving $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ involves 2 iterations of the first bit-decomposition gadget and 4 iterations of the second exponentiation gadget. Namely, steps (0) through (3) of $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ (from Figure 3) show $g_{\mathsf{in}}^{\mathsf{sk}_1}$ and $\mathsf{PK}_2^{\mathsf{sk}_1}$ by using the first gadget once to show bit-decomposition of sk_1 and the second gadget twice to respectively show exponentiation of g_{in} and PK_2 by sk_1. One additional constraint is used in step (0) to check equality, which is how the predetermined PK_1 is tied into $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$. Steps (4) through (7) show $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k$ and $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}(\mathsf{msg}, \mathsf{PK}_1, \mathsf{PK}_2)^k$; the first gadget is used once to show bit-decomposition of k, and then the same bit-decomposition is used by two separate iterations of the second gadget.

The eighth step of $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ ($r \leftarrow \beta \cdot r_1 + r_2 \in \mathbb{F}_p$) is simply one constraint and is where the leftover hash lemma is invoked. The ninth step of $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ is shown outside the Bulletproofs argument system by $g_{\mathsf{out}}^r = R$ and its proof of knowledge of discrete logarithm as noted above. The summary is that our prover for $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ computes Bulletproofs for a constraint system with $2 \cdot (\lambda + 2) + 4 \cdot (3\lambda + 2) + 2 = 14\lambda + 14$ ($= 3598$ for $\lambda = 256$) constraints and $14\lambda + 14$ extended witness variables, plus one proof of knowledge of discrete logarithm.

Remark A different proof construction is possible if we instead represent a slight variant of $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ as A, B, C matrices for Bulletproofs, where the PKI group and the DKG group can be made equal (i.e. $\mathbb{G}_{\mathsf{in}} = \mathbb{G}_{\mathsf{out}}$). See Appendix H.

Remark A recent proof technique also mentioned by Boneh et al. [12], Eagen's approach [4, 35, 66] involving concepts from algebraic geometry such as principal divisors and Weil reciprocity has the effect of reducing the number of constraints for our second gadget (Section 5.3) from $3\lambda + 2$ to 7. Applied in our setting, such optimization would bring down the total number of constraints from $14\lambda + 14$ to $2 \cdot (\lambda + 2) + 4 \cdot (7) + 2 = 2\lambda + 34 = 546$ for $\lambda = 256$. The reduction is about 85%.

5.6 Concrete Efficiency, Batch Proving, and Batch Verification

We are using the R1CS instantiation of Bulletproofs [14, 71]. The R1CS constraint system for $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ uses matrices of dimension $N = M = 3598$. We estimate

Statements ($\widetilde{n}$)	Constraints	Prover	Verifier	$\lvert\pi\rvert$	$\widetilde{n}$ proofs	Batch Verification
1	3598	0.3s	0.1s	1.5kb	1.5kb	0.1s
9	24158	1.8s	0.5s	1.9kb	17kb	0.6s
49	126958	6.8s	2.5s	2.1kb	101kb	6.7s
99	255458	13.5s	4.8s	2.2kb	214kb	22.1s

Table 4: Our two-party eVRF performance estimates per zkalc [36] for BLS12-381. Statements measure the number of $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ statements that are proven. Constraints refer to the total number of R1CS constraints (which is also the R1CS extended witness size). Batch verification measures the time required to verify $\widetilde{n}$ proofs simultaneously (as in the DKG application).

the performance of the protocol implemented in arkworks [27], via zkalc [36] using curve BLS12-381 on AWS EC2 m5.2xlarge. We will provide an implementation for the full version, but estimate that it is within a factor of 2 of zkalc's estimate.

In our DKG application, each client sends one proof to each other participant, i.e., creates $\widetilde{n}$ proofs for $\widetilde{n}+1$ participants. For additional efficiency, the prover can alternatively create one proof for the conjunction of $\widetilde{n}$ statements. This can dramatically improve the proof size, as it scales only logarithmically with the size of the statement. It can also improve the prover and verifier time, as multi-scalar multiplications (MSMs), the most expensive operation, scale as $\lambda \cdot N/\log N$ for MSMs of size N [68].

Finally, even with batch proving, each client will receive and verify $\widetilde{n}$ proofs, one per other client. Bulletproofs enables efficient batch verification [15], where the most expensive verification operation (MSM) can be shared.

Prover efficiency For an R1CS instance with dimensions N and M, the prover's computation is dominated by computing a Pedersen commitment of length $N + 2M$ and a zero-knowledge inner product argument [26, 71] with vectors of length $N + M$. Let $\rho = 2^{\lceil \log_2(N+M) \rceil}$. In the Bulletproofs inner product argument, the prover computes 2 MSMs of size $\rho, \rho/2, \rho/4, \ldots, 1$. In our setting, $N = M = 3598$ and $\rho = 2^{13}$. The prover is therefore dominated by an MSM of size 10794 and two MSMs of size 8192. The field operations are $O(N + M)$ and hardly contribute to the prover time. Including all group and field operations, the zkalc-estimated prover runtime is 305 ms. For larger $\widetilde{n}$, the constraint system for $\mathcal{R}^{\widetilde{n}}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ requires $\widetilde{n} \cdot 2570 + 1028$ constraints (bit-decomposing sk_1 and the $g_{\mathsf{in}}^{\mathsf{sk}_1}$ exponentiation can be re-used). We give estimated constraint counts and prover runtimes in Table 4.

Verifier efficiency The verifier needs to verify an inner product argument. The cost of this is dominated by an MSM of size $2(N+M)+2\lceil \log_2(N+M) \rceil + 3$. The verifier also computes $O(\log N)$ hashes and $O(N+M)$ field operations, all of which add less than 5 ms and are negligible compared to the MSM. For $N = M = 3598$, zkalc estimates a verifier time of 81 ms. In the DKG application, the verifier is

run by the receiver and will verify $\widetilde{n}$ proofs for $\mathcal{R}^{\widetilde{n}}_{\mathsf{T\text{-}eVRF\text{-}DH}}$, assuming $\widetilde{n}+1$ clients. The verifier can use the batch verification technique from Bulletproofs [15]. Using this, the MSM cost only scales as $2(N+M) + \widetilde{n} \cdot 2\lceil \log_2(N+M) \rceil$. The scalar multiplications are upper bounded by $16 \cdot \widetilde{n} \cdot (N+M)$. We give concrete estimates for $\widetilde{n} = 1, 9, 49, 99$ in Table 4.

Proof size From Theorem 1, the proof size for one statement is $2\log_2(N+M)+3 \approx 29$ group elements and 3 field elements for $\lambda = 256$, plus $|\pi_{DL}|$ for some suitable proof of knowledge of discrete logarithm π_{DL}. In the batch proof setting, the proof size only scales logarithmically in $\widetilde{n}$, e.g. for $\widetilde{n} = 99$, the proof size is still only 43 group elements or 2.2 kb when using BLS12-381, which has 48-byte group elements. We present the proof size for different statement sizes in Table 4, and also present the size of $\widetilde{n}$ proofs in light of the DKG context.

6 Golden: Lightweight Non-Interactive Distributed Key Generation

We next introduce our non-interactive DKG Golden. At a high level, Golden builds upon any two-party eVRF T-eVRF as defined in Section 4, as well as Feldman's verifiable secret sharing as described in Section 3.2.

Golden additionally assumes that all parties prove knowledge of their secret keys prior to performing the protocol, either in a point-to-point manner or using a separate entity such as a PKI. We give more information on the zero-knowledge proof relation and a concrete instantiation in Appendix I.

6.1 Protocol Description

Each participant $i \in [n]$ in the DKG is initialized with their own identity key $(\mathsf{sk}_i^I \in \mathbb{F}_s, \mathsf{PK}_i^I \in \mathbb{G}_{\mathsf{in}})$, as well as the (assumed to be valid) public identity keys of all participants $\{\mathsf{PK}_j^I\}_{j\in[n]}$, where $\mathsf{PK}_j^I \in \mathbb{G}_{\mathsf{in}}$.

In the first round of the DKG, each participant i begins by sampling a random secret $\omega_i \leftarrow\!\!\$\ \mathbb{Z}_p$, and then performs Feldman secret sharing to obtain the shares $\{\bar{x}_{i,j}\}_{j=1}^n$, such that $\bar{x}_{i,j} \in \mathbb{Z}_p$, and the commitment $\bar{C}_i = (A_{i,0}, \ldots, A_{i,t-1})$, such that $A_{i,j} \in \mathbb{G}_{\mathsf{out}}, j \in \{0, \ldots, t-1\}$. Here, $\bar{C}_i$ is a vector commitment which commits to a random polynomial f_i such that $\bar{x}_{i,j} = f_i(j)$ and $f_i(0) = \omega_i$.

Each participant i then samples a random message $\mathsf{msg}_i \leftarrow\!\!\$\ \{0,1\}^\lambda$. Next, for all participants $j \in [n], j \neq i$, each participant i does the following. They first generate $(r_{i,j}, R_{i,j}, \pi_{i,j})$ by performing $\mathsf{T\text{-}eVRF.Evaluate}(\mathsf{sk}_i^I, \mathsf{PK}_j^I, \mathsf{msg}_i)$, such that $r_{i,j} \in \mathbb{F}_p$ and $R_{i,j} \in \mathbb{G}_{\mathsf{out}}$. Then they generate the ciphertext $z_{i,j} \in \mathbb{F}_p$ by performing $z_{i,j} \leftarrow r_{i,j} + \bar{x}_{i,j}$, encrypted using the nonce $r_{i,j}$ output from the two-party eVRF. They finally set $\sigma_{i,j} \leftarrow (R_{i,j}, z_{i,j})$, where $R_{i,j} = g^{r_{i,j}}$ is also determined by the two-party eVRF ($g \in \mathbb{G}_{\mathsf{out}}$). Each participant finally sets $\mathsf{bmsg}_i \leftarrow \{(\mathsf{msg}_i, \bar{C}_i, \sigma_{i,j}, \pi_{i,j})\}_{j\in[n], j\neq i}$ to be a broadcast message, and broadcasts bmsg_i to all other participants.

$\mathsf{Round}_0(n, t, i, \mathsf{sk}_i^I, \{(j, \mathsf{PK}_j^I)\}_{j\in[n]})$ // *Party i performs the protocol.*

$\omega_i \leftarrow\!\!\$\ \mathbb{Z}_p$ // *Sample a random contribution to the DKG shared secret*

$(\{(j, \bar{x}_{i,j})\}_{j=1}^n, \bar{C}_i = (A_{i,0}, \ldots, A_{i,t-1})) \leftarrow\!\!\$\ \mathsf{Feldman.Share}(\omega_i, n, t)$

$\mathsf{msg}_i \leftarrow\!\!\$\ \{0,1\}^\lambda$ // *Sample a random message*

for $j \in [n], j \neq i$ **do**

  $(r_{i,j}, R_{i,j}, \pi_{i,j}) \leftarrow \mathsf{T\text{-}eVRF.Evaluate}(\mathsf{sk}_i^I, \mathsf{PK}_j^I, \mathsf{msg}_i)$

  $z_{i,j} \leftarrow r_{i,j} + \bar{x}_{i,j}$ // *Encrypt the share to player j*

  $\sigma_{i,j} \leftarrow (R_{i,j}, z_{i,j})$

$\mathsf{st}_i \leftarrow \bar{x}_{i,i}$ // *Keep one share for itself*

$\mathsf{bmsg}_i \leftarrow \{(\mathsf{msg}_i, \bar{C}_i, \sigma_{i,j}, \pi_{i,j})\}_{j\in[n], j\neq i}$

Broadcast bmsg_i

$\mathsf{Round}_1(n, t, i, \mathsf{sk}_i^I, \{(j, \mathsf{PK}_j^I)\}_{j\in[n]}, \{\mathsf{bmsg}_j\}_{j\in[n]}, \mathsf{st}_i)$

$\bar{x}_{i,i} \leftarrow \mathsf{st}_i$

for $j \in [n], j \neq i,$ **do**

  Parse $\{(\mathsf{msg}_j, \bar{C}_j, \sigma_{j,k}, \pi_{j,k})\}_{k\in[n],k\neq j} \leftarrow \mathsf{bmsg}_j$

  Parse $(A_{j,0}, \ldots, A_{j,t-1}) \leftarrow \bar{C}_j$; $(R_{j,k}, z_{j,k}) \leftarrow \sigma_{j,k}, k \in [n], k \neq j$

for $j \in [n], j \neq i,$ **do**

  for $k \in [n], k \neq j,$ **do**

    return $\perp$ **if** $\mathsf{T\text{-}eVRF.Verify}(\mathsf{PK}_j^I, \mathsf{PK}_k^I, \mathsf{msg}_j, R_{j,k}, \pi_{j,k}) \neq 1$

    $\bar{X}_{j,k} \leftarrow \prod_{\ell=0}^{t-1} A_{j,\ell}^{k^\ell}$ // *$\bar{X}_{j,k}$ is the commitment $g^{f_j(k)}$*

    return $\perp$ **if** $g^{z_{j,k}} \neq R_{j,k} \cdot \bar{X}_{j,k}$ // *Check the ciphertext is valid; $g \in \mathbb{G}_{\mathsf{out}}$*

for $j \in [n], j \neq i,$ **do**

  $(r_{j,i}, \cdot, \cdot) \leftarrow \mathsf{T\text{-}eVRF.Evaluate}(\mathsf{sk}_i^I, \mathsf{PK}_j^I, \mathsf{msg}_j)$ // *Does not require $(R_{j,i}, \pi_{j,i})$*

  $\bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i}$ // *Decrypt secret share from participant j*

$\mathsf{sk}_i \leftarrow \sum_{j=1}^n \bar{x}_{j,i}$ // *Sum of all shares $f_j(i)$*

$\mathsf{PK}_\ell \leftarrow \prod_{k=1}^n \bar{X}_{k,\ell}$ for $\ell \in [n]$ // *Equal to g^{sk_ℓ}*

$\mathsf{PK} \leftarrow \prod_{k=1}^n A_{k,0}$ // *Equal to $g^{\sum_{k=1}^n \omega_k}$*

return $(\mathsf{PK}, \{\mathsf{PK}_j\}_{j=1}^n, \mathsf{sk}_i)$

Fig. 4: Golden, a non-interactive distributed key generation protocol that requires only one round of interaction. PK^I denotes identity public keys. The two-party eVRF T-eVRF is defined in Section 4, and Feldman in Section 3.2. Note we require $\mathsf{PK}_i^I \in \mathbb{G}_{\mathsf{in}}$ for $i \in [n]$, and $g \in \mathbb{G}_{\mathsf{out}}$. We discuss the types of all elements in Section 6.1.

After receiving and parsing bmsg_j, from all other participants $j \in [n], j \neq i$, each (receiving) participant i verifies that the received two-party eVRF proofs are valid for all participants $k \in [n], k \neq j$, by checking the relation $\mathsf{T\text{-}eVRF.Verify}(\mathsf{PK}_j^I, \mathsf{PK}_k^I, \mathsf{msg}_j, R_{j,k}, \pi_{j,k}) = 1$, if the check does not hold, the participant aborts. Because each bmsg_j is sent over a broadcast channel, if this check fails for any one participant, it will likewise fail for others.

Next, each participant derives $\bar{X}_{j,k} \leftarrow \prod_{\ell=0}^{t-1} A_{j,\ell}^{k^\ell}$, where $\bar{X}_{j,k} = g^{\bar{x}_{j,k}} = g^{f_j(k)}$ if the protocol was followed correctly. They then check the validity of the ciphertext, by checking the relation $g^{z_{j,k}} \stackrel{?}{=} R_{j,k} \cdot \bar{X}_{j,k}$, aborting if the check fails. Again, because all values are sent over broadcast, if any participant aborts, all other participants will abort.

Next, each participant i will decrypt their shares, by first deriving $(r_{j,i}, \cdot, \cdot) \leftarrow \mathsf{T\text{-}eVRF.Evaluate}(\mathsf{sk}_i^I, \mathsf{PK}_j^I, \mathsf{msg}_j)$, where j is the sending party and i is the receiving party. Note that unlike in the first round of the protocol, this step does not require the commitment $R_{j,i}$ or the proof $\pi_{j,i}$, and so can be performed more efficiently. Then, each participant will decrypt their secret share from each participant $j \neq i$ by performing $\bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i}$.

Finally, each participant obtains their secret key share sk_i by deriving $\mathsf{sk}_i \leftarrow \sum_{j=1}^n \bar{x}_{j,i}$ using the shares sent from all parties. Additionally, participants derive the public key shares $\mathsf{PK}_i \in \mathbb{G}_{\mathsf{out}}$ for all participants $i \in [n]$, using the commitments $\bar{X}_{j,k} = g^{\bar{x}_{j,k}} = g^{f_j(k)}$. Participants derive the public key using the commitments $A_{i,0}$ via $\mathsf{PK} = \prod_{i=1}^n A_{i,0} = g^{\sum_{i=1}^n f_i(0)} = g^{\sum_{i=1}^n \omega_i} \in \mathbb{G}_{\mathsf{out}}$.

Correctness. Golden is correct assuming T-eVRF is correct and Feldman secret sharing (which builds upon Shamir secret sharing) is correct.

Firstly, when party i generates $(r_{i,j}, R_{i,j}, \pi_{i,j}) \leftarrow \mathsf{T\text{-}eVRF.Evaluate}(\mathsf{sk}_i^I, \mathsf{PK}_j^I, \mathsf{msg}_i)$ for party j, if T-eVRF is correct, then party j will obtain $(r'_{j,i}, R'_{j,i}, \pi'_{j,i}) \leftarrow \mathsf{T\text{-}eVRF.Evaluate}(\mathsf{sk}_j^I, \mathsf{PK}_i^I, \mathsf{msg}_i)$ with msg_i from party i, such that $r'_{j,i} = r_{i,j}$.

For this reason, after deriving $r_{j,i}$, each party i will recover the correct share $\bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i}$. Recall that each received secret share $\bar{x}_{j,k}, k \in [n]$ is a secret share of party j's secret ω_j, such that $\omega_j = f_j(0)$ and each share $\bar{x}_{j,k} = f_j(k)$, where f_j is a random $(t-1)$-degree polynomial defined by the Feldman verifiable secret sharing algorithm Feldman.Share. As such, after each party derives their secret key share sk_i via $\mathsf{sk}_i \leftarrow \sum_{j=1}^n \bar{x}_{j,i}$, the resulting secret key sk is the following: $\mathsf{sk} = \sum_{i \in C} \mathsf{sk}_i L_i(0)$, for all $C \subseteq [n]$, such that $|C| \geq t$. As such, $\mathsf{sk} = \sum_{i \in C} \mathsf{sk}_i L_i(0) = \sum_{i \in C} \sum_{j=1}^n \bar{x}_{j,i} L_i(0)$ and so by polynomial interpolation of Feldman secret sharing, $\mathsf{sk} = \sum_{i=1}^n \omega_i$. Furthermore, $\mathsf{PK} = \prod \mathsf{PK}_i = g^{sk}$, because $\mathsf{PK} \leftarrow \prod_{k=1}^n A_{k,0} = g^{\sum_{i=1}^n \omega_i} = g^{\mathsf{sk}}$. Similarly, each participant's public key PK_i is likewise derived by performing "interpolation in the exponent," via $\mathsf{PK}_i \leftarrow \prod_{k=1}^n \prod_{j=0}^{t-1} A_{k,j}^{i^j} = g^{\sum_{j=1}^n f_j(i)} = g^{\mathsf{sk}_i}$.

Key Resharing We note that similarly to other distributed key generation schemes, Golden can be adapted to support distributed key resharing, where participants rotate their secret shares sk_i, but where the group keys sk and PK remain the same. The common technique to doing so is using zero secret sharing,

Participants	Comm. (unopt.)	Comm. (opt.)	Runtime (unopt.)	Runtime (opt.)
2	1.7 kb	1.7 kb	0.4 s	0.4 s
10	126 kb	22 kb	9.3 s	2.4 s
50	3.7 MB	223 kb	209 s	13.5 s
100	15.1 MB	699 kb	823 s	35.8 s

Table 5: Communication and estimated runtime for both the optimized and unoptimized variant of Golden. We consider running the DKG in an n-of-n configuration. Comm. represents the bandwidth overhead for each participant; kb represents kilobytes, and MB represents megabytes. Runtime represents the running time for each participant of performing both Round 0 and Round 1. Unopt. represents the unoptimized variant of Golden as presented in Figure 4, and Opt. represents the optimized variant discussed in Section 6.2.

where participants perform the protocol as defined in Figure 4, with the following changes. First, instead of sampling ω_i at random, each participant instead sets $\omega_i = 0$. Second, each participant will perform one additional check in Round_1, by checking that $A_{j,0}$ equals the identity of the group, for all $j \in [n], j \neq i$ (thereby checking that $f_j(0) = 0$).

6.2 Optimized Golden via Batching

We next review an optimized variant of Golden, which uses batching techniques to minimize communication and computational overhead. We present Golden in Figure 4 without these optimizations for simplicity and ease of understanding. We calculate our performance estimates with respect to the curve BLS12-381, for closest comparison to the Groth DKG implementation [47, 52].

Batch Proving and Verification. As aforementioned, batch proving allows each participant to generate one (as opposed to $n-1$) proof showing correctness of $n-1$ two-party eVRF outputs. Hence, each participant verifies $n-1$ proofs from others as opposed to $(n-1)^2$ proofs. Below is a summary of the optimizations.

1. While the statement size of the proof naturally scales linearly, the size of the proof itself scales logarithmically in n, e.g. for 100 participants instead of generating 99 proofs of size 1.5 kb each, a prover sends only one proof of size 2.2 kb. For a DKG, the communication overhead is in fact dominated by each participant sending and receiving the $\bar{C}_i$ values, not the proofs.
2. The constant for the linear scaling of the proof's statement size (number of constraints) actually reduces by about 29%. As noted before, the reason is that some of the gadgets can be re-used for each additional $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ statement due to the fact that a prover uses the same PKI key pair to compute $n-1$ two-party eVRF evaluations. Namely, $4\lambda + 4$ constraints out of $14\lambda + 14$ can be re-used.

3. Batch proving also benefits from computation time that is in fact slightly sublinear (by a logarithmic factor) in the statement size [68].
4. Batch verification [15] enables sharing the most expensive operation (MSM) across all $n-1$ proofs. Checking 1 proof (for 99 two-party eVRF outputs) might take 4.8 seconds, but checking 99 proofs only takes 22.1 seconds.

In Table 5, we present the overall optimized and unoptimized runtime and communication complexity of Golden for different numbers of participants.

7 Security

We prove security for Golden as presented in Section 6, with respect to the DKG $\mathcal{F}_{\mathsf{KeyGen}}^{\mathsf{B}}$ ideal functionality given in Figure 1, and an idealized zero-knowledge proof functionality $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ (described in Appendix I.1) for the relation $\mathcal{R}_{pok} = \{(g, \mathsf{PK}, (\mathsf{sk})) : \mathsf{PK} = g^{\mathsf{sk}}\}$. Our DKG assumes the security of a two-party eVRF T-eVRF, as defined in Section 4.

We first define Golden completely, in the setting where parties establish the validity of their public keys during an initialize phase. In practice, parties can interact with a PKI that likewise provides this functionality. We call this full, end-to-end protocol $\Pi_{\mathsf{Golden\text{-}PKI}}$, which is simply Golden, conjoined with an initialize function, defined as follows:

Protocol 1 ($\Pi_{\mathsf{Golden\text{-}PKI}}$) : **Initialize – for all n parties:**

1. Message 1: Party i receives as input (Init, i)
   (a) Perform $\mathsf{T\text{-}eVRF.KeyGen}() \to (\mathsf{sk}_i, \mathsf{PK}_i)$.
   (b) Send $(\mathsf{Prove}, i, j, \mathsf{sk}_i^I, \mathsf{PK}_i^I)$ to $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ for all $j \in [n]$.
   (c) Send $(\mathsf{Init}, i, \mathsf{PK}_i)$ to all other parties, using PK_i as the session identifier sid_i.
2. Message 2: Party i receives as input $(\mathsf{Init}, j, \mathsf{PK}_j)$:
   (a) If $(\mathsf{Prove}, i, j, \mathsf{PK}_i^I)$ is received from $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$, then proceed, otherwise, ignore.
   (b) Send $(\mathsf{Init}, j, \mathsf{PK}_j)$ to all other parties.
3. Final Step: Receive as input $(\mathsf{Init}, k_j, \mathsf{PK}_k^j)$ for all $j \in [n]$ and $k \in [n]$:
   (a) If the same message $(\mathsf{Init}, k, \mathsf{PK}_k)$ is received from all parties, store $(\mathsf{Init}, k, \mathsf{PK}_k)$, otherwise ignore.

Theorem 3. *$\Pi_{\mathsf{Golden\text{-}PKI}}$ securely realizes $\mathcal{F}_{\mathsf{KeyGen}}^{\mathsf{B}}$ in the $(\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}, \mathcal{F}_{\mathsf{T\text{-}eVRF}})$-hybrid model, with respect to a static adversary $\mathcal{A}$ that corrupts up to $t-1$ number of parties.*

We give the full proof for Theorem 3 in Appendix J.

Proof Intuition. The core of the proof lies in defining a simulating algorithm SIM which interacts with the adversary $\mathcal{A}$ controlling $t-1$ corrupt parties, and simulates honest parties in $\Pi_{\mathsf{Golden\text{-}PKI}}$ with respect to a challenge polynomial commitment $(Y_i)_{i=0}^{t-1} \in \mathbb{G}_{\mathsf{out}}^t$ given by $\mathcal{F}_{\mathsf{KeyGen}}^{\mathsf{B}}$. SIM follows the protocol honestly for all honest parties except for one simulated party $\tau \in \mathsf{hon}$.

For party τ, SIM simulates Shamir secret sharing with respect to $g^{f_\tau(0)} = A_{\tau,0} = Y \cdot \prod_{k \in \mathsf{hon}, k \neq \tau} (A_{k,0})^{-1}$, setting the $t-1$ corrupt parties' shares $\bar{x}_{\tau,j}$ with respect to the ideal functionality inputs $f^\dagger(j), j \in \mathsf{corr}$, and the other honest parties' shares. SIM defines f_τ "in the exponent" as the $(t-1)$-degree polynomial defined by the t points $f_\tau(j) = \bar{x}_{\tau,j}$ and $A_{\tau,0}$. SIM then follows the protocol honestly to define $(R_{\tau,j}, z_{\tau,j})$ for $j \in \mathsf{corr}$. However, SIM must also simulate $(R_{\tau,k}, z_{\tau,k})$ for all $k \in \mathsf{hon}, k \neq \tau$ (even though SIM does not know $\bar{x}_{\tau,k}, k \in \mathsf{hon}, k \neq \tau$). SIM does so by deriving the correct commitment to $g^{\bar{x}_{\tau,k}}$, by deriving $g^{\bar{x}_{\tau,k}} = \bar{X}_{\tau,k} = A_{\tau,0} \cdot \prod_{j=1}^{t-1} A_{\tau,j}^{k^j}$ via "interpolation in the exponent," where $A_{\tau,j}$ is the VSS commitment to the j^{th} coefficient of the polynomial f_τ. SIM then samples $z_{\tau,k} \leftarrow\!\!\$\, \mathbb{Z}_p$ for all $k \in \mathsf{hon}, k \neq \tau$, and sets $R_{\tau,k} = g^{z_{\tau,k}} \cdot \bar{X}_{\tau,k}$. The simulation is perfect due to the security of $\mathsf{T\text{-}eVRF\text{-}DH}$, and because $z_{\tau,k}$ is sampled uniformly at random, so is likewise the product $g^{z_{\tau,k}} \cdot \bar{X}_{\tau,k}$.

After Round 0 is completed, because SIM knows corrupt parties' identity keys $\{\mathsf{sk}_j^I\}_{j \in \mathsf{corr}}$ from the initialize step for the PKI, SIM can decrypt and derive all corrupt parties' shares to all other parties $\{\bar{x}_{i,j}\}, i \in \mathsf{corr}, j \in [n]$. SIM can then derive each corrupt party's $i \in \mathsf{corr}$ sharing polynomial $f_i(x) = \omega_j + a_{j,1}x + a_{j,2}x^2 + a_{j,t-1}x^{t-1}$ using the decrypted shares. Specifically, to derive ω_j for each $j \in \mathsf{corr}$, SIM can simply perform $\omega_j \leftarrow \mathsf{Feldman.Recover}(t, \{(i, \bar{x}_{j,i})\}_{i \in [n]})$, and using similar techniques for each remaining coefficient $a_{j,\ell}, \ell \in \{1, \ldots, t-1\}$. Then, Δ_0 is derived by performing $\Delta_0 = \sum_{i \in \mathsf{corr}} \omega_i$, and each remaining $\Delta_\ell = \sum_{i \in \mathsf{corr}} a_{i,\ell}$.

Due to how $A_{\tau,0}$ is defined, honest parties' contributions to the DKG satisfy $\sum_{k \in \mathsf{hon}} \omega_k = \mathsf{dlog}(Y)$, Hence, $\mathsf{PK} = Y \cdot \prod_{j \in \mathsf{corr}} g^{\omega_j} = Y \cdot g^{\Delta}$. Similar reasoning applies for each $\mathsf{PK}_i, i \in \mathsf{hon}$, thus satisfying the requirements for $\mathcal{F}_{\mathsf{KeyGen}}^{\mathsf{B}}$.

Acknowledgments

The authors would like to thank Faxing Wang for feedback. This work was supported by NSF grant 2239975 and generous gifts from Alpen Labs, Chaincode, and Google. Many thanks to Francois Garillot for motivating this work and challenging us to investigate the feasibility of improved round-optimal DKGs.

References

[1] M. Abe and S. Fehr. "Adaptively Secure Feldman VSS and Applications to Universally-Composable Threshold Cryptography". In: *CRYPTO 2004*. Ed. by M. K. Franklin. Vol. 3152. LNCS. Springer, 2004, pp. 317–334.

[2] T. Attema, S. Fehr, and M. Klooß. "Fiat-shamir transformation of multi-round interactive proofs". In: *Theory of Cryptography Conference*. 2022.

[3] R. Bacho and A. Kavousi. "SoK: Dlog-Based Distributed Key Generation". In: *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*. Ed. by M. Blanton, W. Enck, and C. Nita-Rotaru. IEEE, 2025, pp. 614–632. DOI: 10.1109/SP61157.2025.00127. URL: https://doi.org/10.1109/SP61157.2025.00127.

[4] A. Bassa. *Soundness Proof for Eagen's Proof of Sums of Points*. https://repo.getmonero.org/-/project/54/uploads/eb1bf5b4d4855a3480c38abf895bd8e8/Veridise_Divisor_Proofs.pdf. 2022.

[5] M. Bellare, E. C. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu. "Better than Advertised Security for Non-interactive Threshold Signatures". In: *CRYPTO 2022*. Ed. by Y. Dodis and T. Shrimpton. Vol. 13510. LNCS. Springer, 2022, pp. 517–550.

[6] M. Bellare and G. Neven. "Multi-signatures in the plain public-key model and a general forking lemma". In: *Proceedings of the 13th ACM conference on Computer and communications security*. 2006, pp. 390–399.

[7] D. J. Bernstein. "Curve25519: New Diffie–Hellman Speed Records". In: *Public Key Cryptography - PKC 2006*. Ed. by M. Yung, Y. Dodis, A. Kiayias, and T. Malkin. Vol. 3958. LNCS. Springer, 2006, pp. 207–228.

[8] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. "From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again". In: *Innovations in Theoretical Computer Science 2012, Cambridge, MA, USA, January 8-10, 2012*. Ed. by S. Goldwasser. ACM, 2012, pp. 326–349. DOI: 10.1145/2090236.2090263. URL: https://doi.org/10.1145/2090236.2090263.

[9] A. Boldyreva. "Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme". In: *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*. Ed. by Y. Desmedt. Vol. 2567. Lecture Notes in Computer Science. Springer, 2003, pp. 31–46. DOI: 10.1007/3-540-36288-6_3. URL: https://doi.org/10.1007/3-540-36288-6_3.

[10] D. Boneh, T. Datta, R. Fernando, K. Nazirkhanova, and A. Tomescu. "Dekart-Proof: Efficient Vector Range Proofs and Their Applications". In: *Cryptology ePrint Archive* (2025).

[11] D. Boneh, M. Drijvers, and G. Neven. "Compact multi-signatures for smaller blockchains". In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2018, pp. 435–464.

[12] D. Boneh, I. Haitner, Y. Lindell, and G. Segev. "Exponent-VRFs and Their Applications". In: *EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part VII*. Ed. by S. Fehr and P. Fouque. Vol. 15607. Lecture Notes in Computer Science. Springer, 2025, pp. 195–224. DOI: 10.1007/978-3-031-91098-2_8. URL: https://doi.org/10.1007/978-3-031-91098-2_8.

[13] L. Brandão and R. Peralta. *NIST First Call for Multi-Party Threshold Schemes*. https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8214C.ipd.pdf. 2023.

[14] B. Bünz. *Improving the Privacy Scalability and Ecological Impact of Blockchains*. Stanford University, 2023.

[15] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. "Bulletproofs: Short proofs for confidential transactions and more". In: *2018 IEEE symposium on security and privacy (SP)*. IEEE. 2018, pp. 315–334.

[16] R. Canetti, R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. "Adaptive Security for Threshold Cryptosystems". In: *CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999*. Ed. by M. J. Wiener. Vol. 1666. LNCS. Springer, 1999, pp. 98–115.

[17] R. Canetti and T. Rabin. "Universal Composition with Joint State". In: *CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*. Ed. by D. Boneh. Vol. 2729. Lecture Notes in Computer Science. Springer, 2003, pp. 265–281. DOI: 10.1007/978-3-540-45146-4_16. URL: https://doi.org/10.1007/978-3-540-45146-4_16.

[18] I. Cascudo and B. David. "Publicly Verifiable Secret Sharing Over Class Groups and Applications to DKG and YOSO". In: *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part V*. Ed. by M. Joye and G. Leander. Vol. 14655. Lecture Notes in Computer Science. Springer, 2024, pp. 216–248. DOI: 10.1007/978-3-031-58740-5_8. URL: https://doi.org/10.1007/978-3-031-58740-5_8.

[19] I. Cascudo and B. David. "SCRAPE: Scalable Randomness Attested by Public Entities". In: *Applied Cryptography and Network Security (ACNS) 2017*. Ed. by D. Gollmann, A. Miyaji, and H. Kikuchi. Vol. 10355. LNCS. Springer, 2017, pp. 537–556.

[20] I. Cascudo, B. David, O. Shlomovits, and D. Varlakov. "Mt. Random: Multi-tiered Randomness Beacons". In: *Applied Cryptography and Network Security - 21st International Conference, ACNS 2023, Kyoto, Japan, June 19-22, 2023*. Ed. by M. Tibouchi and X. Wang. Vol. 13906. LNCS. Springer, 2023, pp. 645–674. DOI: 10.1007/978-3-031-33491-7_24. URL: https://doi.org/10.1007/978-3-031-33491-7_24.

[21] D. Cash, E. Kiltz, and V. Shoup. "The Twin Diffie–Hellman Problem and Applications". In: *EUROCRYPT 2008*. Ed. by N. P. Smart. Vol. 4965. LNCS. Springer, 2008, pp. 127–145.

[22] D. Chaum and T. P. Pedersen. "Wallet databases with observers". In: *Annual international cryptology conference*. 1992.

[23] M. Chen, P. Dey, C. Ganesh, P. Mukherjee, P. Sarkar, and S. Sasmal. "Universally Composable Non-interactive Zero-Knowledge from Sigma Protocols via a New Straight-Line Compiler". In: *Public-Key Cryptography - PKC 2025 - 28th IACR International Conference on Practice and Theory of Public-Key Cryptography, Røros, Norway, May 12-15, 2025, Proceedings, Part I*. Ed. by T. Jager and J. Pan. Vol. 15674. LNCS. Springer, 2025, pp. 381–417. DOI: 10.1007/978-3-031-91820-9_13. URL: https://doi.org/10.1007/978-3-031-91820-9_13.

[24] K. Choi, A. Manoj, and J. Bonneau. "Sok: Distributed randomness beacons". In: *2023 IEEE Symposium on Security and Privacy (SP)*. 2023.

[25] H. Chu, P. Gerhart, T. Ruffing, and D. Schröder. "Practical Schnorr Threshold Signatures Without the Algebraic Group Model". In: *CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023*. Ed. by H. Handschuh and A. Lysyanskaya. Vol. 14081. Lecture Notes in Computer Science. Springer, 2023, pp. 743–773. DOI: 10.1007/978-3-031-38557-5_24. URL: https://doi.org/10.1007/978-3-031-38557-5_24.

[26] H. Chung, K. Han, C. Ju, M. Kim, and J. H. Seo. "Bulletproofs+: Shorter proofs for a privacy-enhanced distributed ledger". In: *Ieee Access* (2022).

[27] arkworks contributors. *arkworks zkSNARK ecosystem*. 2022. URL: https://arkworks.rs.

[28] R. Cramer, I. Damgård, and Y. Ishai. "Share Conversion, Pseudorandom Secret-Sharing and Applications to Secure Computation". In: *Theory of Cryptography, Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005, Proceedings*. Ed. by J. Kilian. Vol. 3378. Lecture Notes in Computer Science. Springer, 2005, pp. 342–362. DOI: 10.1007/978-3-540-30576-7_19. URL: https://doi.org/10.1007/978-3-540-30576-7_19.

[29] E. Crites, C. Komlo, and M. Maller. *How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures*. Cryptology ePrint Archive, Paper 2021/1375. 2021. URL: https://eprint.iacr.org/2021/1375.

[30] E. Crites, C. Komlo, and M. Maller. *On the Adaptive Security of Key-Unique Threshold Signatures*. Cryptology ePrint Archive, Paper 2025/943. 2025. URL: https://eprint.iacr.org/2025/943.

[31] S. Das, L. Ren, and Z. Yang. "Adaptively Secure Threshold ElGamal Decryption from DDH". In: *IACR Cryptol. ePrint Arch.* (2025).

[32] Y. Desmedt. "Threshold cryptosystems". In: *international workshop on the theory and application of cryptographic techniques*. 1992.

[33] DFINITY. *Applied Crypto: Introducing Noninteractive Distributed Key Generation*. https://medium.com/dfinity/applied-crypto-one-public-key-for-the-internet-computer-ni-dkg-4af800db869d. 2025.

[34] Y. Dodis and A. Yampolskiy. "A verifiable random function with short proofs and keys". In: *International Workshop on Public Key Cryptography*. 2005.

[35] L. Eagen. "Zero knowledge proofs of elliptic curve inner products from principal divisors and weil reciprocity". In: *Cryptology ePrint Archive* (2022).

[36] J. Ernstberger et al. "zk-Bench: A Toolset for Comparative Evaluation and Performance Benchmarking of SNARKs". In: *Security and Cryptography for Networks*. Ed. by C. Galdi and D. H. Phan. Springer Nature Switzerland, 2024, pp. 46–72. ISBN: 978-3-031-71070-4.

[37] P. Feldman. "A Practical Scheme for Non-interactive Verifiable Secret Sharing". In: *28th Annual Symposium on Foundations of Computer Science*. 1987.

[38] H. Feng, T. Mai, and Q. Tang. "Scalable and Adaptively Secure Any-Trust Distributed Key Generation and All-hands Checkpointing". In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*. Ed. by B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie. ACM, 2024, pp. 2636–2650. DOI: 10.1145/3658644.3690253. URL: https://doi.org/10.1145/3658644.3690253.

[39] A. Fiat and A. Shamir. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems". In: *CRYPTO '86*. Vol. 263. LNCS. Springer, 1986, pp. 186–194.

[40] M. Fischlin. "Communication-Efficient Non-interactive Proofs of Knowledge with Online Extractors". In: *CRYPTO 2005*. Ed. by V. Shoup. Vol. 3621. LNCS. Springer, 2005, pp. 152–168.

[41] P. Fouque and J. Stern. "One Round Threshold Discrete-Log Key Generation without Private Channels". In: *Public Key Cryptography, 4th International Workshop on Practice and Theory in Public Key Cryptography, PKC 2001, Cheju Island, Korea, February 13-15, 2001, Proceedings*. Ed. by K. Kim. Vol. 1992. Lecture Notes in Computer Science. Springer, 2001, pp. 300–316. DOI: 10.1007/3-540-44586-2_22. URL: https://doi.org/10.1007/3-540-44586-2_22.

[42] E. S. V. Freire, D. Hofheinz, E. Kiltz, and K. G. Paterson. "Non-Interactive Key Exchange". In: *Public-Key Cryptography - PKC 2013*. Ed. by K. Kurosawa and G. Hanaoka. Vol. 7778. LNCS. Springer, 2013, pp. 254–271.

[43] S. D. Galbraith, P. Wang, and F. Zhang. *Computing Elliptic Curve Discrete Logarithms with Improved Baby-step Giant-step Algorithm*. Cryptology ePrint Archive, Paper 2015/605. 2015. URL: https://eprint.iacr.org/2015/605.

[44] R. Gao, Z. Wan, Y. Hu, and H. Wang. "A succinct range proof for polynomial-based vector commitment". In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2024.

[45] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. "Secure Applications of Pedersen's Distributed Key Generation Protocol". In: *CT-RSA 2003*. 2003, pp. 373–390. ISBN: 978-3-540-36563-1.

[46] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. "Secure Distributed Key Generation for Discrete-Log Based Cryptosystems". In: *J. Cryptol.* 20.1 (2007), pp. 51–83.

[47] J. Groth. *Non-interactive distributed key generation and key resharing*. Cryptology ePrint Archive, Paper 2021/339. 2021. URL: https://eprint.iacr.org/2021/339.

[48] K. Gurkan, P. Jovanovic, M. Maller, S. Meiklejohn, G. Stern, and A. Tomescu. "Aggregatable Distributed Key Generation". In: *EUROCRYPT 2021*. Ed. by A. Canteaut and F. Standaert. Vol. 12696. LNCS. Springer, 2021, pp. 147–176.

[49] R. Impagliazzo, L. A. Levin, and M. Luby. "Pseudo-random generation from one-way functions". In: *Proceedings of the twenty-first annual ACM symposium on Theory of computing*. 1989, pp. 12–24.

[50] S. Jarecki, H. Krawczyk, and J. Resch. "Threshold Partially-Oblivious PRFs with Applications to Key Management". In: (2018). URL: https://eprint.iacr.org/2018/733.

[51] S. Jarecki and A. Lysyanskaya. "Adaptively Secure Threshold Cryptography: Introducing Concurrency, Removing Erasures". In: *Advances in Cryptology - EUROCRYPT 2000, International Conference on the Theory and Application of Cryptographic Techniques, Bruges, Belgium, May 14-18, 2000, Proceeding*. Ed. by B. Preneel. Vol. 1807. Lecture Notes in Computer Science. Springer, 2000, pp. 221–242. DOI: 10.1007/3-540-45539-6_16. URL: https://doi.org/10.1007/3-540-45539-6_16.

[52] A. Kate, E. V. Mangipudi, P. Mukherjee, H. Saleem, and S. A. K. Thyagarajan. "Non-interactive VSS using Class Groups and Application to DKG". In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 2024, pp. 4286–4300.

[53] A. Kate, P. Mukherjee, P. Sarkar, H. Saleem, N. Shrestha, and D. Yang. *Scalable Distributed Key Generation for Blockchains*. Cryptology ePrint Archive, Paper 2026/072. 2026. URL: https://eprint.iacr.org/2026/072.

[54] J. Katz. "Round Optimal Robust Distributed Key Generation". In: *IACR Cryptol. ePrint Arch.* (2023), p. 1094. URL: https://eprint.iacr.org/2023/1094.

[55] A. Kavousi, Z. Wang, and P. Jovanovic. "SoK: public randomness". In: *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*. 2024.

[56] C. Komlo and I. Goldberg. "FROST: Flexible Round-Optimized Schnorr Threshold Signatures". In: *Selected Areas in Cryptography - SAC 2020*. Ed. by O. Dunkelman, M. J. J. Jr., and C. O'Flynn. Vol. 12804. LNCS. Springer, 2020, pp. 34–65.

[57] E. Kushilevitz, Y. Lindell, and T. Rabin. “Information-Theoretically Secure Protocols and Security under Composition”. In: *SIAM J. Comput.* 39.5 (2010), pp. 2090–2112.

[58] League of Entropy. *Distributed randomness beacon*. `https://drand.love/`. 2019.

[59] Y. Lindell. *Simple Three-Round Multiparty Schnorr Signing with Full Simulatability*. Cryptology ePrint Archive, Report 2022/374. `https://ia.cr/2022/374`. 2022.

[60] H. D. V. Madrigal. *A Verifiable Encryption Scheme for the Discrete Logarithm Relation with Applications to Distributed Key Generation*. `https://github.com/serai-dex/serai/pull/681`. 2025.

[61] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song. “The Honey Badger of BFT Protocols”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*. Ed. by E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi. ACM, 2016, pp. 31–42. DOI: `10.1145/2976749.2978399`. URL: `https://doi.org/10.1145/2976749.2978399`.

[62] M. Naor, B. Pinkas, and O. Reingold. “Distributed Pseudo-random Functions and KDCs”. In: *EUROCRYPT ’99*. Ed. by J. Stern. Vol. 1592. LNCS. Springer, 1999, pp. 327–346.

[63] J. Nick, T. Ruffing, Y. Seurin, and P. Wuille. “MuSig-DN: Schnorr Multi-Signatures with Verifiably Deterministic Nonces”. In: *CCS ’20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. Ed. by J. Ligatti, X. Ou, J. Katz, and G. Vigna. ACM, 2020, pp. 1717–1731. DOI: `10.1145/3372297.3417236`. URL: `https://doi.org/10.1145/3372297.3417236`.

[64] V. Nikolaenko, S. Ragsdale, J. Bonneau, and D. Boneh. “Powers-of-Tau to the People: Decentralizing Setup Ceremonies”. In: *Applied Cryptography and Network Security - 22nd International Conference, ACNS 2024, Abu Dhabi, United Arab Emirates, March 5-8, 2024, Proceedings, Part III*. Ed. by C. Pöpper and L. Batina. Vol. 14585. Lecture Notes in Computer Science. Springer, 2024, pp. 105–134. DOI: `10.1007/978-3-031-54776-8\_5`. URL: `https://doi.org/10.1007/978-3-031-54776-8\_5`.

[65] P. Paillier. “Public-Key Cryptosystems Based on Composite Degree Residuosity Classes”. In: *Advances in Cryptology - EUROCRYPT ’99, International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2-6, 1999, Proceeding*. Ed. by J. Stern. Vol. 1592. Lecture Notes in Computer Science. Springer, 1999, pp. 223–238. DOI: `10.1007/3-540-48910-X\_16`. URL: `https://doi.org/10.1007/3-540-48910-X\_16`.

[66] L. Parker. *R1CS gadget for the 2^k-bit scaling of a fixed generator in seven multiplicative constraints*. `https://github.com/kayabaNerve/fcmp-plus-plus-paper/blob/divisor-paper/divisors.pdf`. 2024.

[67] T. P. Pedersen. “A Threshold Cryptosystem without a Trusted Party (Extended Abstract)”. In: *EUROCRYPT 1991*. Ed. by D. W. Davies. Vol. 547. LNCS. Springer, 1991, pp. 522–526.

[68] N. Pippenger. “On the evaluation of powers and monomials”. In: *SIAM Journal on Computing* (1980).

[69] T. Ruffing and J. Nick. *ChillDKG: Distributed Key Generation for FROST*. 2025. URL: `https://github.com/BlockstreamResearch/bip-frost-dkg`.

[70] B. Schoenmakers. “A Simple Publicly Verifiable Secret Sharing Scheme and Its Application to Electronic”. In: *CRYPTO ’99, 19th Annual International*

Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999. Ed. by M. J. Wiener. Vol. 1666. Lecture Notes in Computer Science. Springer, 1999, pp. 148–164. DOI: 10.1007/3-540-48405-1_10. URL: https://doi.org/10.1007/3-540-48405-1_10.

[71] G. Segev. "Bulletproofs for R1CS: Bridging the Completeness-Soundness Gap and a ZK Extension". In: *Cryptology ePrint Archive* (2025).

[72] E. Syta et al. "Scalable Bias-Resistant Distributed Randomness". In: *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*. IEEE Computer Society, 2017, pp. 444–460. DOI: 10.1109/SP.2017.45. URL: https://doi.org/10.1109/SP.2017.45.

[73] N. Trapp and D. Ongaro. *Juicebox Protocol: Distributed Storage and Recovery of Secrets Using Simple PIN Authentication*. Cryptology ePrint Archive, Paper 2025/348. 2025. URL: https://eprint.iacr.org/2025/348.

[74] F. Wang, S. Cohney, and J. Bonneau. "SoK: Trusted setups for powers-of-tau strings". In: *International Conference on Financial Cryptography and Data Security*. 2025.

A Exponent Verifiable Random Functions

We recall the definition and notion of security of exponent verifiable random functions (eVRFs) as presented by Boneh et al. [12].

Definition 4. *Let $\{(\mathcal{X}_\lambda, \mathcal{Y}_\lambda)\}$ be an ensemble of domains and ranges, and $\mathcal{Y}_\lambda$ be a finite cyclic group with generator g. An* **exponent verifiable random function (eVRF)** *with respect to $\{(\mathcal{X}_\lambda, \mathcal{Y}_\lambda)\}$ is represented by the following tuple of algorithms:*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{par}$*: Accepts as input the security parameter λ, and outputs public parameters* par*, which are implicitly given as input to all other algorithms.*
- $\mathsf{KeyGen}() \to (\mathsf{sk}, \mathsf{PK})$*: Outputs a secret key* sk *and corresponding verification key* PK*.*
- $\mathsf{Evaluate}(\mathsf{sk}, \mathsf{msg}) \to (\alpha, A, \pi)$*: Accepts as input a secret key* sk *and input to evaluate* $\mathsf{msg} \in \mathcal{X}_\lambda$*, and outputs the evaluation output α, a commitment $A = g^\alpha \in \mathcal{Y}_\lambda$, and proof π that the evaluation was performed correctly.*
- $\mathsf{Verify}(\mathsf{PK}, \mathsf{msg}, A, \pi)$*: Accepts as input a public key* PK*, input to evaluate* $\mathsf{msg} \in \mathcal{X}_\lambda$*, a commitment $A \in \mathcal{Y}_\lambda$, and proof π. Outputs 1 to indicate that A is a valid commitment to α on input* msg*.*

A.1 Game-Based Notions of Security

An eVRF is secure if it is correct, pseudorandom, verifiable, and simulatable.

Correctness. An eVRF is *correct* if the following holds:

$$\Pr\left[\mathsf{Verify}(\mathsf{PK}, \mathsf{msg}, A, \pi) = 1\right] = 1$$

where $\mathsf{par} \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda)$, $(\mathsf{sk}, \mathsf{PK}) \leftarrow\!\!\$\ \mathsf{KeyGen}()$, $\mathsf{msg} \leftarrow\!\!\$\ \mathcal{X}_\lambda$, and $(\alpha, A, \pi) \leftarrow \mathsf{Evaluate}(\mathsf{sk}, \mathsf{msg})$.

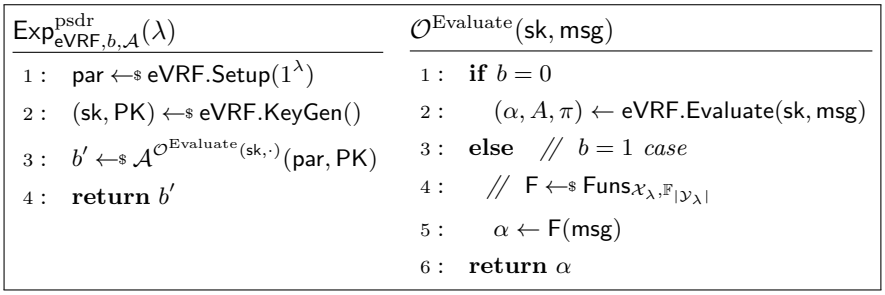

Fig. 5: Pseudorandomness experiment defining the advantage of an adversary $\mathcal{A}$ against an exponent verifiable random function (eVRF) eVRF.

Pseudorandomness. An eVRF is *pseudorandom* if for every PPT adversary $\mathcal{A}$ the following function $\mathsf{Adv}^{\mathrm{psdr}}_{\mathsf{eVRF},\mathcal{A}}(\lambda)$ is negligible (see Figure 5):

$$\mathsf{Adv}^{\mathrm{psdr}}_{\mathsf{eVRF},\mathcal{A}}(\lambda) = \left|\Pr[\mathsf{Exp}^{\mathrm{psdr}}_{\mathsf{eVRF},0,\mathcal{A}}(\lambda) = 1] - \Pr[\mathsf{Exp}^{\mathrm{psdr}}_{\mathsf{eVRF},1,\mathcal{A}}(\lambda) = 1]\right|$$

Verifiability. An eVRF is *verifiable* if for every PPT adversary $\mathcal{A}$ the following function $\mathsf{Adv}^{\mathrm{verf}}_{\mathsf{eVRF},\mathcal{A}}(\lambda)$ is negligible:

$$\mathsf{Adv}^{\mathrm{verf}}_{\mathsf{eVRF},\mathcal{A}}(\lambda) = |\Pr[A_0 \neq A_1 \wedge \mathsf{Verify}(\mathsf{PK},\mathsf{msg},A_i,\pi_i) = 1\ \forall i \in \{0,1\}]|$$

where $\mathsf{par} \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda)$ and $(\mathsf{PK},\mathsf{msg},(A_0,\pi_0),(A_1,\pi_1)) \leftarrow\!\!\$\ \mathcal{A}(\mathsf{par})$.

Simulatability. An eVRF is *simulatable* if there exists a PPT algorithm SIM such that for every PPT distinguishing algorithm $\mathcal{D}$, the following function $\mathsf{Adv}^{\mathrm{sim}}_{\mathsf{eVRF},\mathcal{D},\mathsf{SIM}}(\lambda)$ is negligible:

$$\mathsf{Adv}^{\mathrm{sim}}_{\mathsf{eVRF},\mathcal{D},\mathsf{SIM}}(\lambda) = \left|\Pr[\mathcal{D}^{\mathcal{O}^{\mathsf{Evaluate}}(\mathsf{sk},\cdot)}(\mathsf{par},\mathsf{PK}) = 1] - \Pr[\mathcal{D}^{\mathsf{SIM}(\mathsf{PK},\cdot)}(\mathsf{par},\mathsf{PK}) = 1]\right|$$

where $\mathsf{par} \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda)$, $(\mathsf{sk},\mathsf{PK}) \leftarrow\!\!\$\ \mathsf{KeyGen}()$, and $\mathcal{O}^{\mathsf{Evaluate}}(\mathsf{sk},\cdot)$ returns the output of $\mathsf{Evaluate}(\mathsf{sk},\cdot)$.

A.2 Simulation-Based Notion of Security

Boneh et al. [12] show the equivalence of an eVRF proven to be secure with respect to the game-based notions of security, and a simulation-based notion of security for an idealized functionality $\mathcal{F}_{\mathsf{eVRF}}$. We provide the functionality $\mathcal{F}_{\mathsf{eVRF}}$ in Appendix D.

B Non-Interactive Key Exchange (NIKE)

Our new two-party eVRF builds upon a (two-party) Non-Interactive Key Exchange (NIKE) [7, 21, 42] scheme, as defined below. We build upon prior NIKE definitions in the literature [21, 42]; these notions allow an adversary to act as an active participant, and require it to perform a distinguishing attack on a shared key between the public keys associated with two identities of the adversary's choosing.

Definition 5. *A* **Non-Interactive Key Exchange (NIKE)** NIKE *is the tuple* $\mathsf{NIKE} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{GetSharedKey})$*, where*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{par}$*: Accepts as input the security parameter* λ*, and outputs public parameters* par*, which are implicitly given as input to all other algorithms.*
- $\mathsf{KeyGen}() \to (\mathsf{sk}, \mathsf{PK})$*: Generates a secret key* sk *and corresponding public key* PK*.*
- $\mathsf{GetSharedKey}(\mathsf{sk}_1, \mathsf{PK}_2) \to S$*: Accepts the first party's secret key* sk_1 *and the second party's public key* PK_2*. Outputs the shared key* S *generated by the combination of one party's secret key* sk_1 *and the other party's public key* PK_2*.*
- *For correctness, we require that, for all* $(\mathsf{sk}_1, \mathsf{PK}_1) \leftarrow\!\!\$\ \mathsf{NIKE.KeyGen}(\lambda)$ *and* $(\mathsf{sk}_2, \mathsf{PK}_2) \leftarrow\!\!\$\ \mathsf{NIKE.KeyGen}(\lambda)$*, the following conditions hold:*

$$\mathsf{NIKE.GetSharedKey}(\mathsf{sk}_1, \mathsf{PK}_2) = \mathsf{NIKE.GetSharedKey}(\mathsf{sk}_2, \mathsf{PK}_1)$$

In order to be secure, a NIKE must be *session-key unrecoverable*, which we discuss next.

Session-Key Unrecoverability. A NIKE which is *session-key unrecoverable* ensures that given the public keys of two parties (q_t pairs of them), it is hard for an adversary to distinguish the shared secret(s) from one(s) sampled randomly.

The advantage of an adversary $\mathcal{A}$ against NIKE in the unrecoverability experiment as defined in Figure 6 is

$$\mathsf{Adv}^{\mathrm{unrec}}_{\mathsf{NIKE},\mathcal{A}}(\lambda, q_t) = \left|\Pr[\mathsf{Exp}^{\mathrm{unrec}}_{\mathsf{NIKE},0,\mathcal{A}}(\lambda, q_t) = 1] - \Pr[\mathsf{Exp}^{\mathrm{unrec}}_{\mathsf{NIKE},1,\mathcal{A}}(\lambda, q_t) = 1]\right|$$

where q_t denotes the number of test queries made by $\mathcal{A}$.

Definition 6. *A NIKE is* **session-key unrecoverable** *if for all probabilistic polynomial time adversaries* $\mathcal{A}$*, the function* $\mathsf{Adv}^{\mathrm{unrec}}_{\mathsf{NIKE},\mathcal{A}}(\lambda, q_t)$ *is negligible.*

C Simulation-Based Security for eVRFs

We review the simulation-based notion of security for an eVRF by Boneh et al. [12], as the idealized functionality $\mathcal{F}_{\mathsf{eVRF}}$. The authors prove this functionality is in fact

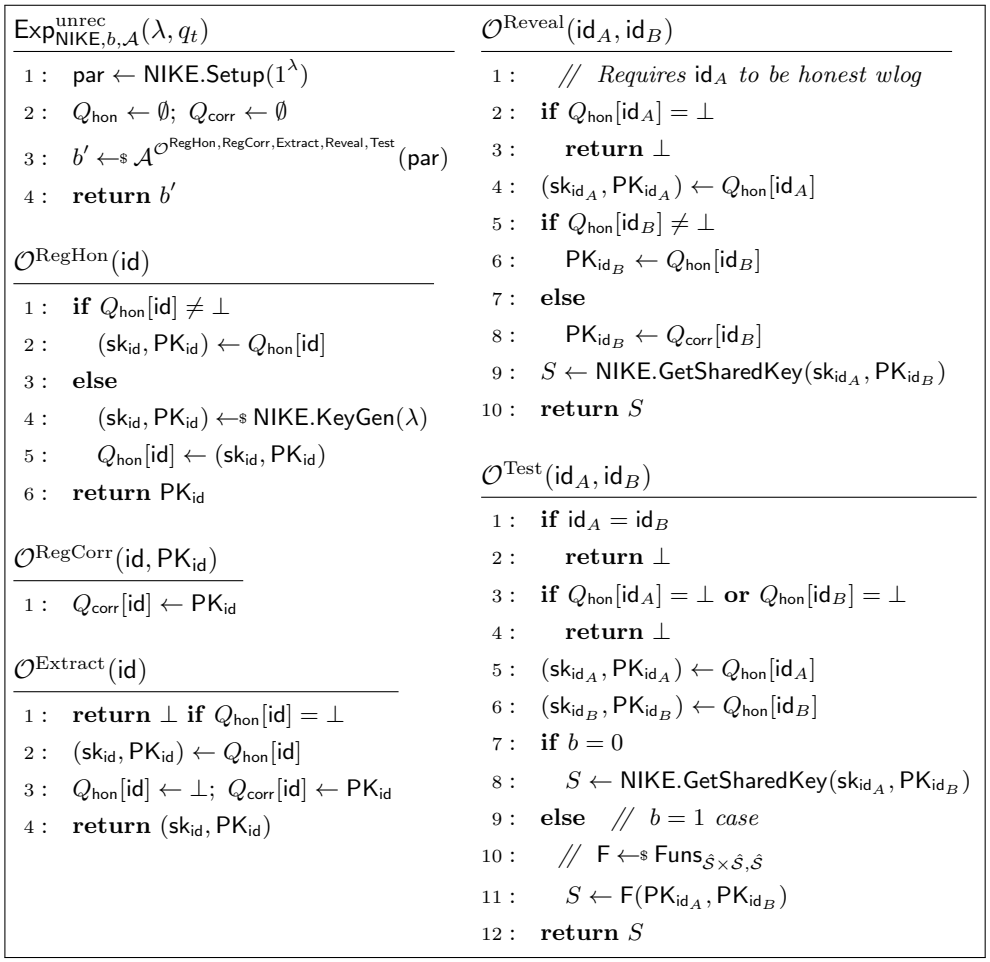

Fig. 6: Unrecoverability experiment defining the advantage of an adversary $\mathcal{A}$ against a non-interactive key exchange (NIKE) NIKE. To prevent trivial wins, the adversary cannot make queries to $\mathcal{O}^{\mathrm{Reveal}}$ with inputs to $\mathcal{O}^{\mathrm{Test}}$, and similarly, cannot make $\mathcal{O}^{\mathrm{Extract}}$ queries for identities involved in $\mathcal{O}^{\mathrm{Test}}$ queries. The number of $\mathcal{O}^{\mathrm{Test}}$ queries made by $\mathcal{A}$ is denoted by q_t, and the domain of shared keys is denoted by $\hat{\mathcal{S}}$.

implied by the eVRF game-based notions in conjunction with a zero-knowledge proof of knowledge of the private key. We show $\mathcal{F}_{\mathsf{eVRF}}$ in Figure 7.

As observed by Boneh et al. [12], the environment can distinguish between different eVRF instances by accepting as input the verification key PK as a parameter. Hence, the functionality $\mathcal{F}_{\mathsf{eVRF}}$ is presented without requiring an additional session identifier sid.

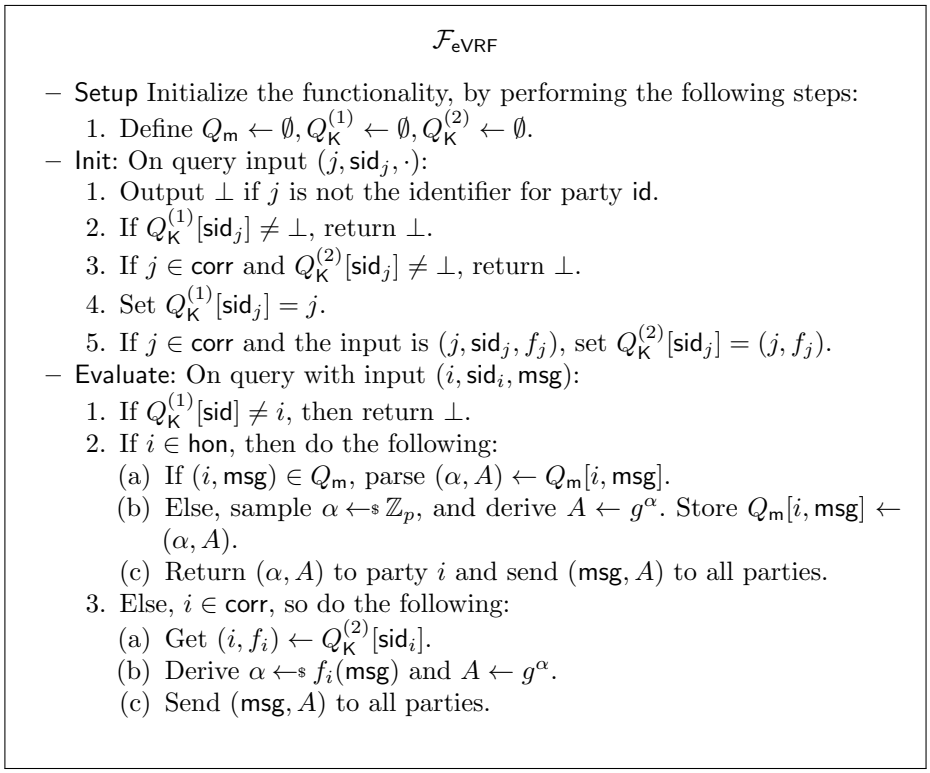

Fig. 7: Ideal functionality for an exponent VRF, as defined by Boneh et al. [12]. Note we define Query as a separate oracle to learn the registered identities for all parties, instead of broadcasting directly after registration as done by Boneh et al. [12].

D Simulation-Based Security for Two-Party eVRFs

We give the simulation-based notion of security for a two-party eVRF T-eVRF, as the idealized functionality $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$. Our notion builds on the single-party eVRF notion by Boneh et al. [12], with minor differences to account for the second party and symmetry among queries.

In particular, the differences between $\mathcal{F}_{\mathsf{eVRF}}$ and $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ largely stem from the fact that the Evaluate oracle in $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ additionally accepts as input a second party's identifier and session identifier. In the case of honest parties' queries, the functionality stores outputs with respect to both identifiers, to preserve symmetry. In the case of corrupt parties' queries, if the other identifier is an honest party, the functionality behaves in the same way was $\mathcal{F}_{\mathsf{eVRF}}$. However, if corrupt parties query Evaluate on two corrupt parties' identifiers, the functionality will generate

the two-party eVRF output using both functions provided by the corrupt parties, in sorted order (to preserve symmetry).

In [12], the authors give a proof of equivalence that the single-party eVRF functionality $\mathcal{F}_{\mathsf{eVRF}}$ is implied by the eVRF game-based notions in conjunction with a zero-knowledge proof of knowledge of the private key. We show $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ in Figure 8. We discuss a proof of equivalence in Appendix E, likewise with minor differences to the proof by Boneh et al. [12].

As observed by Boneh et al. [12], the environment can distinguish between different eVRF instances by accepting as input the verification keys $(\mathsf{PK}_i, \mathsf{PK}_j)$ as a parameter. Hence, in practice, sid_{ij} is simply the (unordered) tuple $(\mathsf{PK}_i, \mathsf{PK}_j)$.

In Figure 8, the functionality maintains state in the sets $Q_{\mathsf{m}} \leftarrow \emptyset$, $Q_{\mathsf{K}}^{(1)} \leftarrow \emptyset$, and $Q_{\mathsf{K}}^{(2)} \leftarrow \emptyset$. The set $Q_{\mathsf{m}} \leftarrow \emptyset$ maintains pairs (i, j, msg) to two-party eVRF outputs (α, A). The set $Q_{\mathsf{K}}^{(1)}$ associates the pairs (sid_j, j), where sid_j is a session identifier and j is a party identifier. The set $Q_{\mathsf{K}}^{(2)}$ is only to maintain queries from corrupt parties, and maintains the tuples (sid_j, j, f_j), where f_i, f_j are the functions that corrupt parties i, j use to derive α on the input msg.

We assume the functionality has some mechanism to authenticate queries. $\mathsf{Evaluate}$ serves simply to evaluate these function on inputs for corrupt parties, or perform the real operation for honest parties.

E Equivalence Between Game-Based Notions and Ideal Functionality for Two-Party eVRF

We next describe why the two-party eVRF that satisfies our game-based security notions also satisfies our two-party eVRF ideal functionality given in Appendix D, when all parties additionally prove knowledge of their secret eVRF keys, by interacting with a zero-knowledge ideal functionality $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$. Our proof for Theorem 4 largely follows from a similar proof given by Boneh et al. [12, Theorem 2], with minor differences due to the two-party setting. We describe these differences below.

Protocol $\Pi_{\mathsf{T\text{-}eVRF\text{-}DH}}$

- Initialize – for all n parties:
  1. Message 1: Party i receives as input (Init, i)
     (a) Perform $\mathsf{T\text{-}eVRF.KeyGen}() \rightarrow (\mathsf{sk}_i, \mathsf{PK}_i)$.
     (b) Send $(\mathsf{Prove}, i, j, \mathsf{sk}_i^I, \mathsf{PK}_i^I)$ to $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ for all $j \in [n]$.
     (c) Send $(\mathsf{Init}, i, \mathsf{PK}_i)$ to all other parties, using PK_i as the session identifier sid_i.
  2. Message 2: Party i receives as input $(\mathsf{Init}, j, \mathsf{PK}_j)$:
     (a) If $(\mathsf{Prove}, i, j, \mathsf{PK}_i^I)$ is received from $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$, then proceed, otherwise, ignore.
     (b) Send $(\mathsf{Init}, j, \mathsf{PK}_j)$ to all other parties.
  3. Final Step: Receive as input $(\mathsf{Init}, k_j, \mathsf{PK}_k^j)$ for all $j \in [n]$ and $k \in [n]$:

$\mathcal{F}_{\mathsf{T\text{-}eVRF}}$

- Setup Initialize the functionality, by performing the following steps:
  1. Define $Q_{\mathsf{m}} \leftarrow \emptyset, Q_{\mathsf{K}}^{(1)} \leftarrow \emptyset, Q_{\mathsf{K}}^{(2)} \leftarrow \emptyset$.
- Init: On query input $(j, \mathsf{sid}_j, \cdot)$:
  1. Output $\perp$ if j is not the identifier for party id.
  2. If $Q_{\mathsf{K}}^{(1)}[\mathsf{sid}_j] \neq \perp$, return $\perp$.
  3. If $j \in \mathsf{corr}$ and $Q_{\mathsf{K}}^{(2)}[\mathsf{sid}_j] \neq \perp$, return $\perp$.
  4. Set $Q_{\mathsf{K}}^{(1)}[\mathsf{sid}_j] = j$.
  5. If $j \in \mathsf{corr}$ and the input is (j, sid_j, f_j), set $Q_{\mathsf{K}}^{(2)}[\mathsf{sid}_j] = (j, f_j)$.
- Evaluate: On query with input $(j, i, \mathsf{sid}_j, \mathsf{sid}_i, \mathsf{msg})$:
  1. Output $\perp$ if j is not the identifier for party id.
  2. If $Q_{\mathsf{K}}^{(1)}[\mathsf{sid}_j] \neq j$, then return $\perp$.
  3. If $Q_{\mathsf{K}}^{(1)}[\mathsf{sid}_i] \neq i$, then return $\perp$.
  4. If $i \wedge j \in \mathsf{hon}$, then do the following:
     (a) If $\{i, j, \mathsf{msg}\} \in Q_{\mathsf{m}}$, parse $(\alpha, A) \leftarrow Q_{\mathsf{m}}[\{i, j, \mathsf{msg}\}]$.
     (b) Else, sample $\alpha \leftarrow\!\!\$\ \mathbb{Z}_p$, and derive $A \leftarrow g^{\alpha}$. Store $Q_{\mathsf{m}}[\{i, j, \mathsf{msg}\}] \leftarrow (\alpha, A)$.
     (c) Return (α, A) to party i and send (msg, A) to all parties.
  5. Else, because $i \vee j \in \mathsf{corr}$, do the following:
     - If $i \in \mathsf{hon}$:
       (a) Get $\{j, f_j\} \leftarrow Q_{\mathsf{K}}^{(2)}[\mathsf{sid}_j]$.
       (b) Derive $\alpha \leftarrow\!\!\$\ f_j(\mathsf{msg})$ and $A \leftarrow g^{\alpha}$.
     - If $j \in \mathsf{hon}$:
       (a) Get $\{i, f_i\} \leftarrow Q_{\mathsf{K}}^{(2)}[\mathsf{sid}_i]$.
       (b) Derive $\alpha \leftarrow\!\!\$\ f_i(\mathsf{msg})$ and $A \leftarrow g^{\alpha}$.
     - Otherwise, both $i, j \in \mathsf{corr}$, and so do the following:
       (a) Get $\{i, f_i\} \leftarrow Q_{\mathsf{K}}^{(2)}[\mathsf{sid}_i]$.
       (b) Get $\{j, f_j\} \leftarrow Q_{\mathsf{K}}^{(2)}[\mathsf{sid}_j]$.
       (c) Derive $(\alpha_1, \alpha_2) \leftarrow \mathsf{ord}(i, j)$, where ord is a deterministic function to determine order.
       (d) Derive $\alpha \leftarrow\!\!\$\ f_{\alpha_1}(f_{\alpha_2}(\mathsf{msg}))$ and $A \leftarrow g^{\alpha}$.
     - Send (msg, A) to all parties.

Fig. 8: Ideal functionality for a two-party eVRF. The functionality presents minor differences with the single-party eVRF as given by Boneh et al. [12], by additionally including a second party's identifier for oracle queries.

(a) If the same message $(\mathsf{Init}, k, \mathsf{PK}_k)$ is received from all parties, store $(\mathsf{Init}, k, \mathsf{PK}_k)$, otherwise ignore.

– Evaluate –

1. Message 1: Party i receives as input $(\mathsf{Evaluate}, i, j, \mathsf{sid}_j, \mathsf{sid}_i, \mathsf{msg})$.
   (a) $(\alpha, A, \pi) \leftarrow \mathsf{Evaluate}(\mathsf{sk}_i, \mathsf{PK}_j, \mathsf{msg})$
   (b) Send $(\mathsf{Evaluate}, i, j, \mathsf{sid}_{ij}, \alpha, A, \pi)$ to all parties $k \in [n]$.
2. Upon receiving as input $(\mathsf{Evaluate}, k, j, \mathsf{sid}_k, \mathsf{sid}_j, \alpha, A, \pi)$:
   (a) Verify $(\mathsf{Init}, k, \mathsf{PK}_k)$ and $(\mathsf{Init}, j, \mathsf{PK}_j)$ have been stored.
   (b) If $\mathsf{Verify}(\mathsf{PK}_k, \mathsf{PK}_j, \mathsf{msg}, A, \pi) \neq 1$, ignore.
   (c) Otherwise, output $(\mathsf{Evaluate}, k, j, \mathsf{sid}_k, \mathsf{sid}_j, \alpha, A)$.

Theorem 4. *Let* $\mathsf{T\text{-}eVRF} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Evaluate}, \mathsf{Verify})$ *be a secure two-party exponent random function with respect to* $\{(\mathcal{X}_\lambda, \mathcal{Y}_\lambda)\}$*. Then,* $\Pi_{\mathsf{T\text{-}eVRF\text{-}DH}}$ *securely realizes with abort the ideal functionality* $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$*, as given in Appendix D, in the* $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$*-hybrid model, assuming a static adversary* $\mathcal{A}$ *corrupting any number of parties.*

The differences between the proof for Theorem 4 and the proof given by Boneh et al. [12] are minor and so we omit the full proof. The main differences between the proofs are the inputs and outputs exchanged between functionalities and in the protocols. Specifically:

- In the differences between inputs given to the ideal functionalities $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ versus $\mathcal{F}_{\mathsf{eVRF}}$, shown in Appendix D and Appendix C, respectably. Specifically, $\mathsf{Evaluate}$ in the eVRF functionality $\mathcal{F}_{\mathsf{eVRF}}$ accepts as input $(i, \mathsf{sid}_i, \mathsf{msg})$, whereas $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ accepts as input $(j, i, \mathsf{sid}_j, \mathsf{sid}_i, \mathsf{msg})$.
- In the differences between inputs given in the protocols $\Pi_{\mathsf{T\text{-}eVRF\text{-}DH}}$ and the related protocol given by Boneh et al. [12, Protocol 1]. The differences are due to the inputs and outputs for the $\mathsf{Evaluate}$ algorithms defined in the two-party eVRF setting versus the eVRF setting.

Other than the differences mentioned above, it is possible to define a simulating algorithm for Theorem 4 in the same manner as the simulating algorithm given by [12, Theorem 2]. Furthermore, the hybrid steps needed to transform $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ into the real protocol $\Pi_{\mathsf{T\text{-}eVRF\text{-}DH}}$ are the same as the three hybrid steps given in [12] to transform $\mathcal{F}_{\mathsf{eVRF}}$ into Protocol 1 (as shown in [12]).

F Leftover Hash Lemma

We need a simple extractor within our construction to ensure that pseudorandomness is satisfied. To show this in the security proof, we use the leftover hash lemma [49], which bounds the statistical distance between corresponding probability distributions.

Statistical Distance. Let X and Y be two discrete random variables over a countable set A. The statistical distance between X and Y is the quantity

$$\Delta(X, Y) = \frac{1}{2} \sum_{a \in A} \left| \Pr\{X = a\} - \Pr\{Y = a\} \right|.$$

For any predicate $p : A \to \{0,1\}$, the following inequality holds:

$$|\Pr[p(X) = 1] - \Pr[p(Y) = 1]| \leq \Delta(X, Y).$$

Lemma 1 (Leftover Hash Lemma [49]). *Let $ext : \mathcal{K} \times \mathcal{X} \to \mathbb{F}_q$ be a universal hash, i.e. $\Pr_k[ext(k,x) = ext(k,x')] \leq \frac{1}{q}$ for all distinct $x, x' \in \mathcal{X}$. Let $K, X_1, \ldots, X_m$ be mutually independent random variables, where K is uniformly distributed over $\mathcal{K}$, and each X_i is distributed over $\mathcal{X}$ such that for all i, the guessing probability $\max_{x \in \mathcal{X}} \Pr[X_i = x] \leq \gamma$. Let Δ be the statistical distance between*

$$(K,\ ext(K, X_1),\ \ldots,\ ext(K, X_m))$$

and the uniform distribution on $\mathcal{K} \times \mathcal{X}^m$. Then

$$\Delta \leq \frac{m}{2} \cdot \sqrt{\gamma q}$$

G Proof of Theorem 2

Proof. While *correctness* holds by construction, *verifiability* and *simulatability* follow from the soundness and the zero-knowledge property of the argument system for $\mathsf{T\text{-}eVRF\text{-}DH}$, respectively.

As for *pseudorandomness*, we show an explicit bound while analyzing our construction via the following game hops:

Game 0. This is the pseudorandomness experiment (Figure 2) with $b = 0$, instantiated with $\mathsf{T\text{-}eVRF\text{-}DH}$. Let $\mathcal{A}$ be an adversary playing against Game 0, and let W_0 be the event that $\mathcal{A}$ wins Game 0 (and similarly W_i for Game i). Then the probability that $\mathcal{A}$ wins Game 0 is denoted by $\Pr[W_0]$.

Game 1. Instead of $S_{i,j} \leftarrow \mathsf{PK}_j^{\mathsf{sk}_i}$ (refer to Figure 3), we take $S_{i,j}$ to be a truly random group element.

Reduction to Session-Key Unrecoverability of the NIKE. We now define a reduction $\mathcal{B}_1$ that uses $\mathcal{A}$ as a subroutine to win against the session-key unrecoverability of the Diffie-Hellman NIKE, as shown in Figure 6, with $q_t = \binom{n}{2} = \frac{n(n-1)}{2}$ (where q_t denotes the number of test queries made by $\mathcal{B}_1$).

To begin, $\mathcal{B}_1$ accepts the challenges $\{(\mathsf{PK}_i, \mathsf{PK}_j, \widetilde{S}_{i,j})\}_{i,j \in [n], i<j}$ from the Diffie-Hellman NIKE session-key unrecoverability challenger, after making $q_t = \binom{n}{2} = \frac{n(n-1)}{2}$ test queries with inputs $\{(\mathsf{PK}_i, \mathsf{PK}_j)\}_{i,j \in [n], i<j}$ (assume $\mathsf{PK}_i < \mathsf{PK}_j$ for $i < j$ without loss of generality). When $\mathcal{A}$ makes queries to $\mathcal{O}^{\mathrm{Evaluate}}(i, j, \mathsf{msg})$ for (say) all possible pairs of (i, j), we assume without loss of generality that at most one query is made with the same input (i, j, msg) (a different formulation may simply allow the query output on (i, j, msg) to be recorded and returned for additional queries on the same input, for consistency) and that $i < j$ ($\mathcal{B}_1$ can always interpret (i, j) as $(\min\{i, j\}, \max\{i, j\})$, to preserve symmetry).

Then $\mathcal{B}_1$ computes the following:

- $\widetilde{k}_{i,j} \leftarrow \widetilde{S}_{i,j}.X \in \mathbb{F}_p$
- $\widetilde{T}_{i,j,1} \leftarrow \mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)^{\widetilde{k}_{i,j}} \in \mathbb{G}_{\mathsf{in}}$
- $\widetilde{T}_{i,j,2} \leftarrow \mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)^{\widetilde{k}_{i,j}} \in \mathbb{G}_{\mathsf{in}}$
- $\widetilde{r}_{i,j,1} \leftarrow \widetilde{T}_{i,j,1}.X \in \mathbb{F}_p$
- $\widetilde{r}_{i,j,2} \leftarrow \widetilde{T}_{i,j,2}.X \in \mathbb{F}_p$
- $\widetilde{r}_{i,j} \leftarrow \beta \cdot \widetilde{r}_{i,j,1} + \widetilde{r}_{i,j,2} \in \mathbb{F}_p$

where β is randomly sampled by $\mathcal{B}_1$. As $\mathcal{B}_1$ acts as a challenger to $\mathcal{A}$ with the above values and outputs whichever result that $\mathcal{A}$ outputs, the simulation is perfect; $\mathcal{A}$'s Game 1 corresponds to the $b = 1$ case (for $\mathcal{B}_1$'s challenge instance with $q_t = \frac{n(n-1)}{2}$) while Game 0 corresponds to the $b = 0$ case.

Difference between Game 0 and Game 1. As a result, we bound the distinguishing advantage between Game 0 and Game 1 by:

$$\begin{aligned} |\Pr[W_1] - \Pr[W_0]| &\leq \left| \Pr[\mathsf{Exp}^{\mathrm{unrec}}_{\mathsf{DH},1,\mathcal{A}}(\lambda, \binom{n}{2}) = 1] - \Pr[\mathsf{Exp}^{\mathrm{unrec}}_{\mathsf{DH},0,\mathcal{A}}(\lambda, \binom{n}{2}) = 1] \right| \\ &= \mathsf{Adv}^{\mathrm{unrec}}_{\mathsf{DH},\mathcal{B}_1}(\lambda, \binom{n}{2}) \end{aligned} \tag{6}$$

Game 2. Instead of $T_{i,j,1} \leftarrow \mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)^{k_{i,j}}$ and $T_{i,j,2} \leftarrow \mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)^{k_{i,j}}$ (refer to Figure 3), we sample truly random group elements from $\mathbb{G}_{\mathsf{in}}$.

Here, we in fact introduce a series of game hops, i.e. Game 1_0 (= Game 1), Game 1_1, Game 1_2, ..., Game $1_{\binom{n}{2}}$ (= Game 2), such that we fix the pair (i, j) sequentially along some ordering of (i, j) values (without loss of generality), eventually for all $\binom{n}{2}$ values, and change $\mathcal{O}^{\mathrm{Evaluate}}(i, j, \mathsf{msg})$ only for those queries including (i, j) as input by sampling $T_{i,j,1}$ and $T_{i,j,2}$ from random.

In other words, we sequentially (across game hops) sample $T_{i,j,1}$ and $T_{i,j,2}$ from random so that we sample all $\{T_{i,j,1}, T_{i,j,2}\}_{i,j\in[n],i<j}$ from random by Game 2. The idea is that we are able to make a reduction to DDH per game hop, i.e. $\binom{n}{2}$ in total, and so we focus on some (i, j) pair (which is where we make the change to $\mathcal{O}^{\mathrm{Evaluate}}$) below and repeat the argument $\binom{n}{2}$ times to hop to Game 2.

Reduction to DDH. We now define a reduction $\mathcal{B}_{2,i,j}$ that uses $\mathcal{A}$ as a subroutine to win against DDH, i.e. distinguishing between (g^a, g^b, g^{ab}) and (g^a, g^b, g^c). The reduction is standard as in the case of DDH PRF [62], where $g = g_{\mathsf{in}}$, except we account for the fact that the distribution of $k_{i,j}$ for $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)^{k_{i,j}}$ in our case is such that $k_{i,j} \leftarrow\!\!\$\ \mathcal{P}$ (denoting the set of x-coordinates of points in $\mathbb{G}_{\mathsf{in}}$) where $\mathcal{P} \subset \mathbb{F}_p$ instead of $k_{i,j} \leftarrow\!\!\$\ \mathbb{F}_s$.

To begin, $\mathcal{B}_{2,i,j}$ accepts the challenge instance $(\hat{u}_{i,j}, \hat{v}_{i,j}, \hat{w}_{i,j})$ from the DDH challenger (for DDH in $\mathbb{G}_{\mathsf{in}}$) and then computes the following:

- $\widetilde{T}_{i,j,1} \leftarrow \hat{w}_{i,j} \in \mathbb{G}_{\mathsf{in}}$
- $\widetilde{T}_{i,j,2} \leftarrow \hat{w}_{i,j}^{r_{i,j,0}} \in \mathbb{G}_{\mathsf{in}}$
- $\widetilde{r}_{i,j,1} \leftarrow \widetilde{T}_{i,j,1}.X \in \mathbb{F}_p$

- $\widetilde{r}_{i,j,2} \leftarrow \widetilde{T}_{i,j,2}.X \in \mathbb{F}_p$
- $\widetilde{r}_{i,j} \leftarrow \beta \cdot \widetilde{r}_{i,j,1} + \widetilde{r}_{i,j,2} \in \mathbb{F}_p$

where $r_{i,j,0}$ and β are randomly sampled by $\mathcal{B}_{2,i,j}$, and $\mathcal{B}_{2,i,j}$ uses above to respond to the first query involving (i,j) by $\mathcal{A}$.

In other words, $\hat{u}_{i,j}$ takes the role of $g^{k_{i,j}}$, and $\mathcal{B}_{2,i,j}$ programs the random oracle $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)$ with $\hat{v}_{i,j}$ and re-randomizes it with a linear factor (e.g. with $r_{i,j,0}$) to program the random oracle $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)$. This means $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)^{k_{i,j}}$ and $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},2}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)^k_{i,j}$ are replaced by $\hat{w}_{i,j}$ and re-randomized $\hat{w}_{i,j}$ (e.g. $\hat{w}_{i,j}^{r_{i,j,0}}$), respectively, if $(\hat{u}_{i,j}, \hat{v}_{i,j}, \hat{w}_{i,j})$ is a DDH tuple. Likewise, $\mathcal{B}_{2,i,j}$ re-randomizes (in fact twice, to account for the two random oracles) $\hat{v}_{i,j}$ and $\hat{w}_{i,j}$ with freshly sampled randomness per additional evaluate query by $\mathcal{A}$ with (i,j,msg) over choices of msg and provides the outputs to $\mathcal{A}$.

As $\mathcal{B}_{2,i,j}$ acts as a challenger to $\mathcal{A}$ with the above values, we need to further account for the fact that g^k (for any $k \leftarrow\!\!\$\ \mathcal{P}$ in general) and $\hat{u}_{i,j}$ are not identically distributed. In particular, k in our DDH PRF $\mathsf{H}_{\mathbb{G}_{\mathsf{in}},1}(\mathsf{msg}, \mathsf{PK}_i, \mathsf{PK}_j)^k$ is not uniform over $\mathbb{F}_s$ but uniform over $\mathcal{P} \subset \mathbb{F}_p$. The reason this creates a hurdle to be resolved is that there can be (say) two different values $k_1, k_2 \in \mathcal{P}$ such that $g^{k_1} = g^{k_2}$, meaning $\{g^k\}$ might not yield a uniform distribution even if k is uniform over $\mathcal{P}$. Also, $|\mathcal{P}| = \frac{s-1}{2}$ (each non-identity element of $\mathbb{G}_{\mathsf{in}}$ forms a pair with a different non-identity element having the same x-coordinate).

Therefore, we resolve this by incorporating into our argument the statistical distance Δ_1 between $\{g^a\}$ where $a \leftarrow\!\!\$\ \mathbb{F}_s$ and $\{g^k\}$ where $k \leftarrow\!\!\$\ \mathcal{P}$. As the statistical distance is the upper bound on the distinguishing probability given two distributions (Appendix F), $\mathcal{B}_{2,i,j}$ can successfully use $\mathcal{A}$ as a subroutine to win against DDH with probability $1-\Delta_1$, i.e. the advantage of $\mathcal{B}_{2,i,j}$ is at least $1-\Delta_1$ times the distinguishing advantage of $\mathcal{A}$ between Game 1_{m-1} (corresponding to the DDH tuple case for $\mathcal{B}_{2,i,j}$) and Game 1_m (corresponding to the random case for $\mathcal{B}_{2,i,j}$) where $m \in [\frac{n(n-1)}{2}]$.

It remains to show $\frac{1}{1-\Delta_1} \leq \frac{4hp}{p-2h}$. From the Hasse bound, we note that $p \leq 2hs$ as long as $p \geq 12$. This means that there are at most $2h$ values of k that map to the same g^k value, and this further means that Δ_1 is maximized, over possibilities of $\mathcal{P}$, when there are as many distinct g^k values each of which has $2h$ values of k mapping to it (i.e. preimage of size $2h$) as possible.

In other words, $\{g^a\}_{a \leftarrow\!\!\$\ \mathbb{F}_s}$ is a uniform distribution such that each g^a value has a preimage of size 1, and so $\{g^k\}_{k \leftarrow\!\!\$\ \mathcal{P}}$ needs to deviate from this uniform distribution as much as possible to maximize $\Delta_1 = \Delta(\{g^a\}_{a \leftarrow\!\!\$\ \mathbb{F}_s}, \{g^k\}_{k \leftarrow\!\!\$\ \mathcal{P}})$. Nonetheless, the fact that the size of each g^k value's preimage is bounded from above by $2h$ conceptually bounds the extent of the statistical distance.

Specifically, we choose $\mathcal{P}_1 \subset \mathbb{F}_p$, with $|\mathcal{P}_1| = \frac{s-1}{2}$, so that in the distribution $\{g^k\}_{k \leftarrow\!\!\$\ \mathcal{P}_1}$ there are $\lfloor \frac{|\mathcal{P}_1|}{2h} \rfloor$ distinct g^k values each with a preimage of size $2h$ and 1 distinct value of g^k with a preimage of size $|\mathcal{P}_1| \bmod 2h$. We compute and

bound the statistical distance as follows:

$$\begin{aligned}
\Delta_1 &= \Delta(\{g^k\}_{k \xleftarrow{\$} \mathcal{P}_1}, \{g^a\}_{a \xleftarrow{\$} \mathbb{F}_s}) && (7)\\
&= \frac{1}{2}\left[\lfloor\frac{|\mathcal{P}_1|}{2h}\rfloor|\frac{2h}{|\mathcal{P}_1|} - \frac{1}{s}| + |\frac{|\mathcal{P}_1| \bmod 2h}{|\mathcal{P}_1|} - \frac{1}{s}| + (s - \frac{|\mathcal{P}_1|}{2h} - 1)|0 - \frac{1}{s}|\right] && (8)\\
&\leq \frac{1}{2}\left(\lfloor\frac{|\mathcal{P}_1|}{2h}\rfloor|\frac{2h}{|\mathcal{P}_1|} - \frac{1}{s}| + |\frac{|\mathcal{P}_1| \bmod 2h}{|\mathcal{P}_1|} - \frac{1}{s}| + \frac{1}{s}(\lfloor\frac{|\mathcal{P}_1|}{2h}\rfloor + 1 - \frac{|\mathcal{P}_1|}{2h})\right)\\
&\quad + \frac{1}{2}\left(\frac{1}{s}(\lfloor\frac{|\mathcal{P}_1|}{2h}\rfloor + 1 - \frac{|\mathcal{P}_1|}{2h}) + (s - \frac{|\mathcal{P}_1|}{2h} - 1)|\frac{1}{s}|\right) && (9)\\
&= 1 - \frac{|\mathcal{P}_1|}{2hs} && (10)
\end{aligned}$$

where (8) follows from the definition of statistical distance, (9) follows from adding two extra positive terms of $\frac{1}{2} \cdot \frac{1}{s}(\lfloor\frac{|\mathcal{P}_1|}{2h}\rfloor + 1 - \frac{|\mathcal{P}_1|}{2h})$, and (10) follows via rearranging.

This implies that

$$\frac{1}{1 - \Delta_1} \leq \frac{2hs}{|\mathcal{P}_1|} \leq \frac{4hp}{p - 2h} \tag{11}$$

using the aforementioned Hasse bound ($p \leq 2hs$ for $p \geq 12$) provided that $p > 2h$. We conveniently use the above inequality in the following.

Difference between Game 1_{m-1} *and Game* 1_m*.* As a result, we bound the distinguishing advantage between Game 1_{m-1} and Game 1_m for $m \in [\binom{n}{2}]$ by:

$$\begin{aligned}
\left|\Pr[W_{1_m}] - \Pr[W_{1_{m-1}}]\right| &\leq \frac{1}{1 - \Delta_1}\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}_{\mathsf{in}}, \mathcal{B}_{2,i,j}}(\lambda) && (12)\\
&\leq \frac{4hp}{p - 2h}\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}_{\mathsf{in}}, \mathcal{B}_{2,i,j}}(\lambda) && (13)
\end{aligned}$$

Difference between Game 1 and Game 2. As a result, we bound the distinguishing advantage between Game 1 and Game 2 by:

$$|\Pr[W_2] - \Pr[W_1]| \leq \frac{2hpn^2}{p - 2h}\mathsf{Adv}^{\mathrm{DDH}}_{\mathbb{G}_{\mathsf{in}}, \mathcal{B}_2}(\lambda) \tag{14}$$

where we sum over all pairs of (i, j) values and take a factor of $\frac{n^2}{2}$ instead of $\binom{n}{2}$ in the inequality for simplicity.

<u>*Game 3.*</u> Instead of $r \leftarrow \beta \cdot r_1 + r_2 \in \mathbb{F}_p$ (refer to Figure 3), we sample a truly random $r \in \mathbb{F}_p$. This is the pseudorandomness experiment (Figure 2) with $b = 1$.

Difference between Game 2 and Game 3. Considering this, we are able to bound the distinguishing advantage between Game 2 and Game 3 using the leftover hash lemma (Appendix F) directly as follows. To apply the leftover hash lemma

to $(\beta, r = \beta \cdot r_1 + r_2)$ for a single sample (i.e. one evaluate query by $\mathcal{A}$), we note that the guessing probability of (r_1, r_2), where $r_1, r_2 \leftarrow\!\!\$\ \mathcal{P}$, is bounded by $\frac{1}{|\mathcal{P}|^2}$, and we denote this by γ.

Then from the leftover hash lemma, the statistical distance Δ_2 between the distribution of (β, r) and the uniform distribution on $\mathbb{F}_p^2$ is given by

$$\Delta_2 \leq \frac{\sqrt{p}}{2} \cdot \sqrt{\gamma} \leq \frac{2h\sqrt{p}}{p - 2h} \tag{15}$$

which is deduced from the aforementioned Hasse bound ($p \leq 2hs$ for $p \geq 12$) provided that $p > 2h$. To account for multiple queries by $\mathcal{A}$ which makes q_e number of queries to $\mathcal{O}^{\text{Evaluate}}$, we note that extracting from q_e samples increases Δ_2 by a factor of q_e.

As a result, we bound the distinguishing advantage between Game 2 and Game 3 by:

$$|\Pr[W_3] - \Pr[W_2]| \leq \frac{2h\sqrt{p}}{p - 2h} q_e \tag{16}$$

Finishing the proof. From (6), (14), and (16), we conclude with the following bound:

$$\mathsf{Adv}^{\text{psdr}}_{\mathsf{T\text{-}eVRF\text{-}DH},\mathcal{A}}(\lambda, n) = |\Pr[W_0] - \Pr[W_3]| \tag{17}$$

$$\leq \mathsf{Adv}^{\text{unrec}}_{\mathsf{DH},\mathcal{B}_1}(\lambda, \binom{n}{2}) + \frac{2hpn^2}{p - 2h} \mathsf{Adv}^{\text{DDH}}_{\mathbb{G}_{\mathsf{in}},\mathcal{B}_2}(\lambda) + \frac{2h\sqrt{p}}{p - 2h} q_e \tag{18}$$

□

H A Different Approach in Proof Construction

A different proof construction is possible if we forgo the proof of discrete logarithm (for the last exponentiation of $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$) and instead represent $\mathcal{R}_{\mathsf{T\text{-}eVRF\text{-}DH}}$ in its entirety as A, B, C matrices for Bulletproofs, albeit with a slight modification of the ninth step to be $R \leftarrow g_{\mathsf{in}}^r \in \mathbb{G}_{\mathsf{in}}$. The subtle advantage of this construction is that we can use the same group for the Diffie-Hellman key exchange and the two-party eVRF commitment output, i.e. R is a pseudorandom group element in $\mathbb{G}_{\mathsf{in}}$ not $\mathbb{G}_{\mathsf{out}}$. While a second group $\mathbb{G}_{\mathsf{out}}$ is required for the Bulletproofs argument system either way, this may be desirable in use cases where the PKI group and the DKG group must be equal for a more intuitive implementation.

The disadvantage is the increased number of constraints. Not only the modified ninth step (needing each iteration of the first and second gadgets amounting to $4\lambda + 4$ additional constraints), the eighth step would also add overhead. To ensure pseudorandomness, the leftover hash lemma must then be applied with a prime modulus different from the one used by all other constraints (such operation is called a *non-native arithmetic*), which can add a few thousands of constraints.

I Zero-Knowledge Proof for Public Identify Keys

Our DKG construction Golden requires that all parties accept as input valid public keys $\{\mathsf{PK}_i\}_{i=1}^n$ from other parties, such that each party has additionally proven the relation

$$\mathcal{R}_{pok} = \{(g, \mathsf{PK}, (\mathsf{sk})) : \mathsf{PK} = g^{\mathsf{sk}}\} \tag{19}$$

We first describe the ideal functionality describing security for $\mathcal{R}_{pok}$. We then give a concrete construction for $\mathcal{R}_{pok}$, and then discuss the ideal functionality $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ that we later use when proving security of Golden.

I.1 Simulation-Based Security for $\mathcal{R}_{pok}$

We first describe an idealized functionality $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ for the relation $\mathcal{R}_{pok}$, following the definition by Boneh et al. [12]. $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ is defined as follows. The functionality receives a Prove message from party i with input $(i, \mathsf{PK}, \mathsf{sk})$. If $(\mathsf{PK}, \mathsf{sk}) \in \mathcal{R}_{pok}$, then the functionality sends (i, PK) to all parties $j \in [n]$. Otherwise, the functionality returns $\perp$.

Note the Prove oracle in the definition by Boneh et al. [12] accepted as input a message $(i, j, \mathsf{PK}, \mathsf{sk})$, and if the input satisfied the relation, would send the message to party j. We instead give our definition where the Prove oracle sends this message to all parties, as opposed to delivering the message specifically to a single party.

I.2 A Concrete Protocol Realizing $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$

To realize $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$, we need a zero-knowledge proof system to demonstrate knowledge of sk. However, this zero-knowledge proof system must be online-extractable, to prove security in the UC framework, as rewinding is not allowed.

One possible direction is to use zero-knowledge proofs with Fischlin's transform [40], which can be proven secure using standard assumptions. However, doing so introduces additional technical and performance complexity.

A Lightweight Protocol in the CRS Model. We next define a concrete relation that realizes $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ in a lightweight manner. We show this relation in Equation 20

The zero-knowledge proof relation $\mathcal{R}_P$ given in Equation 19 demonstrates that an ElGamal ciphertext is correct, and either that the prover knows the message that is encrypted (which is sk), or the prover knows the discrete logarithm of a public value h. Because h is chosen at random from $\mathbb{G}$ at the time of setup, then this proof by inference demonstrates that the prover knows sk, if the discrete logarithm problem is hard in $\mathbb{G}$.

$$\mathcal{R}_P = \Big\{(g, h, \mathsf{PK}, c = (C_0, C_1), z, R_g, R_h) \; ; \; (\mathsf{sk}, r)\Big\} \quad \text{iff}$$
$$\begin{aligned} &(1) \quad C_0 = g^r; \; C_1 = h^r \cdot g^{\mathsf{sk}} \\ &(2) \quad g^z = R_g \cdot (g^{\mathsf{sk}})^{c'} \\ &(3) \quad h^z = R_h \cdot (h^{\mathsf{sk}})^{c'} \end{aligned} \tag{20}$$

It is straightforward to demonstrate that $\mathcal{R}_P$ securely realizes $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ in the UC framework, due to the non-interactive proof straight-line compiler given by Chen et al. [23]. Concretely, applying the compiler defined by Chen et al. [23] to a Schnorr proof of knowledge gives the Chaum-Pedersen zero-knowledge proof given in Equation 20.

J Proof for Theorem 3

Proof. We establish the proof by showing that $\mathsf{Real}_{\Pi_{\mathsf{Golden\text{-}PKI}},\mathcal{A}}$ is indistinguishable from $\mathsf{Ideal}_{\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}},\mathcal{S}}$ with respect to $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$, by a sequence of games.

Without loss of generality, we assume the adversary completes the Initialize step of the protocol correctly., and so the DKG protocol runs with the full set of n participants. Note the proof can easily be extended to the setting where only a subset of $\ell < n, \ell \geq t$ parties completed the Initialize step correctly, by simply performing the DKG among this subset of ℓ parties.

Game 0. This is the indistinguishability game $\mathsf{Real}_{\Pi,\mathcal{A}}$ defined in Section 3.1 instantiated with $\Pi = \Pi_{\mathsf{Golden\text{-}PKI}}$.

Game 1. In Game 1, we make the following modifications.

In Round 0 of the DKG, each party i queries $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ on its $\mathsf{Evaluate}$ oracle (as defined in Figure 8) n times on inputs (i, j, msg_i) for $j \in [n]$, and receives $(r_{i,j}, R_{i,j}), j \in [n]$ in return. Each party then follows the remainder of Round 0 as in Game 1. In Round 1, each party similarly queries $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ n times on inputs (j, i, msg_j) for $j \in [n]$, receiving $(r_{j,i}, R_{j,i}), j \in [n]$ in return. Recall that in $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$, inputs and outputs are generated and delivered to all parties honestly, and so parties do not need additional zero-knowledge proofs of correctness or to perform Verify on the outputs.

Difference between Game 0 and Game 1. The difference between Game 0 and Game 1 is bounded by the indistinguishability of $\mathsf{T\text{-}eVRF\text{-}DH}$ from $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$. We show $\mathsf{T\text{-}eVRF\text{-}DH}$ is a secure two-party eVRF, as established in the proof for Theorem 2.

Because Game 0 employs an equivalent initialize functionality as π_{EF}, then $\mathsf{T\text{-}eVRF\text{-}DH}$ in conjunction with this initialize functionality satisfies the requirements for this equivalence to hold, as shown by Theorem 4 given in Appendix E

Hence,

$$\Pr[W_0] = \Pr[W_1]$$

Game 2. We next describe a simulating algorithm SIM that interacts with $\mathcal{A}$ in a black-box manner. SIM accepts as input first parameters (n, t), and then a tuple containing a polynomial commitment and t shares $(\{Y_i\}_{i=0}^{t-1}, \{f^\dagger(j)\}_{j\in C})$ from the functionality. SIM simulates $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ and $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$. We describe SIM in detail next.

Initialize. On input (n, t), SIM does the following.

1. Send (n, t) to $\mathcal{A}$, receive corr in return. Output corr.
2. Receive as input $(\{Y_i\}_{i=0}^{t-1}, \{f^\dagger(j)\}_{j\in C})$ from the functionality.
3. Set $\mathsf{hon} \leftarrow [n] \setminus \mathsf{corr}$.
4. Initialize the tables $Q_{\mathsf{K}}^{(1)} \leftarrow \emptyset$, $Q_{\mathsf{K}}^{(2)} \leftarrow \emptyset$, $Q_{\mathsf{K}'} \leftarrow \emptyset$, $Q_{\mathsf{T\text{-}eVRF}} \leftarrow \emptyset$.

Participant Initialization. SIM simulates the initialize step for each participant as defined in Game 2, as follows.

1. For all honest parties $i \in \mathsf{hon}$, SIM performs key generation honestly, sampling $\mathsf{sk}_i^I \leftarrow\!\!\$\ \mathbb{F}_s$, and setting $\mathsf{PK}_i^I \leftarrow g^{\mathsf{sk}_i^I}$.
2. SIM then simulates each honest party $i \in \mathsf{hon}$ during the initialization phase of $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$, by following the ideal functionality steps exactly, setting $Q_{\mathsf{K}}^{(1)}[\mathsf{PK}_i^I] \leftarrow i$ for received values.
3. When the adversary queries the ideal functionality $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ on Init with input (j, f_j) where $j \in \mathsf{corr}$, SIM follows the actions of $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ exactly, by setting $Q_{\mathsf{K}}^{(2)}[\mathsf{PK}_j^I] = (j, f_j)$ if it has not already been set, and similarly for $Q_{\mathsf{K}}^{(1)}$.
4. When the adversary queries Prove on input $(j, \mathsf{PK}_j^I, \mathsf{sk}_j^I)$, SIM updates $Q_{\mathsf{K}'} \leftarrow Q_{\mathsf{K}'} \cup \{(\mathsf{PK}_j^I, j, \mathsf{sk}_j^I)\}$, and otherwise follows the actions of $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ exactly.

Simulating $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ Evaluate. SIM simulates the $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ $\mathsf{Evaluate}$ oracle as defined in Figure 8 as follows:

1. For corrupt parties $k \in \mathsf{corr}$, when $\mathcal{A}$ queries $\mathsf{Evaluate}$ on input (k, j, msg), SIM follows the actions of $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ exactly. Finally, SIM outputs A.
2. When SIM (later in the simulation of Round 1) simulates honest parties' interactions with $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ for honest parties $j \in \mathsf{hon}, j \neq \tau$, SIM follows the actions of $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ exactly, with the following exception. Instead of sampling α at random and deriving A, SIM instead sets A to be the particular input (as described below). SIM then updates $Q_{\mathsf{T\text{-}eVRF}}[(k, j, \mathsf{msg})] \leftarrow A$.

Simulating Key Generation. Next, SIM must simulate the protocol on behalf of honest parties to $\mathcal{A}$. We show how SIM does that next.

Round 0. To simulate honest parties' contributions for round 0 of Golden, SIM does the following:

1. For each honest party $k \in \mathsf{hon}, k \neq \tau$, SIM follows the protocol in the same manner as defined in Game 2, and simulates $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ for honest parties as defined above.

2. However, for the last honest party queried $\tau \in \mathsf{hon}$, SIM simulates the protocol as follows:
   (a) Follows the protocol honestly to sample a message $\mathsf{msg}_\tau \leftarrow\!\!\$\ \{0,1\}^\lambda$.
   (b) Set corrupt parties' shares to be

$$\bar{x}_{\tau,j} \leftarrow f^\dagger(j) - \sum_{k \in \mathsf{hon}, k \neq \tau} \bar{x}_{k,j} \tag{21}$$

   for all $j \in \mathsf{corr}$, using the inputs from the ideal functionality and then "inverting" honest parties' contributions.
   (c) SIM then simulates the polynomial commitment $\bar{C}_\tau$ to the polynomial f_τ, as follows:
      i. Set

$$A_{\tau,0} \leftarrow Y_0 \cdot \prod_{k \in \mathsf{hon}, k \neq \tau} (A_{k,0})^{-1} \tag{22}$$

      where $A_{k,0}$ are from the honest parties $k \in \mathsf{hon}, k \neq \tau$ polynomial commitments $\bar{C}_k$, and Y_0 is an input from the ideal functionality.
      ii. SIM computes the Lagrange polynomials $\{L'_0(Z), \{L'_j(Z)\}_{j \in \mathsf{corr}}\}$ for the set (of x-coordinates) $\{0 \cup \mathsf{corr}\}$, as defined in Section 3.2.
      iii. For $i \in \{1, \ldots, t-1\}$, SIM then sets the remaining commitments of f_τ "in the exponent," as follows:

$$A_{\tau,i} \leftarrow A_{\tau,0}^{L'_0(i)} \cdot \prod_{j \in \mathsf{corr} \cup \{0\}} (g^{\bar{x}_{\tau,j}})^{L'_j(i)}$$

      iv. Set $\bar{C}_\tau = (A_{\tau,0}, \ldots, A_{\tau,t-1})$. Here, $\bar{C}_\tau$ is the commitment to f_τ, defined by the $t-1$ corrupt parties' shares, and Y_0, such that $g^{f_\tau(0)} = A_{\tau,0}$ as given in Equation 22.
   (d) For each corrupt party $j \in \mathsf{corr}$, simulate $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ on inputs $(\tau, \mathsf{msg}_{\tau,j} =$ "$(\mathsf{PK}_j^I, \mathsf{msg}_\tau)$"$)$ for $j \in \mathsf{corr}$, as defined above. Then, set $z_{\tau,j} \leftarrow r_{\tau,j} + \bar{x}_{\tau,j}$.
   (e) For $\ell \in \mathsf{hon}$, define

$$\bar{X}_{\tau,\ell} \leftarrow A_{\tau,0} \cdot \prod_{i=1}^{t-1} A_{\tau,i}^{\ell^i} \tag{23}$$

   to be the commitment to the (unknown) ℓ^{th} honest party's share $\bar{x}_{\tau,\ell}$.
   (f) Then, simulate the encryption $z_{\tau,\ell}$ to honest parties' $\ell \in \mathsf{hon}$ shares (which are unknown) by performing the following steps:
      i. Sample $z_{\tau,\ell} \leftarrow\!\!\$\ \mathbb{Z}_p$.
      ii. Set

$$R_{\tau,\ell} \leftarrow g^{z_{\tau,\ell}} \cdot \bar{X}_{\tau,\ell} \tag{24}$$

      Simulate $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ as described above for $R_{\tau,\ell}$.
   (g) For all parties $j \in [n]$, follow the protocol to set $\sigma_{\tau,j} \leftarrow (R_{\tau,j}, z_{\tau,j})$.
   (h) Set $\mathsf{bmsg}_\tau \leftarrow (\mathsf{msg}_\tau, \bar{C}_\tau, \{\sigma_{\tau,j}\}_{j \in [n]})$.
3. Output $\{\mathsf{bmsg}_j\}_{j \in \mathsf{hon}}$ for all honest parties.

Round 1. When $\mathcal{A}$ queries honest parties on Round 1 with inputs $\{\mathsf{bmsg}_j\}_{j\in\mathsf{corr}}$, SIM does not output anything (as is the case in the real protocol).

Analysis. The analysis of SIM as compared to Game 2 is based on the following:

(1) Simulation of $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$
(2) Simulation of $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$
(3) Simulation of honest parties' outputs of Round 0
(4) Correctness of corrupt parties' checks in Round 1

For (1), SIM's simulation of $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$ is indistinguishable from Game 2, because SIM follows the same steps as the idealized $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$, but simply maintains state in $Q_{\mathsf{K}'}$.

For (2), SIM's simulation of $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ is indistinguishable from the ideal functionality as defined in Figure E, for the following reasons. First, SIM follows the same Init steps as the ideal functionality, and simply stores values in a local table. Secondly, for $\mathsf{Evaluate}$ queries for all parties $k \in \mathsf{hon}, k \neq \tau$ and for corrupt parties, SIM simulates $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ in the same manner as the ideal functionality. For the simulated honest party $\tau \in \mathsf{hon}$, SIM's simulation of $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$is likewise perfect, because $R_{\tau,\ell} = g^{z_{\tau,\ell}} \cdot \bar{X}_{\tau,\ell}$ as defined as in Equation 24, where $\bar{X} = g^{f_\tau}(\ell)$, as defined in Equation 23. Because $z_{\tau,\ell}$ is sampled uniformly at random from $\mathbb{Z}_p$, then $g^{z_{\tau,\ell}}$ will likewise be a uniformly random element in $\mathbb{G}$, and hence so will be $g^{z_{\tau,\ell}} \cdot \bar{X}_{\tau,\ell}$. Consequently, $R_{\tau,\ell}$ for all $\ell \in \mathsf{hon}$ will be indistinguishable from $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$'s real output for an honest party.

For (3), SIM's simulation of honest parties' outputs from Round 0 is indistinguishable from Game 2 for the following reasons:

- When generating Shamir secret shares for τ, corrupt parties receive random shares $\bar{x}_{\tau,j}$, as defined in Equation 21, which are defined by randomly sampled shares from the ideal functionality, and corrupt party shares chosen honestly. Each corrupt party's share is a point on the (random) polynomial f_τ, such that $g^{f_\tau} = \bar{C}_\tau$, as shown in Equation 22.
- For all honest parties $j \in \mathsf{hon}, j \neq \tau$, SIM simulates $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ exactly as in Game 2.
- However, for the honest party $\tau \in \mathsf{hon}$, SIM's simulation is indistinguishable to $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$,

Lastly, for (4), corrupt parties that follow the protocol will receive valid outputs in Round 1, for the following reasons:

- Due to how $R_{\tau,\ell}$ for $\ell \in \mathsf{hon}$ is defined above, the check $g^{z_{\tau,\ell}} = R_{\tau,\ell} \cdot \bar{X}_{\tau,\ell}$ will hold by definition. All other honest parties' values $(R_{i,j}, z_{i,j})$ for $i \in \mathsf{hon}, j \in [n]$ are defined by following the protocol.
- Each $\mathsf{PK}_i = g^{\bar{x}_i}, i \in [n]$ will be valid and indistinguishable from a real run of the protocol, because $\mathsf{PK}_i = \prod_{j\in[n]} \bar{X}_{j,i}$. Although f_τ is unknown, $\bar{C}_\tau$ commits to f_τ "in the exponent," and f_τ is fully defined by the discrete logarithm of Y_0, and all other honest parties' contributions. Although SIM does not know $\bar{x}_{\tau,i}$ directly, it can derive $\bar{X}_{simid,i}$, simply by performing polynomial interpolation "in the exponent."

– For similar reasons as above, PK will likewise be valid and indistinguishable from a real run of the protocol.

Output. SIM outputs the corrupt parties' contributions to the DKG $\Delta(x)$ to $\mathsf{Ideal}_{\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}},\mathcal{S}}$, as shown in Equation 25.

$$\Delta(x) = \Delta_0 + \Delta_1 x + \Delta_2 x^2 + \ldots + \Delta_{t-1} x^{t-1} \tag{25}$$

such that $\Delta(0) = \sum_{j \in \mathsf{corr}} \omega_j$, and $\Delta_i = \sum_{j \in \mathsf{corr}} a_{j,i}$, where $a_{j,i}$ is the i^{th}coefficient for the corrupt party j's sharing polynomial f_j.

We describe next how SIM derives these values from the protocol transcript next.

First, SIM can derive $\Delta_0 = \sum_{j \in \mathsf{corr}} \omega_j$ by the following steps:

1. Obtain $(\mathsf{sk}^I_k)_{k \in \mathsf{corr}}$ from $Q_{\mathsf{K}'}$.
2. Derive $(r_{j,i}, \cdot, \cdot) \leftarrow \mathsf{T\text{-}eVRF\text{-}DH.Evaluate}(\mathsf{sk}^I_j, \mathsf{PK}^I_i, \mathsf{msg}_j)$ for each $j \in \mathsf{corr}, i \in [n]$, using its knowledge of $\mathsf{sk}^I_j, j \in \mathsf{corr}$.
3. SIM then derives all shares issued by corrupt party $j \in \mathsf{corr}$, by deriving $\bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i}$, for each $i \in [n]$.
4. SIM then derives $\omega_j \leftarrow \mathsf{Feldman.Recover}(t, \{(i, \bar{x}_{j,i})\}_{i \in [n]})$, for each $j \in \mathsf{corr}$.
5. Finally, set $\Delta_0 \leftarrow \sum_{j \in \mathsf{corr}} \omega_j$.

SIM can then derive the remaining $\Delta_\ell, \ell \in \{1, \ldots, t-1\}$ with the following steps.

1. Using the previously derived shares for malicious parties, $\bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i}$, for each $j \in \mathsf{corr}, i \in [n]$, and for each $\ell \in \{1, \ldots, t-1\}$, derive each coefficient in each corrupt parties' sharing polynomial $f_j(x)$ as

$$a_{j,\ell} \leftarrow \omega_j L^S_0(\ell) + \sum_{i=1}^{t} \bar{x}_{i,\ell} L^S_i(\ell)$$

   where $L^S_i(x)$ is the Lagrange polynomial defined by $S = \{0, \ldots, t\}$. Note that SIM corrupt party j's sharing polynomial is fully defined with only t points,
2. SIM then derives the remaining coefficients of $\Delta(x)$, by deriving $\Delta_\ell \leftarrow \sum_{j \in \mathsf{corr}} a_{j,\ell}$ for each $\ell \in \{1, \ldots, t-1\}$.

Finishing the Proof. Let f be the (implicit) sharing polynomial f defined at the end of the DKG, such that

$$f(x) = \mathsf{sk} + \sum_{i=1}^{n} a_{i,1} x + a_{i,2} x^2 + \ldots + a_{i,t-1} x^{t-1}$$

where each coefficient $a_{i,\ell}, \ell \in \{1, \ldots, t-1\}$ is produced by each party performing $\mathsf{Feldman.Share}(\omega_i, n, t)$

At the end of SIM's simulation, to show indistinguishability from $\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}$, we must show that

$$f(x) = (\mathsf{dlog}(Y_0) + \Delta(0)) + \sum_{j \in \mathsf{corr}} (f^{\dagger}(j) + \Delta(j))\lambda'_j \tag{26}$$

where λ'_j is the j^{th} Lagrange coefficient for the set $C = \{0\} \cup \mathsf{corr}$.

First, as shown in Equation 22, party τ's polynomial commitment commits to the constant term $A_{\tau,0}$, which is $Y_0 \cdot \prod_{k \in \mathsf{hon}, k \neq \tau} (A_{k,0})^{-1}$. Hence, $f(0) = \mathsf{dlog}(Y_0) + \Delta(0)$.

Next, as shown in Equation 21, each share $\bar{x}_{\tau,j}$ is such that $\bar{x}_{\tau,j} = f^{\dagger}(j) - \sum_{k \in \mathsf{hon}, k \neq \tau} \bar{x}_{k,j}$, and so:

$$\begin{aligned}
f(j) &= \sum_{i=1}^{n} f_i(j)\lambda''_j \\
&= \sum_{i \in \mathsf{hon}} f_i(j)\lambda''_i + \sum_{k \in \mathsf{corr}} f_k(j)\lambda''_k \\
&= f_\tau(j) + \sum_{i \in \mathsf{hon}, i \neq \tau} f_i(j)\lambda''_i + \sum_{k \in \mathsf{corr}} f_k(j)\lambda''_k \\
&= f^{\dagger}(j) - \sum_{i \in \mathsf{hon}, i \neq \tau} \bar{x}_{i,j} + \sum_{i \in \mathsf{hon}, i \neq \tau} f_i(j)\lambda''_i + \sum_{k \in \mathsf{corr}} f_k(j)\lambda''_k \\
&= \sum_{j \in \mathsf{corr}} (f^{\dagger}(j) + \Delta(j))\lambda''_j
\end{aligned} \tag{27}$$

where λ''_j is the j^{th} Lagrange coefficient for the set $C = [n]$.

Hence, due to how the sharing polynomial is defined in Equations 26 and 27, and how SIM extracts each coefficient of $\Delta(x)$ as described above, SIM's output in Game 3 is indistinguishable from $\mathsf{Ideal}_{\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}, \mathcal{S}}$. This completes the proof. □

K One-Round DKG with Aborts

In this section, we show how to achieve a stronger notion of security (security with aborts) with a recently proposed related DKG that builds on the eVRF DKG by Boneh et al [12]. This DKG construction was first described by Parker and Madrigal [60]. We present a variant in Figure 9 that uses our two-party eVRF to generate ciphertexts; we refer to this variant as $\mathsf{BHL24\text{-}DKG}$. We give the first formal proof of security for the $\mathsf{BHL24\text{-}DKG}$ construction in Appendix K.2. Note one key difference of $\mathsf{BHL24\text{-}DKG}$ with Golden is the reliance on a session identifier sid, which is assumed to be consistent among all parties.

K.1 Security with Aborts

We show the ideal functionality $\mathcal{F}^{\perp}_{\mathsf{KeyGen}}$ as shown in Figure 10, which assumes an adversary that can abort the protocol. Note this ideal functionality was formalized by Katz [54]. In this functionality, either the ideal functionality outputs a public

$\mathsf{Round}_0(n, t, i, \mathsf{sid}, \mathsf{sk}_i^I, \{(j, \mathsf{PK}_j^I)\}_{j\in[n]})$ // *Party i performs the protocol.*

1 : **for** $\ell \in \{0, \ldots, t-1\}$ **do**
2 : $\quad (a_{i,\ell}, A_{i,\ell}, \pi^a_{i,\ell}) \leftarrow \mathsf{eVRF}^{\mathsf{BHL24}}.\mathsf{Evaluate}(\mathsf{sk}_i^I, \mathsf{sid}||\ell),\ \text{for } \ell \in \{0, \ldots, t-1\}$
3 : $\bar{C}_i = (a_{i,0}, \ldots, a_{i,t-1})$
4 : $(\{(j, \bar{x}_{i,j})\}_{j=1}^n, \bar{C}_i = (A_{i,0}, \ldots, A_{i,t-1})) \leftarrow\!\!\$\ \mathsf{Feldman.Share}(\omega_i, n, t)$
5 : **for** $j \in [n], j \neq i$ **do**
6 : $\quad (r_{i,j}, R_{i,j}, \pi_{i,j}) \leftarrow \mathsf{T\text{-}eVRF\text{-}DH.Evaluate}(\mathsf{sk}_i^I, \mathsf{PK}_j^I, \mathsf{sid}_i)$
7 : $\quad z_{i,j} \leftarrow r_{i,j} + \bar{x}_{i,j}$ // *Encrypt the share to player j*
8 : $\quad \sigma_{i,j} \leftarrow (R_{i,j}, z_{i,j})$
9 : $\mathsf{st}_i \leftarrow \bar{x}_{i,i}$ // *Keep one share for itself*
10 : $\mathsf{bmsg}_i \leftarrow \{(\bar{C}_i, (\pi^a_{i,\ell})_{i=0}^{t-1}, \sigma_{i,j}, \pi_{i,j})\}_{j\in[n],j\neq i};$ Broadcast bmsg_i

$\mathsf{Round}_1(n, t, i, \mathsf{sid}, \mathsf{sk}_i^I, \{(j, \mathsf{PK}_j^I)\}_{j\in[n]}, \{\mathsf{bmsg}_j\}_{j\in[n]}, \mathsf{st}_i)$

1 : $\bar{x}_{i,i} \leftarrow \mathsf{st}_i$
2 : **for** $j \in [n], j \neq i,$ **do**
3 : $\quad \mathsf{Parse}\ \{(\bar{C}_j, (\pi^a_{i,\ell})_{i=0}^{t-1}, \sigma_{j,k}, \pi_{j,k})\}_{k\in[n],k\neq j} \leftarrow \mathsf{bmsg}_j$
4 : $\quad \mathsf{Parse}\ \ (A_{j,0}, \ldots, A_{j,t-1}) \leftarrow \bar{C}_j;\ (R_{j,k}, z_{j,k}) \leftarrow \sigma_{j,k}, k \in [n], k \neq j$
5 : $\quad$ **for** $\ell \in \{0, \ldots, t-1\}$ **do**
6 : $\quad\quad$ **return** $\perp$ **if** $\mathsf{eVRF}^{\mathsf{BHL24}}.\mathsf{Verify}(\mathsf{PK}_j^I, \mathsf{sid}, A_{j,\ell}, \pi^a_{j,k}) \neq 1$
7 : **for** $j \in [n], j \neq i,$ **do**
8 : $\quad$ **for** $k \in [n], k \neq j,$ **do**
9 : $\quad\quad$ **return** $\perp$ **if** $\mathsf{T\text{-}eVRF\text{-}DH.Verify}(\mathsf{PK}_j^I, \mathsf{PK}_k^I, \mathsf{msg}_j, R_{j,k}, \pi_{j,k}) \neq 1$
10 : $\quad\quad \bar{X}_{j,k} \leftarrow \prod_{\ell=0}^{t-1} A_{j,\ell}^{k^\ell}$ // *$\bar{X}_{j,k}$ is the commitment $g^{f_j(k)}$*
11 : $\quad\quad$ **return** $\perp$ **if** $g^{z_{j,k}} \neq R_{j,k} \cdot \bar{X}_{j,k}$ // *Check the ciphertext is valid; $g \in \mathbb{G}_{\mathsf{out}}$*
12 : **for** $j \in [n], j \neq i,$ **do**
13 : $\quad (r_{j,i}, \cdot, \cdot) \leftarrow \mathsf{T\text{-}eVRF\text{-}DH.Evaluate}(\mathsf{sk}_i^I, \mathsf{PK}_j^I, \mathsf{msg}_j)$ // *Does not require $(R_{j,i}, \pi_{j,i})$*
14 : $\quad \bar{x}_{j,i} \leftarrow z_{j,i} - r_{j,i}$ // *Decrypt Shamir share from participant j*
15 : $\mathsf{sk}_i \leftarrow \sum_{j=1}^n \bar{x}_{j,i}$ // *Sum of all shares $f_j(i)$*
16 : $\mathsf{PK}_\ell \leftarrow \prod_{k=1}^n \bar{X}_{k,\ell}$ for $\ell \in [n]$ // *Equal to g^{sk_ℓ}*
17 : $\mathsf{PK} \leftarrow \prod_{k=1}^n A_{k,0}$ // *Equal to $g^{\sum_{k=1}^n \omega_k}$*
18 : **return** $(\mathsf{PK}, \{\mathsf{PK}_j\}_{j=1}^n, \mathsf{sk}_i)$

Fig. 9: BHL24-DKG, which builds upon the eVRF DKG by Boneh et al. [12], using our two-party eVRF for publicly-verifiable encryption of shares.

$\mathcal{F}^{\perp}_{\mathsf{KeyGen}}$

1. Let n, t be parameters provided to the functionality.
2. Send (n, t) to the adversary $\mathcal{A}$; receive corr from $\mathcal{A}$.
3. Choose $\gamma \leftarrow\!\!\$\, \mathbb{Z}_p$ and compute $Y \leftarrow g^{\gamma}$.
4. Send Y to the adversary $\mathcal{A}$; receive $(\mathsf{abort}, (\mathsf{sk}_j)_{j \in \mathsf{corr}})$ in return.
5. If $\mathsf{abort} = \perp$, send abort to all honest parties and exit.
6. Otherwise, let f be the random polynomial of degree $t-1$ such that $f(0) = \gamma$ and $f(j) = \mathsf{sk}_j$, for $j \in \mathsf{corr}$.
7. Set $\mathsf{PK} = g^{\gamma}$.
8. Set $\mathsf{sk}_i = f(i)$, for $i \in \mathsf{hon}$.
9. For $i \in [n]$, set $\mathsf{PK}_i = g^{\mathsf{sk}_i}$.
10. Send $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i \in [n]}, \mathsf{sk}_i)$ to each party $i \in [n]$; send $(\mathsf{PK}, \{\mathsf{PK}_i\}_{i \in [n]})$ to the adversary.

Fig. 10: Ideal functionality for key generation with abort, as defined by Katz [54].

key $\mathsf{PK} \in \mathbb{G}_{\mathsf{out}}$ that is uniformly random, or the adversary can force the protocol to abort.

K.2 Proof of Security

We next give the first formal proof of security for BHL24-DKG, the DKG given in Figure 9. While Madrigal [60] argues the security of a close variant of this DKG, the argument by Madrigal [60] requires the use of a non-PPT algorithm A' that solves for the discrete logarithm solution of arbitrary inputs, when given as input the transcript of BHL24-DKG and the state of corrupted parties. Hence, the proof does not establish a reduction to the hardness of the discrete logarithm problem.

We give a proof of security of a protocol $\Pi_{\mathsf{BHL24\text{-}DKG\text{-}PKI}}$, which is the BHL24-DKG DKG conjoined with the initialize phase given in Section 7 (the same initialize phase used to prove the security of Golden).

Theorem 5. *$\Pi_{\mathsf{BHL24\text{-}DKG\text{-}PKI}}$ securely realizes $\mathcal{F}^{\perp}_{\mathsf{KeyGen}}$ in the $(\mathcal{F}^{\mathcal{R}_{EF}}_{\mathsf{zk}}, \mathcal{F}_{\mathsf{T\text{-}eVRF}}, \mathcal{F}_{\mathsf{eVRF}})$-hybrid model, assuming $\mathsf{eVRF}^{\mathsf{BHL24}}$ is a secure exponent verifiable random function, T-eVRF-DH is a secure two-party eVRF, and $\mathcal{A}$ is a static adversary that corrupts up to $t-1$ number of parties.*

Proof. We establish the proof by showing that $\mathsf{Real}_{\Pi_{\mathsf{BHL24\text{-}DKG\text{-}PKI}}, \mathcal{A}}$ is indistinguishable from $\mathsf{Ideal}_{\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}}, \mathcal{S}}$ with respect to $\mathcal{F}^{\perp}_{\mathsf{KeyGen}}$, by a sequence of games.

Without loss of generality, we assume the adversary completes the Initialize step of the protocol correctly, similar to the proof for Theorem 3.

Game 0. This is the indistinguishability game $\mathsf{Real}_{\Pi,\mathcal{A}}$ defined in Section 3.1 instantiated with $\Pi = \Pi_{\mathsf{BHL24\text{-}DKG\text{-}PKI}}$.

Game 1. In Game 1, we make similar modifications as in the proof for Theorem 3, but expand these modifications to include $\mathsf{eVRF}^{\mathsf{BHL24}}$ as well. In particular, we replace each concrete instantiation of T-eVRF-DH and $\mathsf{eVRF}^{\mathsf{BHL24}}$ with $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ and $\mathcal{F}_{\mathsf{eVRF}}$, respectively.

Difference between Game 0 and Game 1. The difference between Game 0 and Game 1 is bounded by the indistinguishability of T-eVRF-DH from $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ and $\mathsf{eVRF}^{\mathsf{BHL24}}$ from $\mathcal{F}_{\mathsf{eVRF}}$, and so is likewise zero.

Game 2. We next describe a simulating algorithm $\mathsf{SIM}^{\dagger}$ that interacts with $\mathcal{A}$ in a black-box manner. $\mathsf{SIM}^{\dagger}$ accepts as input first parameters (n, t), and then a random challenge $Y \leftarrow\!\!\$\ \mathbb{G}$, which $\mathsf{SIM}^{\dagger}$ does not know the discrete logarithm to. $\mathsf{SIM}^{\dagger}$ simulates $\mathcal{F}_{\mathsf{eVRF}}$ and $\mathcal{F}_{\mathsf{zk}}^{\mathcal{R}_{EF}}$. We describe $\mathsf{SIM}^{\dagger}$ in detail next.

Initialize and Participant Initialization. On input (n, t), $\mathsf{SIM}^{\dagger}$ performs the same initialization steps as in the proof for Theorem 3. Then, $\mathsf{SIM}^{\dagger}$ simulates the initialize step for each participant in likewise the same manner.

Simulating $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ Evaluate. $\mathsf{SIM}^{\dagger}$ simulates the $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ evaluate oracle in the same way as in the proof for Theorem 3.

Simulating $\mathcal{F}_{\mathsf{eVRF}}$ Evaluate. $\mathsf{SIM}^{\dagger}$ simulates the $\mathcal{F}_{\mathsf{eVRF}}$ evaluate oracle as defined in Appendix C as follows:

1. For corrupt parties $k \in \mathsf{corr}$, when $\mathcal{A}$ queries Evaluate on input (k, msg), SIM follows the actions of $\mathcal{F}_{\mathsf{eVRF}}$ exactly.
2. When SIM (later in the simulation of Round 1) simulates honest parties' interactions with $\mathcal{F}_{\mathsf{eVRF}}$ for honest parties $j \in \mathsf{hon}, j \neq \tau$, SIM likewise follows the actions of $\mathcal{F}_{\mathsf{eVRF}}$ exactly, with the following exception. Instead of sampling α at random and deriving A, SIM instead sets A to be the particular input (as described below).

Simulating Key Generation. Next, $\mathsf{SIM}^{\dagger}$ must simulate the protocol on behalf of honest parties to $\mathcal{A}$. We show how $\mathsf{SIM}^{\dagger}$ does that next.

1. First, because $\mathsf{SIM}^{\dagger}$ emulates Round 0 for all corrupt parties $\gamma \in \mathsf{corr}$. $\mathsf{SIM}^{\dagger}$ emulates $\mathcal{F}_{\mathsf{eVRF}}$ on inputs $(\mathsf{sk}_{\gamma}^{I}, \mathsf{sid}||\ell))$, and hence learns the associated outputs $(a_{\gamma,\ell}, A_{\gamma,\ell})$, for all $\ell \in \{0, \ldots, t-1\}$,
2. Then, for each honest party $k \in \mathsf{hon}, k \neq \tau$, $\mathsf{SIM}^{\dagger}$ follows the protocol in the same manner as defined in Game 2.
3. However, for the last honest party queried $\tau \in \mathsf{hon}$, $\mathsf{SIM}^{\dagger}$ simulates the protocol as follows:
   (a) Follows the protocol honestly to sample a message $\mathsf{msg}_{\tau} \leftarrow\!\!\$\ \{0,1\}^{\lambda}$.
   (b) Randomly sample corrupt parties' shares $\bar{x}_{\tau,j} \leftarrow\!\!\$\ \mathbb{Z}_p$ for all $j \in \mathsf{corr}$.

(c) $\mathsf{SIM}^{\dagger}$ then simulates the polynomial commitment $\bar{C}_\tau$ to the polynomial f_τ, as follows:

i. Set

$$A_{\tau,0} \leftarrow Y \cdot \prod \gamma \in \mathsf{corr}(g^{a_{\gamma,0}})^{-1} \prod_{k \in \mathsf{hon}, k \neq \tau} (A_{k,0})^{-1} \tag{28}$$

where $A_{k,0}$ are from the honest parties $k \in \mathsf{hon}, k \neq \tau$ polynomial commitments $\bar{C}_k$.

ii. $\mathsf{SIM}^{\dagger}$ computes the Lagrange polynomials $\{L'_0(Z), \{L'_j(Z)\}_{j \in \mathsf{corr}}\}$ for the set (of x-coordinates) $\{0 \cup \mathsf{corr}\}$, as defined in Section 3.2.

iii. For $i \in \{1, \ldots, t-1\}$, SIM then sets the remaining commitments of f_τ "in the exponent," as follows:

$$A_{\tau,i} \leftarrow A_{\tau,0}^{L'_0(i)} \cdot \prod_{j \in \mathsf{corr} \cup \{0\}} (g^{\bar{x}_{\tau,j}})^{L'_j(i)}$$

iv. Set $\bar{C}_\tau = (A_{\tau,0}, \ldots, A_{\tau,t-1})$. Here, $\bar{C}_\tau$ is the commitment to f_τ, defined by the $t-1$ corrupt parties' shares, and Y, such that $g^{f_\tau(0)} = A_{\tau,0}$ as given in Equation 28.

v. Simulate $\mathcal{F}_{\mathsf{eVRF}}$ for each $A_{\tau,i}, i \in \{0, \ldots, t-1\}$ for party τ, as described above.

(d) Set $z_{\tau,j} \leftarrow r_{\tau,j} + \bar{x}_{\tau,j}$.

(e) For $\ell \in \mathsf{hon}$, define $\bar{X}_{\tau,\ell}$ as in Equation 23, to be the commitment to the (unknown) ℓ^{th} honest party's share $\bar{x}_{\tau,\ell}$.

(f) Then, simulate the encryption $z_{\tau,\ell}$ to honest parties' $\ell \in \mathsf{hon}$ shares (which are unknown) by performing the following steps:

i. Sample $z_{\tau,\ell} \leftarrow\!\!\$\ \mathbb{Z}_p$.

ii. Set $R_{\tau,\ell}$ as in Equation 24. Simulate $\mathcal{F}_{\mathsf{T\text{-}eVRF}}$ as described above for $R_{\tau,\ell}$.

(g) For all parties $j \in [n]$, follow the protocol to set $\sigma_{\tau,j} \leftarrow (R_{\tau,j}, z_{\tau,j})$ and bmsg_τ.

4. Output $\{\mathsf{bmsg}_j\}_{j \in \mathsf{hon}}$ for all honest parties.

Finishing the Proof. At the end of $\mathsf{SIM}^{\dagger}$'s simulation, to show indistinguishability from $\mathcal{F}^{\perp}_{\mathsf{KeyGen}}$, we must show that when the protocol does not abort, the public key PK is the challenge Y from $\mathcal{F}^{\perp}_{\mathsf{KeyGen}}$. Furthermore, we must show that $\mathsf{PK}_i = g^{\mathsf{sk}_i}, i \in [n]$, where $\mathsf{sk}_i = f(i)$, such that $g^{f(0)} = \mathsf{PK}$, and $f(j) = \mathsf{sk}_j$, for $j \in \mathsf{corr}$.

Recall that the public key derived by all parties at the end of Round 1 is as follows:

$$\mathsf{PK} \leftarrow \prod_{j=1}^{n} \mathsf{PK}_j$$

Expanding this out, we get:

$$
\begin{aligned}
\mathsf{PK} &= \prod_{j=1}^{n} \mathsf{PK}_j \\
\mathsf{PK} &= \prod_{j=1}^{n} A_{j,0} \\
\mathsf{PK} &= \prod_{j\in[n],j\neq\tau} A_{j,0} \cdot A_{\tau,0} \\
\mathsf{PK} &= \prod_{j\in[n],j\neq\tau} A_{j,0} \cdot Y \cdot \prod_{k\in\mathsf{hon},k\neq\tau} (A_{k,0})^{-1} \cdot \prod_{j\in[n],j\neq\tau} (A_{j,0})^{-1} \quad \textit{// From Equation 28} \\
\mathsf{PK} &= Y
\end{aligned}
\tag{29}
$$

As shown in the relation in Equation 29 between PK, the challenge Y, and corrupt parties' contributions $\{\omega_j\}_{j\in\mathsf{corr}}$, $\mathsf{SIM}^{\dagger}$'s output in Game 3 is indistinguishable from $\mathsf{Ideal}_{\mathcal{F}^{\mathsf{B}}_{\mathsf{KeyGen}},\mathcal{S}}$ with respect to $\mathcal{F}^{\perp}_{\mathsf{KeyGen}}$. This completes the proof. □